

Deep Reinforcement Learning applied to Forex Market: Deep Deterministic Policy Gradient Trading System

Vicente Aser Marques Rebelo Castillo Lorenzo

Thesis to obtain the Master of Science Degree in

Computer Science and Engineering

Supervisor: Prof. Rui Fuentecilla Maia Ferreira Neves

Examination Committee

Chairperson: Prof. Name of the Chairperson
Supervisor: Prof. Rui Fuentecilla Maia Ferreira Neves
Member of the Committee: Prof. Name of First Committee Member

October 2026

Declaration

I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa.

Acknowledgments

Abstract

Most published work on machine learning for currency trading reports what a strategy earned rather than what it would have kept. This dissertation asks whether a reinforcement learning agent can trade the seven major currency pairs profitably once it is charged what a real account charges, and whether the answer holds when one design is applied to every pair without adjustment. Answering it required building the environment first: a trading framework in which a single strategy implementation runs offline, inside the broker platform's own backtester and on a live account, with an execution engine that steps on individual quotes rather than bar closes, reproduces the commercial platform it was built against byte for byte, and charges the spread recorded at the moment of each trade together with the commission of a swap-free raw-spread retail account, drawing on a database of 1.66 billion quotes. Within that environment one Deep Deterministic Policy Gradient (DDPG) agent was trained per pair, observing thirty-eight dimensionless market features and emitting a single continuous action that sets both the direction and the size of a position, which the discrete action spaces used in almost all prior currency work cannot express. Over eleven years all seven agents are profitable, returning between 15.7% and 226.6% net of costs from five different starting balances, and since five of the seven pairs lost value over that period most of this return was earned against falling markets. Six pairs show positive alpha, and on three of them the market exposure, measured as beta against the pair's own movement, is close to zero, while combining the seven into an equal-weight portfolio cuts the maximum drawdown from 34.5% to 15.9%. Two findings generalise: the agents win barely more often than a coin, so the edge lies in sizing rather than prediction, and in every pair the highest-returning candidate failed a permutation test,

meaning that selecting on return alone yields market exposure dressed as skill.

Keywords

Forex Trading; Deep Reinforcement Learning; Deep Deterministic Policy Gradient; Transaction Costs; Portfolio Construction

Resumo

A maioria dos trabalhos publicados sobre aprendizagem automática aplicada à negociação de divisas reporta o que uma estratégia ganhou, e não o que teria conservado. Esta dissertação procura saber se um agente de aprendizagem por reforço consegue negociar com lucro os sete principais pares de divisas quando lhe são cobrados os custos de uma conta real, e se a resposta se mantém quando um único desenho é aplicado a todos os pares sem ajustes. Responder a esta questão exigiu construir primeiro o ambiente: uma plataforma de negociação na qual uma única implementação da estratégia é executada *offline*, dentro do próprio simulador histórico da plataforma da corretora e numa conta real, com um motor de execução que avança cotação a cotação em vez de fecho de barra a fecho de barra, reproduz byte a byte a plataforma comercial contra a qual foi construído, e cobra o *spread* registado no momento de cada operação juntamente com a comissão de uma conta de retalho *raw-spread* sem *swap*, recorrendo a uma base de dados de 1,66 mil milhões de cotações. Nesse ambiente foi treinado um agente Deep Deterministic Policy Gradient (DDPG) por par, que observa trinta e oito variáveis de mercado adimensionais e emite uma única ação contínua que define simultaneamente a direção e a dimensão de uma posição, algo que os espaços de ação discretos usados em quase todo o trabalho anterior sobre divisas não conseguem exprimir. Ao longo de onze anos, os sete agentes são lucrativos, com retornos entre 15,7% e 226,6% líquidos de custos a partir de cinco saldos iniciais diferentes e, como cinco dos sete pares perderam valor nesse período, a maior parte deste retorno foi obtida contra mercados em queda. Seis pares apresentam alfa positivo e, em três deles, a exposição ao mercado, medida como o beta face ao movimento do próprio par, é próxima de zero, enquanto a combinação dos sete numa carteira de pesos iguais reduz a perda máxima de 34,5% para 15,9%. Duas conclusões são gerais: os agentes ganham pouco mais vezes do que uma moeda ao ar, pelo que a vantagem reside no dimensionamento das posições e não na previsão, e, em todos os pares, o candidato com maior retorno falhou um teste de permutação, o que significa que selecionar apenas pelo retorno produz exposição

ao mercado disfarçada de competência.

Palavras Chave

Negociação Forex; Aprendizagem por Reforço Profunda; Deep Deterministic Policy Gradient; Custos de Transação; Construção de Carteiras

Contents

1	Introduction	1
1.1	Motivation	2
1.2	Problem Statement	3
1.3	Objectives	3
1.4	Contributions	4
1.5	Structure	5
2	Background	7
2.1	Financial Markets and Trading	7
2.1.1	Forex Market: Characteristics and Significance	8
2.1.2	Forex Trading: Fundamental Concepts and Operation	8
2.1.3	Forex Costs: Spreads, Commissions and Overnight Financing	10
2.1.4	Financial Analysis: Main Types and Formulas	11
2.1.5	Performance Measurement: Returns, Risk and Ratios	12
2.1.6	Algorithmic and Quantitative Trading	14
2.2	Neural Networks	14
2.2.1	Fundamentals and History	14
2.2.2	Key Concepts and Feed-Forward	15
2.2.3	Learning using Backpropagation	16
2.2.4	Bias-Variance Trade-off and Model Generalisation	17
2.2.5	Architectures for Sequential Data	17
2.2.6	Universal Approximation Theorem	18
2.3	Reinforcement Learning	19
2.3.1	Fundamentals and History	19
2.3.2	Key Concepts and Agent-Environment Interaction	19
2.3.3	Value Functions and Q-Functions	21
2.3.4	Methodological Variants and Learning Methods	22
2.4	Deep Reinforcement Learning	23

2.4.1	Fundamentals and History	23
2.4.2	Deep Value-Based RL Algorithms: Deep Q-Networks	24
2.4.3	Deep Policy-Based RL Algorithms: Deep Policy Gradients	24
3	Related Work	27
3.1	Survey Research: Identifying the State-of-the-Art	27
3.2	Article Research: Summarising Different Implementations	28
3.2.1	Forex Market Implementations	28
3.2.2	Stock Market Implementations	29
3.2.3	Other Market Implementations	30
3.2.4	Comparative Findings	30
3.3	Research Gap: Positioning This Work	31
4	Implementation	35
4.1	Trading Framework: The Environment the Agent Trades In	35
4.1.1	System Architecture: Three Execution Contexts	36
4.1.2	Market Data: Collection, Storage and Resolution	37
4.1.3	Cost Model: Spread, Commission and Account Type	38
4.2	Agent Design: Observation, Action and Reward	40
4.2.1	Observation Space: Indicators, Encoding and Normalisation	40
4.2.2	Action Space: Position Sizing and Risk	41
4.2.3	Reward Function: Log Return, Scaling and Clipping	42
4.2.4	Learning Algorithm: DDPG and its Adaptations	43
4.3	Experimental Procedure: Training, Selection and Deployment	45
4.3.1	Training Procedure: Seeds, Episodes and Data Splits	45
4.3.2	Model Selection: Gates and Composite Ranking	46
4.3.3	Deployment: Decision Frequency and Rebalance Threshold	48
4.3.4	Worked Example: One Position From Decision to Settlement	49
5	Experimental Results	51
5.1	Evaluation Protocol	52
5.2	Benchmarks and Performance Metrics	52
5.3	Results Across the Seven Majors	53
5.3.1	AUD/USD	54
5.3.2	EUR/USD	55
5.3.3	GBP/USD	56
5.3.4	NZD/USD	56
5.3.5	USD/CAD	57

5.3.6	USD/CHF	58
5.3.7	USD/JPY	58
5.4	Trading Behaviour: How the Agents Actually Trade	59
5.4.1	Use of the Continuous Action	59
5.4.2	Direction, Size and Outcome Together	60
5.4.3	Trend Identification and Reversal	61
5.4.4	Holding Periods and Two-Sided Exposure	62
5.4.5	Where the Profit Comes From	63
5.4.6	What the Trading Costs	63
5.5	Path Robustness	65
5.6	Alpha and Beta Decomposition	66
5.7	Statistical Screening and the Rejected Candidates	68
5.8	Out-of-Sample Behaviour: The Held-Out Year	69
5.9	A Negative Result: USD/JPY	71
5.10	Portfolio: The Seven Models Combined	71
5.10.1	Does the Result Depend on the Position Sizes?	73
6	Conclusion	75
6.1	Summary	75
6.2	Limitations	77
6.3	Future Work	77
	Bibliography	79
A	Technical Indicators	87
B	Reviewed Articles	91

List of Figures

2.1	Candlesticks showing upward (bullish) and downward (bearish) movements respectively [1].	10
2.2	Human Neuron compared to an Artificial Neuron [2].	15
2.3	Interaction between the agent and the environment in Reinforcement Learning [3].	20
4.1	Framework architecture. The strategy receives updates and returns actions. It does not know whether the updates come from a live account, from the platform's own backtester or from the stored quote database.	36
4.2	Monthly mean spread in points for the seven major pairs, 2015 to 2026, computed from the full tick tape. EUR/USD is drawn in black. A model evaluated at a fixed spread is evaluated in market conditions that do not occur in any month of this sample.	39
4.3	The actor. Layer normalisation replaces the batch normalisation of the original, so a single observation is processed exactly as a training batch is. The penalty of Equation (4.7) acts on the pre-activation u , before the tangent. This is where saturation would otherwise remove the gradient.	44
4.4	The three stages of the procedure. The Calmar ratio chooses among the episodes of one seed. The composite score of Section 4.3 chooses among seeds.	46
5.1	AUD/USD, EUR/USD and GBP/USD against buying and holding the same pair, rebased to 100. All three benchmarks finish below their starting level.	55
5.2	NZD/USD, USD/CAD and USD/CHF against buying and holding the same pair, rebased to 100. USD/CAD is the only one of the three whose benchmark rises.	57
5.3	USD/JPY against buying and holding the pair. The two lines move closely together. This is what a beta of $+1.012$ looks like, and it is the reason this result is reported as a failure. .	59
5.4	Distribution of the absolute action over the evaluation window. The dashed line marks full size. If the agents had turned the continuous action into a discrete one, these histograms would be concentrated against that line. Instead, most of the mass lies inside the range. .	60

5.5	EUR/USD from 2021 to 2024. From top: the price; the action, positive for long and negative for short; the resulting net position in thousands of units; and account equity. The action changes sign well before the position does, because a change is only acted on when it is larger than the threshold of Equation (4.13).	61
5.6	GBP/USD from mid-July to mid-October 2022. Background shading marks the direction of the net position, red for short and blue for long. The lower panel gives the size of that position. The marked trade is the most profitable single position of the eleven years on this pair.	62
5.7	Left: the share of positions taken long and short. Right: the median holding period in days, on a logarithmic scale. USD/JPY is long in every position it takes and holds for years, which puts it far outside the range of the other six.	63
5.8	Alpha against beta for the seven agents. Points inside the shaded band have almost no market exposure, so their return is attributable to the policy. USD/JPY sits at a beta of one with negative alpha. This is the pattern of a model that has copied its own market. . .	68
5.9	Yearly return of each agent minus the return of buying and holding the same pair, in percentage points. Blue marks outperformance of the benchmark. The solid line marks the end of the training data and the dashed line the end of the validation year, so only the last column was not seen by any part of the procedure.	70
5.10	The equal-weight portfolio against its seven components, all rebased to 100. The portfolio is smoother than any component because the components do not fall together.	72
5.11	Correlation between the daily returns of the seven agents over the evaluation window. The mean of the off-diagonal entries is +0.062.	73

List of Tables

2.1	Summary of Financial Markets and their Economic Role [4].	8
2.2	Summary of the most common Activation Functions [5].	16
2.3	Summary of the most common Loss Functions [5].	16
2.4	Summary of the most common Gradient Descent variants [5].	17
3.1	How this work compares with the eighteen surveyed articles.	31
4.1	Market data collected from cTrader and stored locally, from January 2014 to August 2026. The tick tape takes 295 GB and is the primary record. The bar series point to the ticks that produced them.	37
4.2	Cost and account configuration applied to every evaluation reported in this work.	38
4.3	The sixteen technical indicators that form the market context of the observation. Periods are in hourly bars. Each moving average gives two features, its distance from the price and its own differential, both in units of the average true range. Each momentum reading gives one feature, scaled by the movement expected over its lookahead.	40
4.4	Hyperparameters compared with the original Deep Deterministic Policy Gradient [6]. Three values were changed and five mechanisms were added. The rest are the published values.	43
4.5	Parts of the composite selection score of Equation (4.11). The three groups have equal weight, so return makes up one ninth of the total and does not dominate it. The ratios are those defined in Section 2.1.5.	47
4.6	Effect of the two deployment decisions, measured on one EUR/USD model over the full window with nothing retrained. All four rows use the same weights with different execution settings, so the differences are attributable to the deployment layer alone. This model comes from before the seven-pair campaign and is used here only to isolate the effect of each decision.	48
5.1	Summary of the performance metrics used in this work.	53

5.2	Result for each pair from 2015 to 2026. The return columns give the mean, standard deviation and range across the five starting balances of Section 5.1. Every pair is profitable from all five. Ratios and drawdown are from the reference run. USD/CAD returns the most, and USD/CHF is best on every risk measure.	53
5.3	Each agent against buying and holding its own pair, over the same window and with the same costs. Five of the seven pairs lost value over the eleven years, so on those pairs the return was earned in a falling market. USD/JPY is the only pair whose benchmark rose by a large amount, and the only one the agent did not beat.	54
5.4	Trading behaviour over the evaluation window. Mean $ a $ is the average absolute action, that is, the average fraction of the reference position size held while in the market. Win rate is the share of closed positions with a positive net result after costs.	59
5.5	What the seven agents earned before costs and what they kept, in euros. Cost share is the spread and the commission together, as a fraction of profit before costs. The mean spread column is the commission-weighted average spread paid, in points. It differs from the pair's average spread because it gives weight to the moments when the agent traded.	64
5.6	Total return from each of the five starting balances, in euros. All thirty-five runs are profitable. A broker accepts volume only in discrete steps, so a different opening balance rounds positions differently. It produces a truly different sequence of trades, and not a scaled copy of the same one.	65
5.7	Decomposition of Equation (5.3), from the yearly regression of Equation (5.2) over 2015 to 2025. Alpha is the return attributable to the policy, and the beta return is the part explained by the pair's own movement. The two add up to the mean yearly return.	67
5.8	Candidates rejected by the screening of Section 5.7. On every pair, the candidate with the highest return failed. Ranking on return alone would have chosen these models over the ones published. They show roughly five times the return on paper, but almost none of the measurable skill.	69
5.9	The held-out year compared with the validation year and the full window. No part of the training or selection procedure saw 2025. The year 2024 was used to choose which episode's weights were kept. The last column is the move of the pair itself in 2025.	69
5.10	The equal-weight portfolio of Equation (5.4) against the average of its seven components. Return is the same by construction, so the whole effect is on risk.	72

5.11	The equal-weight portfolio as published, against a version in which every component runs at a common risk level. The common-risk version is built by rescaling each component's per-step returns. This is exact for a strategy whose position size is proportional to balance, but it ignores volume rounding. The two differ little, so the diversification result does not depend on how positions are sized.	74
A.1	Summary of frequently used Trend Technical Indicators [7].	87
A.2	Summary of frequently used Momentum Technical Indicators [7].	88
A.3	Summary of frequently used Volume Technical Indicators [7].	88
A.4	Summary of frequently used Volatility Technical Indicators [7].	89
B.1	Basic features of the related work articles.	92
B.2	Advanced features of the related work articles.	93

List of Algorithms

2.1	Gradient of the Loss Function [5]	17
2.2	Actor-Critic [3]	23
2.3	Deep Deterministic Policy Gradient [6]	25

Acronyms

A2C	Advantage Actor-Critic
A3C	Asynchronous Advantage Actor-Critic
ATR	Average True Range
AUD	Australian Dollar
CAD	Canadian Dollar
CHF	Swiss Franc
DDPG	Deep Deterministic Policy Gradient
DDQN	Double Deep Q-Network
DQN	Deep Q-Network
DRL	Deep Reinforcement Learning
DRQN	Deep Recurrent Q-Network
EMA	Exponential Moving Average
ER	Efficiency Ratio
ETF	Exchange-Traded Fund
EUR	Euro
FNN	Feedforward Neural Network
GBP	British Pound
GDPG	Gated Deterministic Policy Gradient
GDQN	Gated Deep Q-Network

GRU	Gated Recurrent Unit
JPY	Japanese Yen
LSTM	Long Short-Term Memory
MACD	Moving Average Convergence Divergence
MDP	Markov Decision Process
MSE	Mean Squared Error
NN	Neural Network
NZD	New Zealand Dollar
OHLCV	Open, High, Low, Close and Volume
PPO	Proximal Policy Optimisation
ReLU	Rectified Linear Unit
RL	Reinforcement Learning
RNN	Recurrent Neural Network
ROC	Rate of Change
RSI	Relative Strength Index
RV	Realised Volatility
SARSA	State-Action-Reward-State-Action
SD	Standard Deviation
SMA	Simple Moving Average
TD	Temporal Difference
TD3	Twin Delayed Deep Deterministic Policy Gradient
USD	US Dollar
WMA	Weighted Moving Average

Glossary

alpha

The part of a strategy's mean return that its exposure to the market does not explain, taken from the regression of the strategy's returns on the market's returns

ask price

The price at which the market sells, paid to open a long position or to close a short one

backtest

A simulation of a trading strategy on historical prices

balance

The account value from closed positions only, before the result of any open position

base currency

The first currency of a pair, the one being bought or sold; the pair's price is the amount of quote currency one unit of it costs

beta

The sensitivity of a strategy's return to the market's return, the slope of the regression of one on the other

bid price

The price at which the market buys, received when closing a long position or opening a short one

Calmar ratio

The annualised return divided by the maximum drawdown

commission

The fee a broker charges for the volume traded, on opening and on closing a position

currency pair

The price of one currency expressed in another, such as EUR/USD

drawdown

The fall of equity from its previous peak, as a fraction of that peak

equity

The balance plus the unrealised result of the open positions

hedging account

An account that can hold several positions on one instrument, in either direction, at the same time

leverage

The ratio between the size of a position and the account capital that backs it

long position

A position that gains when the price rises

lot

A standard trade size of 100,000 units of the base currency

netting account

An account that holds at most one position per instrument, so an opposite order reduces, closes or reverses it

pip

The fourth decimal place of most currency pair prices, and the second for pairs quoted in Japanese yen

quote currency

The second currency of a pair, in which the price is expressed

Sharpe ratio

The mean excess return divided by the standard deviation of returns

short position

A position that gains when the price falls

Sortino ratio

The mean excess return divided by the standard deviation of the negative returns only

spread

The difference between the ask and the bid prices, paid on every round trip

swap

The interest charged or paid for holding a position past the daily rollover time

tick

A single change in the quoted ask or bid price

walk-forward analysis

An evaluation in which a model is trained on one period and scored on the following one, repeated as the periods move forward in time

1

Introduction

Contents

1.1 Motivation	2
1.2 Problem Statement	3
1.3 Objectives	3
1.4 Contributions	4
1.5 Structure	5

The foreign exchange market is the largest financial market in the world. In April 2025, its average turnover was 9.6 trillion US dollars per day [8]. It trades continuously from Sunday evening to Friday evening. This makes it a good target for automated trading. A program can trade at any hour. The major currency pairs absorb retail order sizes without moving the price. The price history is also available down to single quotes. However, there is little useful signal in the prices. Over short horizons, returns are close to unpredictable [9]. The cost of trading is also high. Many strategies that look profitable in a simplified test lose money once these costs are charged.

This work applies Deep Reinforcement Learning (DRL) to this problem. One Deep Deterministic Policy Gradient (DDPG) [6] agent is trained for each of the seven major currency pairs. Each agent is evaluated over eleven years of tick data. The evaluation charges the spread quoted at the time of the

trade and the commission of a retail account.

1.1 Motivation

Automated currency trading is usually built from fixed rules. Technical indicators are computed from recent prices. A signal is then taken from them, either by comparing them or by a crossover. A separate layer of the system decides the position size [4, 10]. These systems are cheap to run and easy to check, and this is why they are common. Their parameters are set once and do not change, so the system does not adapt when the market changes. This is the first of the two difficulties that this work deals with.

Currencies were chosen over equities for practical reasons [11]. The market trades continuously on weekdays, so a position is never carried through a daily close during which it cannot be traded. There are only seven major pairs. This makes it possible to test a design on all of them, instead of on a sample chosen by the author. A trader can buy or sell with no borrowing cost and no short-selling restrictions. A policy that sells is therefore not penalised by market mechanics that have nothing to do with its signal. Every major pair also has the US dollar on one side. The seven instruments are therefore related, but one cannot replace another. A comparison of the results across them says something about the design and not about one price series.

Reinforcement learning describes the same task without splitting it into a prediction and an action. The agent observes the state of the market and chooses a position. It then receives the profit or loss of that position as its reward [3]. The agent never has to forecast a price. It only has to act. Its action is measured in money, which is also how a trading system is judged in practice.

DDPG fits this task well. It is an actor-critic method for continuous action spaces [6]. The output of the actor can therefore be used directly as a signed position. The sign gives the direction, and the magnitude gives the exposure. Much of the published work on trading uses value-based methods such as the Deep Q-Network [12–14]. These methods need the action to be one of a small discrete set, normally buy, hold and sell. Position sizing is then decided outside the learned policy, by rules that were never trained on the reward.

The second difficulty is cost. It does not depend on the learning algorithm. Results in this field are often reported on bar data, with the spread fixed or ignored and without the broker commission [13, 15]. For a strategy that trades a few times per year, this changes little. For a strategy that changes its position often, it changes a lot. The cost is paid on every change, while the profit per trade stays small. A system that makes money without costs can lose money with them. The test only shows this if it charges what the account would pay [16]. The size of this difference is easy to underestimate. In this work, charging the broker commission on an otherwise identical run removed twenty percentage points of total return from a configuration that changed its position at every decision. Once a threshold was added so that

small changes were skipped, it removed six percentage points. Both figures are reported in Chapter 5.

1.2 Problem Statement

Most published work on deep reinforcement learning for trading uses equity markets, daily bars and a single instrument over a few years [13, 14, 17]. The work that does use currencies normally uses a discrete action space [18–20]. Trading costs are not reported in the same way across studies. Some studies charge both explicit and implicit costs. Others report returns with the spread fixed or left out [15]. This leaves two questions open. The first is whether one design works on several instruments, instead of being fitted to one. The second is whether the reported performance still holds when realistic costs are charged.

This work deals with both questions. The problem is to find out whether a single deep reinforcement learning design produces a profitable and risk-controlled policy on all seven major currency pairs. The design is trained separately for each instrument and is otherwise unchanged. The evaluation runs on tick data. It charges the spread quoted at the time of the trade and the commission of a swap-free raw-spread retail account.

Running one design on seven instruments also protects against a specific failure. A model that is adjusted until it does well on a single pair may only describe the past prices of that pair, and not a strategy. Its results alone cannot tell which of the two it is [21]. Applying the same design without change to every major pair shows the difference, because a result that depends on the instrument will not hold on all of them.

Answering this question needs more than a learning algorithm. It also needs an execution environment that is close enough to a real broker for the answer to mean something. No such environment was available ready to use. Building one took a large part of the effort behind this work. Its design is described in Chapter 4.

1.3 Objectives

The work was organised around five objectives.

- Build a trading framework that can backtest a strategy on tick data with the same costs as the broker. The same strategy must also run inside cTrader, both in its backtesting tab and on a live account. The system that is tested and the system that is deployed are then the same code.
- Implement DDPG in this framework. The observation is built from past prices and technical indicators. The action is continuous and is read as a signed position size.

- Train and evaluate one agent for each of the seven major currency pairs, over the same period and with the same cost model. Any difference between pairs then comes from the instrument and not from the setup.
- Find out whether the performance comes from the policy or from exposure to the movement of the underlying currency pair.
- Combine the seven agents into a single equal-weight portfolio and measure how it behaves. A trader would run the agents together and not one at a time. The behaviour of the combination cannot be read from the seven results taken one by one.

The first objective supports the other three. A result from a simplified simulator cannot be trusted or deployed. The framework was therefore built first, and the learning experiments were run on it.

1.4 Contributions

The contributions of this work are the following.

- **A trading framework where one strategy implementation runs in three places.** The system has two parts. A Python library holds the strategy, the models and the market abstractions. A C# component runs inside cTrader [\[22\]](#) and connects cTrader to the library through shared memory. The same strategy code runs without change in an offline backtest over stored market data, in the backtesting tab of cTrader and on a live account.
- **A market database built from quote-level data.** The historical data was collected from cTrader through the same connector and stored locally. For the seven pairs studied here, it holds about 1.66 billion quotes, which take 295 GB. It also holds 33.0 million bars: 32.5 million at one minute, 547,835 at one hour, 23,386 daily and 1,050 monthly. All of them cover January 2014 to August 2026. For each run, the engine uses the finest series available. The price path inside a bar is therefore rebuilt from the quotes themselves and is not interpolated.
- **A backtesting engine that charges what the account charges.** The engine moves forward tick by tick instead of from one bar close to the next. The spread is the one quoted at the moment of the trade. It is read from the recorded quotes and is not assumed. The commission is computed from the traded volume at the rate of a raw-spread retail account. Stop and target levels are checked against the price path inside the bar instead of at the bar close. This reproduces the full cost structure of a swap-free raw-spread account, which is a type of account that retail brokers really offer. On such an account, the trader pays two charges: the spread and the commission. Both are applied here at the rates that this account pays. To our knowledge, no published work that

applies deep reinforcement learning to currency trading combines quote-level data, a measured spread and a real commission schedule in this way. Table B.1 and Table B.2 summarise what the reviewed studies report. Charging these costs also means that the returns reported in Chapter 5 cannot be compared directly with results obtained on bar closes with a fixed spread or no spread. For the same policy, the returns here are lower than those results would be.

- **DDPG applied to the seven major currency pairs under one protocol.** One agent was trained for each pair, with the same architecture, observation design and reward. Each agent was evaluated from January 2015 to January 2026 with the same cost model. The pairs can therefore be compared with each other. The design is tested for generality and is not tuned to one instrument.
- **Positive and risk-controlled results on all seven pairs.** Over the eleven years, the total return was between +15.7% on USD/JPY and +226.6% on USD/CAD. Every pair stayed profitable when the evaluation was repeated with five different starting balances. The best pair on a risk-adjusted basis was USD/CHF. It had a Sharpe ratio [23] of 0.80, a Sortino ratio [24] of 1.55, a Calmar ratio [25] of 0.49 and a maximum drawdown of 8.2% over 220 trades.
- **An attribution of the returns, including one negative result.** Each model's yearly return is regressed on the yearly movement of its own currency pair. This separates the return that comes from the policy from the return that comes from exposure to the market. Six of the seven pairs had a positive intercept, and three of those had a market sensitivity close to zero. The exception is USD/JPY. Its model follows the underlying pair almost exactly and adds nothing to it. This case is reported here, together with the mechanism that causes it.

1.5 Structure

The rest of this thesis is organised as follows. Chapter 2 gives the background needed to follow the text. It explains how the foreign exchange market works and how technical analysis summarises a price series. It then explains how neural networks and reinforcement learning combine into the methods used here, ending with DDPG. Chapter 3 reviews the published work that applies these methods to trading, grouped by market and by algorithm. It also states which questions the existing work leaves open. Chapter 4 describes the system that was built. This covers the framework and its connection to cTrader, the backtesting engine and its cost model, and the design of the observation, the action and the reward. Chapter 5 first reports the evaluation protocol and the result for each of the seven pairs. It then shows how the agents behave while trading and how much the trading cost. It tests the robustness of the results to the entry path, and it attributes the returns to the policy and to market exposure. It also reports the screening that rejected weaker candidates, what happened in the year held back from training, and

the result of running all seven agents together as a portfolio. Chapter 6 summarises what was achieved, states the limitations of the study and describes the work that follows from it.

This work uses two fields that developed separately. Trading gives the market, the instruments, the costs and the way performance is measured. Reinforcement learning gives the learning algorithm and the way the problem is set up. Both are needed before the system itself can be described. They are covered in the next chapter.

2

Background

Contents

2.1 Financial Markets and Trading	7
2.2 Neural Networks	14
2.3 Reinforcement Learning	19
2.4 Deep Reinforcement Learning	23

This chapter covers the background needed to follow the rest of this thesis. It starts with financial markets, and in particular with the Forex market, how trading works there and what it costs. It then covers neural networks and reinforcement learning, the two fields of machine learning this work uses. It ends with Deep Reinforcement Learning, which joins the two, and with the algorithm used in this work.

2.1 Financial Markets and Trading

Financial markets move capital between those who have it and those who need it. In these markets, participants trade assets, hedge risk and raise funds. Each type of market has a different purpose, and Table 2.1 summarises the main ones. The markets are connected. A movement in one often passes

on to the others, so none of them is studied on its own. The market participants themselves set the prices. They range from private investors to banks and investment funds. Regulators set the rules that these participants must follow. Electronic platforms and algorithmic execution have opened the markets to more participants and have made prices adjust faster [4].

Table 2.1: Summary of Financial Markets and their Economic Role [4].

Market	Economic Role	Asset Examples
Equity Market	Raising capital by trading company shares.	Stocks, Equities
Bond Market	Debt financing, where entities borrow from investors.	Government and Corporate Bonds
Commodities Market	Trading basic raw materials and agricultural goods.	Oil, Gold, Wheat
Forex Market	Currency exchange for international trade.	USD, EUR, JPY, GBP
Derivatives Market	Trading contracts for hedging or speculation.	Futures, Options, Swaps
Indices Market	Combining and tracking the performance of a market or sector.	S&P 500, NASDAQ
Cryptocurrency Market	Exchanging digital currencies.	Bitcoin, Ethereum

2.1.1 Forex Market: Characteristics and Significance

The Foreign Exchange market, or Forex market, is where currencies are exchanged. It is the largest financial market by turnover. In April 2025 it traded 9.6 trillion US dollars per day on average, 28% more than the 7.5 trillion recorded three years earlier. It trades continuously across time zones, because the banks and institutions that quote its prices are spread around the world. Spot transactions, the instrument traded in this work, make up 3 trillion of that daily total. The US Dollar is on one side of 89.2% of all trades, and the ten most traded currency pairs all include it [8]. Prices react to interest rate decisions, economic releases and political events. Participants normally trade with leverage. Leverage allows a position much larger than the capital deposited, and it scales profits and losses in the same proportion. Its market depth, its continuous operation, its sensitivity to news and the routine use of leverage make it suitable for automated trading. The same properties make it hard to model. Its behaviour is noisy and non-stationary, which means that its statistical properties change over time. This is why adaptive, data-driven strategies are of interest [11].

2.1.2 Forex Trading: Fundamental Concepts and Operation

Trading a currency pair means buying one currency and selling another at the same time. Buying the EUR/USD pair means buying Euros and selling US Dollars, expecting that the Euro will strengthen against the US Dollar. This work uses the seven major currency pairs: EUR/USD, USD/JPY, GBP/USD, AUD/USD, USD/CHF, NZD/USD and USD/CAD. They are built from the Euro (EUR), US Dollar (USD), Japanese Yen (JPY), British Pound (GBP), Australian Dollar (AUD), Swiss Franc (CHF) and New Zealand

Dollar (NZD). Every one of them has the US Dollar on one side. Deciding whether to buy or sell a pair comes down to predicting the strength of one currency relative to the other [11].

A position is the exposure a trader holds in a pair. It is the amount of the pair that the trader has bought or sold and not yet closed. A long position makes money when the pair rises and loses money when it falls. A short position does the opposite. A currency pair is a ratio between two currencies, so short selling is as natural as buying and needs no borrowing. In equity markets this is not the case. Positions are opened and closed with orders. A market order is executed at once, at the price quoted at that moment. A limit order is executed only at a given price or better. A stop order is executed once the price passes a given level. Two stop orders are often attached to an open position. A stop loss closes the position when the loss reaches a chosen level. A take profit closes it when a target is reached.

Forex trading takes place in a decentralised market that works worldwide. There is no single exchange where all trades happen. Instead, trades are handled by a network of market makers and brokers. The market makers are mainly banks, and the market between them is known as the interbank market. Interbank trades are large, and most of them are made by institutions such as multinational companies and large hedge funds. Retail traders take part through brokers, who have access to that network and act as intermediaries. Market makers and brokers always quote two prices. The bid price is the price at which they will buy the pair. The ask price is the price at which they will sell it. The difference between the two is the spread. Without a central trading venue, one might expect the market to be poorly regulated. However, competition between market makers keeps their quoted prices close to one another and keeps spreads narrow [11].

Traders look at prices on charts, and the candlestick chart is the standard type. A candlestick chart is made of candlesticks. Each one shows how the price moved over a period of time, which can be anything from minutes to years. A tick is a single update of the quoted price. It is published whenever a market maker changes its bid or its ask. The name comes from its shape, which looks like a candle with a wick at both ends. Each candlestick shows the opening price (O), the highest price (H), the lowest price (L) and the closing price (C) of its period (see Figure 2.1). The colour of the candlestick shows the direction of the move. A lighter shade means the price went up, and a darker shade means it went down. This lets traders see the direction of the market quickly, just by looking at the chart. Candlestick charts do not show volume (V), which is normally drawn separately below the price. Forex has no central venue, so there is no consolidated record of traded size. Brokers report tick volume instead, which is the number of quote updates in the period. This work uses tick volume throughout [26].

This work uses tick data as its main record, because it has the finest resolution the broker provides. Whenever a coarser view is needed, it computes from the ticks the opening, highest, lowest and closing prices and the tick volume, called OHLCV data for short [26]. Indicators computed from that data give the model more context, as described below.

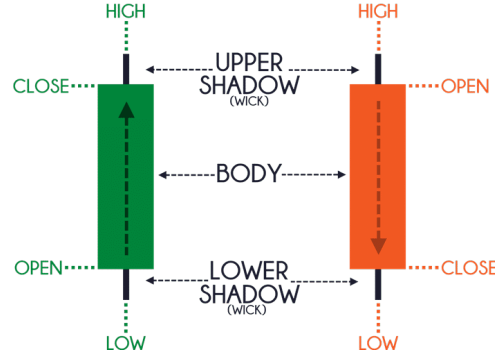


Figure 2.1: Candlesticks showing upward (bullish) and downward (bearish) movements respectively [1].

2.1.3 Forex Costs: Spreads, Commissions and Overnight Financing

Trading is not free, and its costs are large compared with the profit a single position can make. Three costs apply in Forex, and each is charged separately from the others. Prices are quoted in units that depend on the pair. A pip is the fourth decimal place of the quote for most pairs, or the second for pairs quoted against the Japanese Yen. A point is one tenth of a pip. It is the smallest price increment a broker quotes. The size of a position is given in units of the base currency, the first currency of the pair, and one standard lot is 100,000 units. For a position of volume q at price p , the notional value, which is the value of the position in the quote currency (the second currency of the pair), is

$$V = q \cdot p \quad (2.1)$$

The **spread** is the first cost. It is an implicit cost, paid through the price. A position is opened at the ask and closed at the bid, so a full round trip, opening and then closing, crosses the spread once. For a position of volume q , the cost is

$$C_{spread} = (p_{ask} - p_{bid}) \cdot q \quad (2.2)$$

The spread is not constant. It widens when liquidity falls, around economic releases and at the daily rollover. A strategy tested with a fixed spread is therefore tested on a market that does not exist in practice.

The **commission** is the second cost, and it is an explicit cost. Some accounts quote the raw inter-bank spread. On these accounts the broker makes its money through a commission charged on the traded volume, instead of through a wider spread. If the commission is given as a number of points c , and s_{point} is the size of one point, the cost of a position of volume q is

$$C_{commission} = c \cdot s_{point} \cdot q \quad (2.3)$$

The spread and the commission are separate costs, and a trader pays both. On a raw-spread account, treating the spread as if it already included the commission makes trading look cheaper than it is.

The **swap**, or overnight financing, is the third cost. It comes from the interest rate differential between the two currencies of the pair. It is charged when a position is still open at the daily rollover, the moment when the trading day ends. On one day of the week the charge is multiplied, to cover the weekend. Depending on the direction of the position and the sign of the differential, the swap can be a cost or a credit. Some account types remove it completely, so that clients can avoid interest for religious reasons. These are known as swap-free accounts. Finally, leverage decides how much capital a position needs. With leverage L , the margin, which is the capital set aside for a position of notional value V , is

$$M = \frac{V}{L} \quad (2.4)$$

Leverage does not change the profit or loss of a position. It only changes the capital set aside for it. So leverage affects the return of the account, and the return of the trade stays the same. These costs are not the same on every type of account. Choosing the account is therefore part of designing a strategy. A raw-spread account quotes the interbank spread with no markup and makes its money through the commission. This makes the total cost of a position lower. It also makes the cost explicit, which is more useful. The implicit part of the cost is reduced to the market spread, and the rest is a published rate. A swap-free account charges no overnight financing at all. An account with both features suits a strategy that holds positions for days. Such a strategy pays the spread and the commission only a few times per position, but it would otherwise pay financing for every night the position stays open. This work is evaluated on an account that has both features. The spread and the commission are therefore the only costs applied, and the financing cost is exactly zero, so it does not need to be estimated.

2.1.4 Financial Analysis: Main Types and Formulas

Representing the data is one half of the problem. The other half is analysing it to decide what to trade. Financial analysis is usually divided into three types [27]:

- **Fundamental Analysis:** studies the economic factors that affect currency prices. These include GDP growth, employment rates, interest rates and monetary policy, which help to predict currency trends from the economic health of a country. However, much of this data is qualitative. This makes it hard to use in computer models, which work best with structured numerical input.
- **Sentiment Analysis:** tries to measure the mood of the market by analysing news, market commentary and similar qualitative data. It aims to measure how the shared emotions of traders affect

market trends. It is complex and needs advanced natural language processing, so it is harder to include in a model.

- **Technical Analysis:** analyses past OHLCV data to predict future market movements. It assumes that past market behaviour will repeat itself, so patterns or measures found in current data can predict future movements. It is preferred because its results are numbers, which complex machine learning models can use.

Of the three, this work uses technical analysis. It produces numerical series directly from price data, so a model can use them without any further interpretation. Technical analysis is divided into indicators and candlestick patterns. Indicators are measures calculated from past market data, created mainly by mathematicians and economists. They give a numerical view of how the market moves. They are usually grouped into four main types [27]:

- **Trend Indicators:** Used to find the main direction of the market by filtering out market noise. See Table A.1 for details of frequently used indicators.
- **Momentum Indicators:** Used to measure the momentum of a trend, whether it is strong or weak, and whether a reversal is expected. See Table A.2 for details of frequently used indicators.
- **Volume Indicators:** Used to look at the trading volume, which gives clues about the strength of a price movement. See Table A.3 for details of frequently used indicators.
- **Volatility Indicators:** Used to measure the level of uncertainty or risk that comes with the volatility of the price. See Table A.4 for details of frequently used indicators.

The four tables list indicators in common use. They do not describe the contents of any one system. The framework built for this work implements the moving average family and the crossovers between its members, the Rate of Change, the Moving Average Convergence Divergence, the Average True Range, the Realised Volatility and the Efficiency Ratio. It implements no volume indicators. The tick volume a Forex broker reports counts quote updates, not traded size, and this limits what those indicators can measure. Ten instances of the implemented set form the observation the agent receives. They are listed in Chapter 4. Candlestick patterns work differently. They do not compute a value. They describe shapes, formed by one or more candlesticks, that traders link to a change in behaviour [26]. Such patterns can be encoded as numbers and given to a model in the same way as an indicator [7]. This work does not use them. The observation is built from indicators only.

2.1.5 Performance Measurement: Returns, Risk and Ratios

A trading strategy is judged by what it earns and by the risk it takes to earn it. A single return figure is therefore not enough to compare two strategies. The measures below are the ones reported in

Chapter 5. They are defined on an equity series, which is the value of the account over time. They therefore apply without change to a portfolio of several strategies. They are then computed on the combined equity of the portfolio, and not on each strategy separately. This matters because the risk of a portfolio depends on how its parts move together, and not only on the risk of each part.

The total return over a period compares the final and the initial equity E of the account:

$$R = \frac{E_{final} - E_{initial}}{E_{initial}} \quad (2.5)$$

Periods differ in length, so returns are annualised, that is, converted to a yearly rate, before they are compared. With a duration Δt in seconds and a year of 365 days,

$$R_{annual} = (1 + R)^{\frac{365 \cdot 86400}{\Delta t}} - 1 \quad (2.6)$$

Risk is measured in two ways. The first is volatility, which is the standard deviation of returns, annualised by the rate at which the strategy trades. The second is drawdown. At any moment, the drawdown is the distance between the current equity and the highest equity reached so far:

$$D(t) = \frac{E(t) - \max_{s \leq t} E(s)}{\max_{s \leq t} E(s)} \quad (2.7)$$

The maximum drawdown is the worst value of $D(t)$ over the period. It is the largest loss from a peak that an investor would have suffered. The mean drawdown is the average of $D(t)$. It shows how far below its peak the account is on a typical day, instead of at its worst moment. Four ratios combine return and risk. All of them divide an annualised excess return, which is the return above the risk-free rate, by a measure of risk. They differ only in the measure of risk they use. The absolute value of that measure is used, because a drawdown is negative under the definition above. With R_f as the risk-free rate,

$$\text{Ratio} = \frac{R_{annual} - R_f}{|\text{risk}|} \quad (2.8)$$

The **Sharpe ratio** [23] uses the annualised volatility, so it penalises upward and downward moves equally. The **Sortino ratio** [24] uses the downside volatility instead, computed from negative returns only. The argument is that an investor does not mind upside variability. The **Calmar ratio** uses the maximum drawdown, so it measures return against the worst loss suffered. The **Sterling ratio** uses the mean drawdown. This makes it less sensitive to a single bad period, and closer to what holding the strategy is like on a typical day. The Calmar and Sterling ratios both have several definitions in the literature, and the ones used here follow [25]. By construction, the mean drawdown is smaller than the maximum drawdown. So, for a strategy with a positive excess return, the Sterling ratio is the larger of the two ratios. When the excess return is negative, the order is reversed.

2.1.6 Algorithmic and Quantitative Trading

More and more of the trading in financial markets is done by algorithms. Algorithmic trading uses computer programs to execute trades faster and in larger volume than a human can. The programs analyse large datasets, read market conditions and act on set rules. Algorithmic trading is used for pre-trade analysis, signal generation and trade execution. It is what allows institutions to run complex strategies at a large scale. Quantitative trading is a subfield of algorithmic trading. It uses mathematical and statistical models to find opportunities in market data, through the analysis of past data and through machine learning. It follows systematic rules instead of the discretionary judgement of a trader, and it needs knowledge of both finance and mathematics [10]. Together, the two have changed how markets work. They brought practices such as high-frequency trading, and they shortened the time that prices take to react to new information. They have also made machine learning central to the field. Neural networks are used to find structure in market data, and reinforcement learning is used to decide what to do with it [7]. These two fields are the subject of the next sections.

2.2 Neural Networks

Machine learning covers algorithms that find patterns in data and act on them, without being programmed with the rules in advance. Tom Mitchell states the idea as follows [28]:

"A computer program is said to learn from experience E with respect to some task T and some performance measure P , if its performance on T , as measured by P , improves with experience E ."

The definition is useful here because it describes exactly what a trading agent does. The task is choosing a position. The performance measure is the money that position makes. The experience is the market data the agent has already seen. Neural networks, the subject of this section, are the class of models used to represent that behaviour.

2.2.1 Fundamentals and History

Neural Networks (NN) are algorithms designed for pattern recognition. They are named after the human brain, because their structure is similar to it. In the brain, a neuron receives signals through structures called dendrites and processes them in the nucleus. It then passes the result down its axon to the dendrites of nearby neurons. A neural network works in a similar way. Each node is an artificial neuron. It receives an input, processes it with a mathematical function and passes the output to the next layer of nodes. In both cases, the strength and the type of the connections between neurons decide how the network behaves [5]. Figure 2.2 illustrates these ideas.

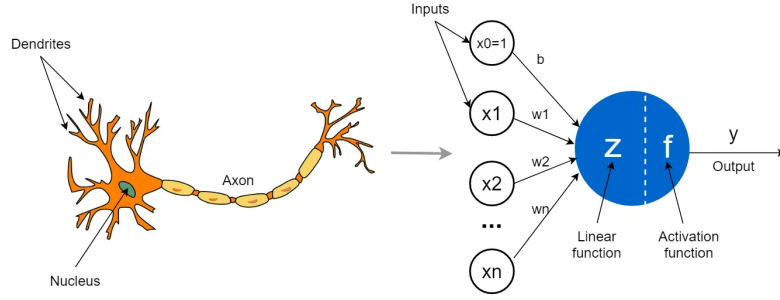


Figure 2.2: Human Neuron compared to an Artificial Neuron [2].

The idea of neural networks goes back to the 1950s and the perceptron, a simple model of a biological neuron developed by Frank Rosenblatt [29]. The perceptron was the start of NN research. However, it could only learn linearly separable datasets, in which a straight line or a flat plane can separate the classes. This limit led to some periods of low interest in the field, known as “AI winters”. A major advance came in the 1980s with the backpropagation algorithm. It allowed complex multilayer neural networks to learn patterns in datasets that are not linearly separable [30].

2.2.2 Key Concepts and Feed-Forward

In a neural network, the feed-forward computation in each layer l uses the output of the layer before it. For the first layer, this input x is the raw data itself. For the later layers, it is the output of the previous layer, $h^{(l-1)}(x)$. The computation in each layer can be written as [5]:

$$z^{(l)}(x) = W^{(l)}h^{(l-1)}(x) + b^{(l)} \quad (2.9)$$

Here, the weight matrix $W^{(l)} \in \mathbb{R}^{K_l \times K_{l-1}}$ and the bias vector $b^{(l)} \in \mathbb{R}^{K_l}$ are used in the computation of layer l . K_l is the number of neurons, or units, in layer l , and K_{l-1} is the number of neurons in the previous layer $l-1$. For the first layer, $W^{(1)} \in \mathbb{R}^{K_1 \times D}$, where D is the size of the input data. The activation function g is then applied to the pre-activation $z^{(l)}(x) \in \mathbb{R}^{K_l}$ to give the activated output $h^{(l)}(x) \in \mathbb{R}^{K_l}$ [5]:

$$h^{(l)}(x) = g(z^{(l)}(x)) \quad (2.10)$$

The choice of activation function matters in two places. In the hidden layers, the function must be non-linear. A stack of linear layers is equal to a single linear transformation, so the network would gain nothing from its depth. In the output layer, the task decides the function. A bounded function is used when the output must stay within a range, and no activation is used when it does not [5]. Table 2.2 summarises the most common activation functions.

Table 2.2: Summary of the most common Activation Functions [5].

Activation Function	Formula	Description
Linear	$a(x) = x$	Suitable for regression tasks.
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	Maps inputs to (0, 1), for binary classification.
Hyperbolic Tangent	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	Maps inputs to (-1, 1), with stronger gradients than sigmoid.
ReLU	$\text{ReLU}(x) = \max(0, x)$	Leads to sparse activation, helps gradient flow.
Softmax	$\text{Softmax}(z)_i = \frac{e^{z_i}}{\sum_{k=1}^K e^{z_k}}$	Turns raw scores (logits) into probabilities, for multi-class classification.

2.2.3 Learning using Backpropagation

Neural networks learn by adjusting their weights and biases step by step, to reduce the error between the predicted outputs and the true target values. A loss function measures this error and guides the learning. The goal is to find the parameters that minimise the loss function. Different problems need different loss functions, such as mean squared error for regression tasks and cross-entropy for classification [5]. Table 2.3 gives their formulas.

Table 2.3: Summary of the most common Loss Functions [5].

Loss Function	Formula	Application
Mean Squared Error (MSE)	$\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$	Regression
Negative Log-Likelihood (Cross-Entropy)	$-\sum_{i=1}^n y_i \log(\hat{y}_i)$	Classification

The main algorithm used to train neural networks is called backpropagation [30]. It computes the gradient of the loss function with respect to the weights and biases of the network. The gradient points in the direction of steepest increase of the loss. Algorithm 2.1 shows how these gradients are calculated. Gradient descent then moves the weights and biases in the opposite direction, which reduces the loss. Its variants, listed with their formulas in Table 2.4, suit different data and computing needs. Batch gradient descent uses the whole dataset for each update. It is stable but expensive on large datasets. Stochastic gradient descent updates the parameters after every data point. It is faster but noisier. Mini-batch gradient descent updates on subsets of the data. It is the compromise used in practice [5].

In Table 2.4, the learning rate η is a hyperparameter that controls the size of each update. A rate that is too large overshoots the minimum. A rate that is too small converges slowly and can get stuck in a local minimum. Some methods adapt η during training. They change the step size as the optimisation goes on, and this deals with both problems. Normalising or standardising the input data has the same aim, because features on very different scales produce gradients on very different scales [5].

Algorithm 2.1: Gradient of the Loss Function [5].

Require: f : network function, θ : network parameters, x : input data, y : true label, l : number of layers, L : loss function

Compute the output of the network $\hat{y} = f(x; \theta)$

Compute the gradient of the loss function with respect to the output layer before activation:

$$\nabla_{z^{(l+1)}} L(f(x; \theta), y) = \frac{\partial L(f(x; \theta), y)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{(l+1)}}$$

for $\ell = l + 1$ **to** 1 **do**

 Compute gradients of hidden layer parameters:

$$\nabla_{W^{(\ell)}} L(f(x; \theta), y) = \nabla_{z^{(\ell)}} L(f(x; \theta), y) \cdot h^{(\ell-1)T}$$

$$\nabla_{b^{(\ell)}} L(f(x; \theta), y) = \nabla_{z^{(\ell)}} L(f(x; \theta), y)$$

 Compute gradient of previous hidden layer:

$$\nabla_{h^{(\ell-1)}} L(f(x; \theta), y) = W^{(\ell)T} \cdot \nabla_{z^{(\ell)}} L(f(x; \theta), y)$$

 Compute gradient of previous hidden layer (before activation):

$$\nabla_{z^{(\ell-1)}} L(f(x; \theta), y) = \nabla_{h^{(\ell-1)}} L(f(x; \theta), y) \odot g'(z^{(\ell-1)})$$

Table 2.4: Summary of the most common Gradient Descent variants [5].

Algorithm	Formula	Description
Batch Gradient Descent	$\theta := \theta - \eta \cdot \nabla_{\theta} J(\theta; X, y)$	Updates the parameters using all the data
Mini-Batch Gradient Descent	$\theta := \theta - \eta \cdot \nabla_{\theta} J(\theta; X^{(i:i+n)}, y^{(i:i+n)})$	Updates the parameters using subsets of the data
Stochastic Gradient Descent	$\theta := \theta - \eta \cdot \nabla_{\theta} J(\theta; x^{(i)}, y^{(i)})$	Updates the parameters after each data point

2.2.4 Bias-Variance Trade-off and Model Generalisation

How well a neural network generalises, that is, how well it works on data it has not seen, depends on a balance between bias and variance. Bias leads to underfitting. The model makes assumptions that are too simple and misses the real trends in the data. Variance leads to overfitting. The model is too complex and treats noise as if it were a signal. Both give poor results on new data. This matters in Forex trading, because the signal being sought is weak and the noise around it is large. Cross-validation is often used to measure how well a model generalises, on separate training, validation and testing datasets. Regularisation methods such as L1 and L2 limit the complexity of a model by adding penalties to the loss function. Dimensionality reduction methods such as principal component analysis reduce the number of inputs the model has to fit. Together, these methods make it more likely that the model performs well on out-of-sample data, which is data not used in training [5]. This work divides the period into training, validation and testing windows, described in Chapter 4. The performance on the validation window decides which model is kept. Complexity is not controlled through a penalty added to the loss.

2.2.5 Architectures for Sequential Data

The network described so far is feed-forward. The whole input is given at once, and the output depends only on that input. Financial data is a sequence, so an obvious question is whether a network that reads

a sequence directly would do better.

Two families of networks do that. A recurrent neural network keeps an internal state that is carried from one step to the next, so each observation is read together with the ones before it. The long short-term memory network is the variant used in practice. It adds gates that control what the state keeps and what it drops. This stops the gradient from vanishing, that is, from shrinking towards zero, as it is propagated back over many steps [5]. Attention-based networks reach the same goal in a different way. They let the network weigh every position in a window directly, instead of passing information along a chain.

Much of the work reviewed in Chapter 3 uses recurrent networks, and several of those studies report that they helped. This work does not use them, and this was a deliberate design decision. Recurrence is one way to give a network access to the past. Another way is to compute measures of the past explicitly and give them to the network as part of the input. The observation described in Chapter 4 uses the second way. It includes momentum and moving-average readings over horizons from one day to one year. The history that a recurrent state would have to learn to summarise is therefore measured directly and given to the network.

This choice has three consequences. First, a feed-forward network on engineered features has far fewer parameters than a recurrent network on raw prices. This matters when the signal is weak and the sample is one instrument over eleven years. Second, it is faster to train. Training speed decided how many independent runs were possible, and this turned out to matter more for the quality of the final result than any gain from a different architecture would have. Third, it keeps the input interpretable. Each feature is a named quantity with a known meaning, so when a policy behaves strangely, the observation can be inspected instead of guessed at.

This choice also has a cost. A recurrent network can, in principle, find a summary of the past that no fixed set of indicators contains. This work gives up that possibility. Whether it would have helped here has not been tested, and it is listed among the extensions in Section 6.3.

2.2.6 Universal Approximation Theorem

The universal approximation theorem, proved by Hornik, Stinchcombe and White in 1989, can be stated as follows [31]:

A neural network with one hidden layer and a linear output can approximate any continuous function arbitrarily well, given enough hidden units.

The theorem shows that neural networks are universal function approximators, and this is what justifies their use here. It means that a large enough network can represent the function that maps market observations to a trading decision. It does not say whether such a function can be found from data, or

whether it exists in a form stable enough to be useful. Those questions are the subject of the reinforcement learning methods covered next.

2.3 Reinforcement Learning

Reinforcement Learning (RL) is the branch of machine learning that studies an agent that interacts with an environment and learns which decisions to make. Unlike supervised learning, it is not given the correct answer for any situation. The agent acts, observes what happens and changes its behaviour according to the reward it receives. Learning therefore happens by trial and error, without labelled examples. This suits problems where the right action is not known in advance but its outcome can be measured. Robotics and finance are two fields where this is the case.

2.3.1 Fundamentals and History

Reinforcement Learning grew out of the combination of several other fields, such as psychology, computer science and control theory. It was inspired by the way humans and animals learn from their environment. In RL, an agent makes decisions and receives rewards or penalties for its actions, with the aim of maximising good outcomes over time. This process is based on operant conditioning, a type of learning studied by psychologists such as B. F. Skinner [32], in which behaviour is shaped by its consequences. Markov Decision Processes (MDPs) were formalised in the late 1950s. They model decision making where the outcomes are partly random and partly under the control of the decision maker. MDPs became the mathematical basis of RL, because they allowed the development of algorithms that learn optimal strategies in complex environments. One of the major advances in RL was Q-Learning, introduced in the late 1980s. It is an algorithm that learns the value of actions in different states, so an agent can make informed decisions without a model of the environment. Q-Learning and Temporal Difference Learning made it possible to apply RL to a wide range of problems, from playing video games better than humans to optimising financial trading strategies [3].

2.3.2 Key Concepts and Agent-Environment Interaction

Reinforcement Learning is about how an agent interacts with its environment. The interaction involves states, actions and rewards. The state S_t describes the situation of the agent at that moment. When the agent observes S_t , it chooses an action A_t . This action makes the environment transition to a new state S_{t+1} , and the environment gives feedback in the form of a reward R_{t+1} . Figure 2.3 shows this repeating cycle [3].

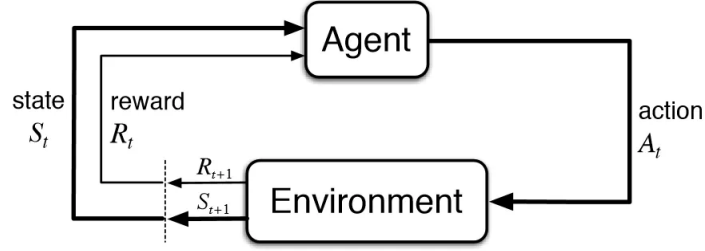


Figure 2.3: Interaction between the agent and the environment in Reinforcement Learning [3].

The goal of the agent is to maximise the return. The return is the discounted sum of the rewards the agent collects from time t onwards [3]:

$$R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (2.11)$$

Here, γ is the discount factor. It sets how much future rewards count compared with immediate ones. A policy π decides the actions of the agent. It can be deterministic, giving one specific action for each state, or stochastic, giving a probability distribution over actions. A main difficulty in RL is the balance between exploration and exploitation. Exploration means trying new actions to find out their effects. Exploitation means choosing actions that are known to give a high reward. A common approach is the ϵ -greedy policy, which usually picks the best-known action but sometimes picks a random one [3]. This method needs a discrete set of actions, because it works by picking one of the alternatives at random. When the action is continuous, as in this work, exploration comes instead from adding noise to the action the policy returns. Section 2.4 describes this approach together with the algorithm that uses it.

Reinforcement Learning is built on the Markov Decision Process (MDP), defined by the tuple (S, A, P, R, γ) . Here, S is the set of all possible states of the environment and A is the set of actions the agent can take. P is the transition probability, the probability of moving from state s_t to state s_{t+1} after taking action a . R is the reward function, which gives the reward for that transition, and γ is the discount factor. MDPs rely on the Markov property. It requires the next state to depend only on the current state and action, and not on any earlier ones [3]:

$$P(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = P(s_{t+1}|s_t, a_t) \quad (2.12)$$

The reward function matters because it is where the goal of the task is defined. It tells the agent what to achieve, but not how to achieve it. The agent will adopt any behaviour that raises the reward. In trading, the reward is normally computed from profit and loss. This drives the agent towards profitable positions without telling it which positions those are [3].

2.3.3 Value Functions and Q-Functions

The value function of a policy π , written $V^\pi(s)$, is the expected return when starting in state s and following policy π after that. It is defined as follows [3]:

$$V^\pi(s) = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, \pi \right] \quad (2.13)$$

Here, γ is the discount factor, which sets how much future rewards count compared with immediate rewards. $\mathbb{E}$ is the expected value, the average of all possible outcomes weighted by their probabilities. The Q-function of a policy π , written $Q^\pi(s, a)$, is the expected return for taking action a in state s and following policy π after that. It is defined as follows [3]:

$$Q^\pi(s, a) = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a, \pi \right] \quad (2.14)$$

The Bellman equations are named after Richard Bellman, who first formulated them. They give a recursive decomposition of the total cumulative reward an agent can expect, starting from a given state or state-action pair. These equations are the basis of dynamic programming methods in RL. They express the value of a state, or of a state-action pair, in terms of the values of the states that follow it. This recursion is what allows the estimated values to be improved step by step, and it is the basis of convergence for most RL algorithms. The equations are as follows [3]:

$$V^\pi(s) = \sum_{a \in A} \pi(a|s) \sum_{s' \in S} P(s'|s, a) [R(s, a, s') + \gamma V^\pi(s')] \quad (2.15)$$

$$Q^\pi(s, a) = \sum_{s' \in S} P(s'|s, a) \left[R(s, a, s') + \gamma \sum_{a' \in A} \pi(a'|s') Q^\pi(s', a') \right] \quad (2.16)$$

$$V^\pi(s) = \sum_{a \in A} \pi(a|s) Q^\pi(s, a) \quad (2.17)$$

Equation (2.17) shows that the value of a state under policy π is the expected value of the Q-function over all possible actions, weighted by the probability the policy gives to each action. For optimal policies, the optimal value function $V^*(s)$ and the optimal Q-function $Q^*(s, a)$ give the highest return that can be achieved from any state or state-action pair, respectively. From the Bellman equation, it follows that [3]:

$$V^*(s) = \max_{a \in A} \sum_{s' \in S} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')] \quad (2.18)$$

$$Q^*(s, a) = \sum_{s' \in S} P(s'|s, a) \left[R(s, a, s') + \gamma \max_{a' \in A} Q^*(s', a') \right] \quad (2.19)$$

$$V^*(s) = \max_{a \in A} Q^*(s, a) \quad (2.20)$$

By applying these equations again and again, RL algorithms gradually approximate the value function and the Q-function. This guides the agent towards the optimal policy, which is the policy that maximises the expected cumulative reward [3].

2.3.4 Methodological Variants and Learning Methods

Reinforcement learning methods can be divided into groups by the properties of the learning method: model-based or model-free, on-policy or off-policy, and value-based or policy-based. In model-based RL, the algorithm builds a model of the environment to simulate and predict the outcomes of actions. The agent can then plan by predicting the next states. This is possible when the environment is simple enough to be modelled. In model-free RL, the algorithm learns a policy or a value function directly from its interactions with the environment, without building a model of it. Model-free methods are often chosen when the environment is hard to model or when no accurate model is available. Within model-free RL, there are on-policy methods, which learn about the policy that is currently used to make decisions, and off-policy methods, which explore with a different policy while they try to learn an optimal one [3].

Another important distinction is between value-based methods, also called critic-only, and policy-based methods, also called actor-only. Value-based methods, such as Temporal Difference Learning (TD), Q-Learning and SARSA Learning, estimate the value of states or of state-action pairs. The policy is not learned directly. It follows from these values. Temporal Difference learning is the update rule that the other two are built on, and it works in both settings. SARSA is on-policy, because it evaluates the policy it is following. Q-Learning is off-policy, because it evaluates the greedy policy, the one that always takes the highest-valued action, whatever policy it actually follows. To take the highest-valued action, the method has to compare every action, so these methods are limited to discrete sets of actions. Policy-based methods, such as REINFORCE, optimise the policy directly. For this reason they extend to continuous actions, where the policy returns the action itself instead of a value for each possible action. Actor-Critic learning combines the two. It has a policy function (the actor) and a value function (the critic). The policy is improved directly, and the critic gives the value estimates that make those updates stable [3]. Algorithm 2.2 gives a detailed implementation.

For finite Markov Decision Processes, standard results guarantee that an optimal policy exists and that it can be taken to be deterministic and stationary. These guarantees do not extend to complex, non-stationary environments such as financial markets [3]. A raw price series does not have the Markov property. The information a trader uses is spread across the recent history, and is not all contained in the latest quote. RL applied to these markets therefore works with an approximation. Technical

Algorithm 2.2: Actor-Critic [3].

Require: $\pi(a | s; \theta)$: a differentiable policy, $Q(s, a; w)$: a differentiable Q -function, N : number of iterations and learning rates $\beta^\theta > 0$ and $\beta^w > 0$

Initialise policy parameter $\theta^{(0)}$ and Q -function parameter $w^{(0)}$

Initialise state s

for $n = 0, 1, \dots, N - 1$ **do**

 Sample $a \sim \pi(\cdot | s; \theta^{(n)})$

 Take action a , observe state s' and reward r

 Sample $a' \sim \pi(\cdot | s'; \theta^{(n)})$

 Compute the TD error: $\delta \leftarrow r + \gamma Q(s', a'; w^{(n)}) - Q(s, a; w^{(n)})$

 Update w : $w^{(n+1)} \leftarrow w^{(n)} + \beta^w \delta \nabla_w Q(s, a; w^{(n)})$

 Update θ : $\theta^{(n+1)} \leftarrow \theta^{(n)} + \beta^\theta \delta \nabla_\theta \ln \pi(a | s; \theta^{(n)})$

$s \leftarrow s'$

indicators and other engineered features put enough of that history into the state for the Markov property to hold approximately. Actor-Critic methods matter here because they are the point where reinforcement learning and neural networks meet. This is the subject of the next section.

2.4 Deep Reinforcement Learning

Deep Reinforcement Learning (DRL) combines the decision making of reinforcement learning with the function approximation of neural networks. The value functions and policies of Section 2.3 are defined over states. In a tabular setting, each state is stored separately, as one entry in a table. This is not possible when the state is a vector of continuous market variables, because there is no limit to the number of distinct states. Replacing the table with a neural network removes that limit. It also lets the agent generalise from states it has seen to states it has not seen.

2.4.1 Fundamentals and History

DRL appeared once it became clear that neural networks could serve as the function approximators that reinforcement learning needed to handle high-dimensional input. The result that started the field was the Deep Q-Network. It learned to play a range of Atari 2600 games directly from raw pixels and reached human-level scores on many of them [12, 33]. It showed that a single architecture could learn control policies from raw, high-dimensional input, without features designed for each task. This result led to interest in applying the same methods to financial markets. There, the input is also high-dimensional, and the mapping from observation to decision is also unknown. However, an Atari game is stationary and fully observable, and a market is neither. For this reason, results carry over to markets much less directly than the comparison with games suggests.

2.4.2 Deep Value-Based RL Algorithms: Deep Q-Networks

Deep value-based methods use a neural network to approximate the Q-function. This lets them work with state spaces too large to list one by one. Neural Fitted Q Iteration was an early method of this kind. It fitted a network to the Q-values of a stored set of transitions, and fitted it again as more transitions were collected [34]. The approach became established with the Deep Q-Network (DQN) algorithm [12], which added two mechanisms that made training stable enough to be practical. The first is a separate target network. This is a delayed copy of the value network, used to compute the learning targets, so that the target does not move as fast as the estimate. The second is experience replay. This is a buffer of past transitions from which mini-batches are drawn at random, which breaks the temporal correlation between consecutive samples. The algorithm used in this work uses both mechanisms.

After DQN, the Double Deep Q-Network (DDQN) dealt with the tendency of DQN to overestimate action values [35]. It separates action selection from action evaluation. The online network, the one being trained, chooses the action, and the target network scores it. Deep value-based methods are not used in this work. They select an action by comparing the values of all available actions, and this requires the set of actions to be discrete.

2.4.3 Deep Policy-Based RL Algorithms: Deep Policy Gradients

Deep policy-based algorithms optimise the policy directly, without deriving it from action values. This is what allows them to work with continuous actions. In trading this matters, because a position has a size as well as a direction. A method limited to a discrete set of actions has to decide that size outside the learned policy. The Deep Deterministic Policy Gradient (DDPG) algorithm is a model-free, off-policy, actor-critic method for continuous action spaces [6]. It keeps four networks. The actor $\mu(s|\theta^\mu)$ maps a state to a single action. The critic $Q(s, a|\theta^Q)$ estimates the value of a state-action pair. The other two are delayed copies of these, the target actor μ' and the target critic Q' . Transitions are stored in a replay buffer, and mini-batches of N transitions are sampled from it at random. The critic is trained in the same way as in DQN. It minimises the squared error between its estimate and a target built from the observed reward and the value of the next state [6]:

$$y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'}) | \theta^{Q'}) \quad (2.21)$$

$$L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2 \quad (2.22)$$

The actor is updated with the deterministic policy gradient, which follows the direction in which the value estimated by the critic increases. Because the action is continuous, the gradient of the value with

respect to the action is defined, and the chain rule carries it back to the parameters of the actor [6]:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a | \theta^Q) \big|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s | \theta^\mu) \big|_{s_i} \quad (2.23)$$

The target networks are not simply copied from the trained networks. After every update, they are moved a small fraction $\tau \ll 1$ towards them. This keeps the learning targets almost stationary, but it also slows down how fast they follow improvements [6]:

$$\theta' \leftarrow \tau \theta + (1 - \tau) \theta' \quad (2.24)$$

The policy is deterministic, so it does not explore by itself, and the exploration problem raised in Section 2.3 comes back here. DDPG solves it by adding a noise process $\mathcal{N}$ to the action during training, so the action actually executed is $\mu(s | \theta^\mu) + \mathcal{N}$. The original work uses an Ornstein-Uhlenbeck process. This process gives noise that is correlated in time, instead of independent at each step. The reason given is that a physical system with momentum explores better when a disturbance lasts for a while than when it changes sign at every step [6]. Algorithm 2.3 shows the complete procedure.

Algorithm 2.3: Deep Deterministic Policy Gradient [6].

Require: an actor $\mu(s | \theta^\mu)$, a critic $Q(s, a | \theta^Q)$, a discount factor γ , a soft update rate $\tau \ll 1$ and a minibatch size N

Randomly initialise the critic parameters θ^Q and the actor parameters θ^μ

Initialise the target parameters $\theta^{Q'} \leftarrow \theta^Q$ and $\theta^{\mu'} \leftarrow \theta^\mu$, and the replay buffer $\mathcal{B}$

for episode = 1, M **do**

Initialise a random process $\mathcal{N}$ for action exploration

Receive the initial observation state s_1

for $t = 1, T$ **do**

Select the action $a_t = \mu(s_t | \theta^\mu) + \mathcal{N}_t$ from the current policy and the exploration noise

Execute the action a_t and observe the reward r_t and the new state s_{t+1}

Store the transition (s_t, a_t, r_t, s_{t+1}) in $\mathcal{B}$

Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from $\mathcal{B}$

Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1} | \theta^{\mu'}) | \theta^{Q'})$

Update the critic by minimising the loss: $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i | \theta^Q))^2$

Update the actor policy using the sampled policy gradient:

$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a | \theta^Q) \big|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s | \theta^\mu) \big|_{s_i}$

Update the target networks:

$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$

$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$

DDPG is powerful, but it is not reliable. It has no guarantee of convergence except under restrictive conditions. It is also known to be sensitive to its hyperparameters and to the random seed, so two runs of the same configuration can end with different policies. A market makes this worse. It is non-stationary and only partially observable, which means the agent does not see the full state, and the guarantees

that apply to finite Markov Decision Processes do not hold. These limits belong to the method itself. They are not problems that can be fixed. Chapter 5 therefore reports how the results vary, instead of reporting a single run.

This work combines the four topics covered here. It needs a market to trade in, networks to approximate the functions involved, a reinforcement learning formulation of the trading decision, and an algorithm that solves it for continuous actions. Combining them is not new. Many published studies have already tried it.

Those studies differ in how they define the state, the action and the reward. They also differ in whether the agent pays any cost for trading. These choices decide whether a reported result would hold on a live account. This makes them the right basis for comparing the published studies and for placing this work among them.

3

Related Work

Contents

3.1 Survey Research: Identifying the State-of-the-Art	27
3.2 Article Research: Summarising Different Implementations	28
3.3 Research Gap: Positioning This Work	31

This chapter reviews the published work on deep reinforcement learning for financial trading. The review had two stages. First, surveys were read to find which problems the field still sees as open and which papers are worth reading in full. Then single articles were read, and their design choices were recorded in one common format. This way, studies that use different markets and different algorithms could be compared on the same points.

Eighteen articles were read in this way. Table B.1 lists their basic features, and Table B.2 lists the design choices inside each agent. The text below refers to both tables.

3.1 Survey Research: Identifying the State-of-the-Art

The review started with surveys. A survey shows the main lines of a field faster than single papers do, and it points back to the work that first showed each result.

Hambly et al. [14] gave the theoretical basis. The survey first explains reinforcement learning and neural networks on their own. It then covers how the two combine into deep reinforcement learning, and how this has been applied in finance. It gives the mathematical formulations and pseudocode for critic-only, actor-only and actor-critic algorithms, together with the papers that introduced each of them. It is the reference that was used to write Chapter 2.

Fischer [13] had the opposite purpose. It is about practice, not theory, and it reviews implementations, not derivations. This work took from it the list of dimensions it uses to compare studies: the dataset, the network under the reinforcement learning agent, the state space, the action space, the reward function, the methods used for the bias-variance and exploration-exploitation trade-offs, and the training procedure. These dimensions became the columns of Table B.1 and Table B.2. In this way the articles below could all be compared on the same points, and not only described one at a time.

Three more surveys were also read [15, 17, 36]. They agree on the general picture. Most work in the field uses equity markets, daily data and discrete trading decisions. Reported results are hard to compare, because the trading environment is different in each paper.

3.2 Article Research: Summarising Different Implementations

Next, eighteen articles with working implementations were read in full. Their features were recorded in two tables that cover the same articles. Table B.1 holds the market, the data and the algorithm used. Table B.2 holds the design choices inside each agent. Below, the articles are grouped by the market they trade.

3.2.1 Forex Market Implementations

Three of the eighteen articles trade currencies. All three limit the agent to a discrete choice between buying, holding and selling.

Carapuço et al. [18] built a Deep Q-Network over a fully connected network and used it on raw tick data from 2010 to 2017. The agent could act only after a fixed number of ticks had passed. This gave about one decision every two hours. It outperformed its benchmark by a wide margin. It is also the only study reviewed here that works from tick data and not from bars. Huang [19] used a Deep Recurrent Q-Network on 15-minute bars from 2012 to 2017. This model puts a long short-term memory network under the Q-function. It outperformed its benchmark with explicit and implicit trading costs charged. This makes it one of the few results in the set that still hold when costs are modelled. Rundo [20] used the same kind of model on one-minute bars from 2004 to 2018, and also reported that it outperformed its benchmark. However, its accuracy was high enough to suggest look-ahead bias, and its maximum

drawdown was large. Taken together, the two results suggest that adding recurrence is not enough on its own to make very short timeframes workable.

3.2.2 Stock Market Implementations

Most of the work reviewed trades in equity markets. Among the critic-only studies, Sim and Kirk [37] trained a Deep Q-Network on daily bars from 2006 to 2022. They tested it on assets it had not seen, and reported that the policy generalised to them. Théate and Ernst [38] used the same algorithm on daily bars from 2012 to 2019. They reported consistent results in both calm and volatile periods. Their agent explored through an inverted copy of the environment, instead of through a noise process. Li et al. [39] compared two agents on minute bars of stocks and futures from 2010 to 2020. The first was a Deep Q-Network over a denoising autoencoder. The second was Asynchronous Advantage Actor Critic over a long short-term memory network. Both beat the benchmark. The asynchronous actor-critic converged better, and the authors attribute this to its parallel training.

The actor-critic studies are closer to the design used in this work. Xiong et al. [40] applied DDPG to daily bars from 2009 to 2018 and outperformed the benchmark. Yousefi [41] compared DDPG with Advantage Actor Critic on daily bars from 2009 to 2021. DDPG was the more stable of the two, and the study attributes this to its replay buffer and its noise process. Gran et al. [42] ran DDPG on monthly bars from 2005 to 2019. It still outperformed the benchmark after explicit and implicit costs were charged. Azhikodan et al. [43] added news sentiment to a state space that already held price data and technical indicators. They reported that this improved the results. However, the evaluation covers about five months, and the exact period is not given. Majidi et al. [44] used the Twin Delayed variant of DDPG on daily bars from 2010 to 2021, on both stocks and cryptocurrencies, with a continuous action in $[-1, 1]$. It beat the cryptocurrency benchmark but not the equity benchmark.

Sagiraju and Mogalla [45] need a separate mention. They compared four algorithms on daily bars from 1992 to 2020. All four beat the benchmark. Proximal Policy Optimisation and the Deep Q-Network were the more consistent, and DDPG and Advantage Actor Critic did worse than expected. It is the only study in the set that reports DDPG doing worse than the other algorithms, so it is a useful contrast to the studies above.

Two more studies combine algorithms instead of choosing one of them. Yang et al. [46] ran DDPG, Advantage Actor Critic and Proximal Policy Optimisation together on daily bars from 2009 to 2020. They switched between the three based on a turbulence measure. The ensemble beat the benchmark with explicit costs charged. Kong and So [47] combined the same three algorithms on daily bars from 2016 to 2020. Their ensemble had unstable returns and underperformed the benchmark. Together, the two results indicate that combining agents does not by itself make the results robust. Wu et al. [48] sit between the two groups. They compared a gated recurrent Q-network with a gated recurrent policy-

gradient agent on daily bars from 2008 to 2018, and the policy-gradient version did better.

3.2.3 Other Market Implementations

Three articles trade instruments other than equities and currencies. Chen and Gao [49] traded exchange-traded funds on daily bars from 2000 to 2018. They compared a Deep Q-Network over a fully connected network with its recurrent version, and the recurrent model did better. Zhang et al. [50] applied Advantage Actor Critic over a long short-term memory network to futures on daily bars from 2005 to 2019. It still outperformed the benchmark after costs. Jia et al. [51] also traded exchange-traded funds, on daily bars from 2002 to 2020, with DDPG over a long short-term memory network. They tested two versions. One held a fixed position size, and the other set the position size continuously. The version with a variable size did better, with explicit and implicit costs charged. This is the closest study in the set to the design used in this work. It is the only one that uses a continuous action to size a position and also charges for trading.

3.2.4 Comparative Findings

Reading the two tables column by column, instead of row by row, shows the choices that most studies share.

The state space is usually price data plus technical indicators computed from it. Studies that report strong results almost always include indicators, and not price alone [18, 19, 46]. This agrees with the argument in Section 2.3. A raw price series is not Markovian, and indicators put enough of the recent history into the state for the property to hold approximately.

The action space is discrete in 15 of the 18 studies, usually with three choices: buy, hold and sell [20, 40, 43]. Three studies use a continuous action [44, 50, 51]. Only one of them uses it to size a position, and not to give a direction [51]. A discrete action set is easier to learn. However, it takes position sizing out of the policy, so the size has to be set by a rule that the agent cannot adapt. The reward is the total return in 14 of the 18 studies. Three use the Sharpe ratio [45, 47, 51] and one uses the log return [44]. A risk-adjusted reward costs more to compute during training, because it needs a window of past returns and not only the latest one. The studies that use it do not report a consistent gain from it.

For the bias-variance trade-off, the usual methods are feature engineering, normalisation, regularisation and changes to the network itself. Typical examples are denoising [39], and normalisation together with regularisation [38]. For exploration, the ϵ -greedy policy is standard when the action set is discrete, and additive Gaussian noise when it is continuous [44, 48].

Trading costs are not recorded in the same way across studies. Five of the 18 studies state that their headline result still holds with explicit and implicit costs [19, 42, 46, 50, 51]. The others report returns

without saying what was deducted, so the reader has to guess whether the figure is gross or net. Training usually uses a train-validation-test split, sometimes with cross-validation. One study trains online and keeps learning as new data arrives [19]. Of all the studies, this one comes closest to the conditions of live deployment.

The data covers long periods, but at a coarse resolution. Twelve of the 18 studies cover ten years or more, and the median span is eleven years. So a long history is normal in this field. The studies are also alike in resolution, which is low: 17 of the 18 work on bars, 13 of those on daily bars, and only one uses quote-level data [18]. This matters for how costs are counted. A bar reports four prices and hides the order in which the price reached them.

3.3 Research Gap: Positioning This Work

The eighteen studies agree that a deep reinforcement learning agent can learn a profitable trading policy. They differ, or say nothing, on the conditions in which this holds. The findings above describe what the field does. When they are read against one another, they also show what the field has not done yet. Table 3.1 sets out the comparison.

Table 3.1: How this work compares with the eighteen surveyed articles.

Dimension	Surveyed literature (18 articles)	This work
Market	Stocks 12, Forex 3, ETFs 2, Futures 1	Forex, the seven major pairs
Data resolution	Bars in 17 of 18, daily in 13; one study uses quotes	Quote-level tick data, with hourly and daily decisions
Data span	Ten years or more in 12 of 18, median eleven years	Eleven years, 2015 to 2026
Action space	Discrete 15, continuous 3; one sizes the position	Continuous, sets direction and size together
Reward function	Total return 14, Sharpe ratio 3, log return 1	Log return; models selected by the Calmar ratio and a risk-adjusted composite score
Trading costs	Charged in 5 of 18	Quoted spread at the moment of the trade plus the commission of a swap-free raw-spread account
Instruments per design	One agent per instrument, or one agent on a basket	One design trained separately on seven pairs, then combined
Reported robustness	The candidate count and the sensitivity to starting conditions are not stated	Five starting balances per pair, with the candidate count stated

The first gap appears only when two of those dimensions are read together. The three studies that trade currencies [18–20] all limit the agent to a discrete choice. The three that give the agent a continuous action [44, 50, 51] all trade something else. In the work reviewed here the two properties never appear together. So there is no direct comparison for a currency agent that sets its own position size. This work uses that combination, and for this reason it cannot be measured against an existing result.

The second gap comes from the cost figures. A strategy tested without costs is not tested against the market it claims to trade. In Forex this omission matters more than the reported returns suggest, because the spread is not fixed. It widens when liquidity falls and around releases of economic data. These are the moments when a model trained on an average spread would expect to act. The third gap is about resolution, not length of history. A long sample is already normal in this literature, so it does not set a study apart, but working inside the bar does. A bar reports four prices and hides the order in which the price reached them. That order decides which spread was paid, and whether a stop was reached before a target.

The fourth gap is about evidence, not design. Each study reports the performance of a model it selected. None of the studies reviewed here states how many candidates that model was selected from. None states how the reported figure changes when the evaluation is repeated from a different starting condition. Deep reinforcement learning is sensitive to initialisation. So a single figure cannot separate what comes from the method and what comes from the particular run. A model chosen as the best of many also tells as much about the sample it was chosen from as about the market it traded [16, 21]. The last gap is about how results are combined. The studies reviewed report either one agent trading one instrument, or one agent trading a basket. None of them trains one design separately on each of several instruments and then measures what those agents do when they run together. That is how a trader who holds several currency pairs would deploy them. The individual results do not show what the combination does, because that depends on how the equity curves move against each other.

This work is placed in these gaps. One design is applied without change to all seven major currency pairs. It uses a continuous action that sets the direction and the size of the position together. The evaluation runs on quote-level data over eleven years. It charges the spread quoted at the moment of each trade, plus the commission of a swap-free raw-spread retail account. This is an account that a retail trader can open, and not a simplification made for modelling. Each pair is reported from five different starting balances, not from a single backtest, and the number of candidates the selected model was drawn from is stated. The seven agents are then combined into an equal-weight portfolio. So this work reports both the seven single results and the one result that a trader running all seven would have seen. Chapter 5 gives both.

There is also one point where this work is the same as the literature. The reward is the log return. One of the surveyed studies makes the same choice [44], so this is not a point of difference. The difference is that the reward the agent optimises and the criteria used to select the final model are, on purpose, not the same measure. During training, candidate runs are scored by the Calmar ratio. The model finally published for each pair is the one ranked highest by a composite score. This score is built from percentile ranks within the pair's own pool of candidates. It uses the annualised return, the Sharpe, Sortino, Calmar and Sterling ratios defined in Section 2.1.5, the maximum and downside risk,

and measures of how the policy behaves. So a policy that earns its return by taking more risk does not win on return alone. Chapter 4 sets out both.

The studies reviewed here answer the question of whether a deep reinforcement learning agent can learn a profitable trading policy. They say much less about the conditions in which that answer holds. Each study usually describes the environment its agent was trained in with a sentence or two, and that description is rarely enough to build the environment again.

This work had to build that environment before it could ask the question. An agent trained on a simplified market learns to exploit the simplification. So the value of any result depends on how the environment charges for trading, moves time forward and fills orders. Chapter 4 describes that environment and the agent that runs in it.

4

Implementation

Contents

4.1 Trading Framework: The Environment the Agent Trades In	35
4.2 Agent Design: Observation, Action and Reward	40
4.3 Experimental Procedure: Training, Selection and Deployment	45

Chapter 3 showed what the reviewed literature leaves open. This chapter describes what was built to address it. It covers the environment the agent trades in, the agent itself, and the procedure used to train the agent and to choose one model for each currency pair.

The sections follow this order on purpose. The agent cannot be described without its environment, because the environment gives the agent its observations and charges it for its actions. No number the agent produces means anything unless that environment is accurate. So the framework comes first, then the agent, and then the experimental procedure.

4.1 Trading Framework: The Environment the Agent Trades In

No existing framework could both train an agent offline on quote-level history and run the same agent on a live account without rewriting it. Building one took a large part of the effort in this work. Its design

choices decide how far the results in Chapter 5 can be trusted. Three properties were required. First, the tested system and the deployed system had to be the same code, so that a result obtained offline says something about live behaviour. Second, the data had to have a fine enough resolution to rebuild the price path inside a bar, because that path decides what a position actually paid. Third, the costs had to be the costs of a real account, not simplified values chosen for convenience.

4.1.1 System Architecture: Three Execution Contexts

The framework has to be written in two languages. The strategy logic, the learning models and the market abstractions are written in Python, which has the numerical and machine learning libraries. Access to the broker is written in C#, because cTrader [22] runs algorithmic strategies as cBots on the .NET runtime and offers no other way to reach its order book. The two parts communicate through named memory-mapped files with paired auto-reset event handles, one pair for each direction. They work as a single-slot request and response exchange. The platform writes an update and signals. Python reads it, writes an action and signals back, and then the platform continues. Two kinds of message cross this boundary. Updates go from the platform to Python, and actions go the other way. Figure 4.1 shows the arrangement. The important result of this design is that the strategy object does not know where its updates come from. The same class, with the same weights, runs in three contexts.

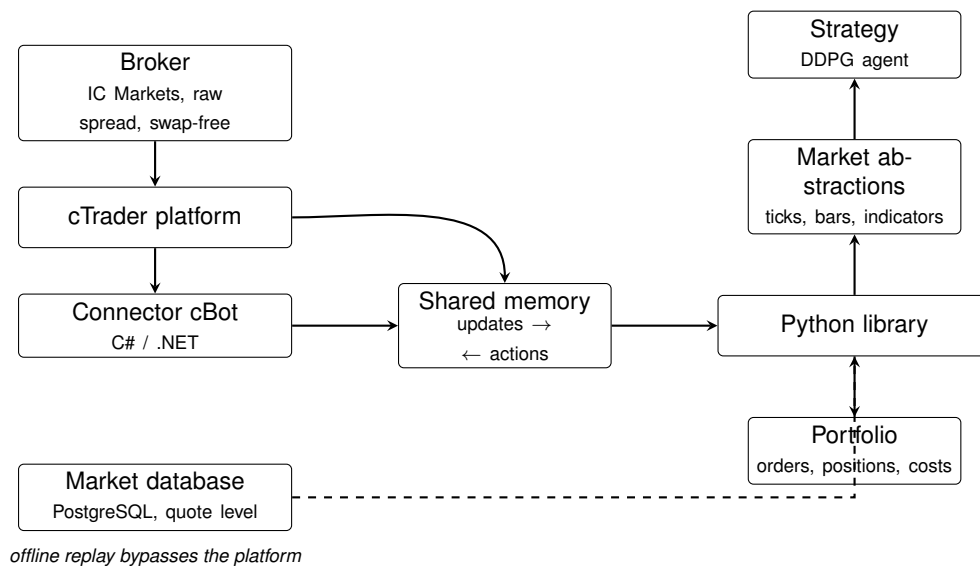


Figure 4.1: Framework architecture. The strategy receives updates and returns actions. It does not know whether the updates come from a live account, from the platform’s own backtester or from the stored quote database.

In the first context the platform is not used. Python replays stored quotes directly from the market database. This is the fastest of the three contexts, and it is the one used for training and for every result reported in this work. In the second context the strategy runs inside cTrader’s own backtesting tab on the

platform's historical data. This gives an independent check of the offline engine. In the third context the same strategy runs on a live account. It receives real quotes and sends real orders. The benefit of this design is that the three contexts differ only in where the updates come from. When a strategy behaves one way in a backtest and another way when deployed, the usual cause is two separate implementations that have diverged over time. Here there is only one implementation, so this problem does not occur.

4.1.2 Market Data: Collection, Storage and Resolution

The historical data was collected through the same connector. The C# component requested it from cTrader, and the Python component wrote it to a local PostgreSQL database. Two types of record are stored. A tick holds the bid and ask quoted at one instant, together with the conversion rates needed to express a profit or loss in the account currency at that instant. A bar holds its open, high, low and close as references to the ticks that produced them. A bar is therefore a view of the tick tape, not a separate copy of the prices. Table 4.1 lists the data held for the seven pairs. The tick tape is the primary record, and the bar series are derived from it. This is what allows the engine to rebuild the path a price took inside a bar, instead of interpolating between its end points.

Table 4.1: Market data collected from cTrader and stored locally, from January 2014 to August 2026. The tick tape takes 295 GB and is the primary record. The bar series point to the ticks that produced them.

Pair	Ticks	M1 bars	H1 bars	D1 bars	MN1 bars
AUD/USD	227,914,614	4,643,990	78,320	3,346	150
EUR/USD	242,847,954	4,636,138	78,145	3,339	150
GBP/USD	281,490,557	4,648,216	78,311	3,339	150
NZD/USD	203,788,307	4,631,614	78,282	3,339	150
USD/CAD	228,356,007	4,635,179	78,319	3,347	150
USD/CHF	194,186,979	4,621,503	78,307	3,338	150
USD/JPY	284,430,458	4,638,775	78,151	3,338	150
Total	1,663,014,876	32,455,415	547,835	23,386	1,050

Two properties of this data must be stated, because otherwise both would look like defects.

Bars are stamped at the close of the broker's trading session, not at midnight. This is at 22:00 UTC, or at 21:00 under daylight saving. A daily series therefore has five bars in a normal week, stamped Sunday to Thursday, and the bar stamped Thursday holds Friday's session. An audit against a calendar-day reference reports every Friday as missing. This is a result of the stamping convention, and no data is lost. The period from 11 to 25 January 2016 is missing from all seven pairs, both in ticks and in bars. The gap comes from the broker. A new download of those days returned nothing, which confirmed that the gap cannot be repaired. The gap is the same for all seven pairs, so it does not bias any comparison between them. It also falls outside the evaluation window used in Chapter 5.

4.1.3 Cost Model: Spread, Commission and Account Type

The execution engine advances on ticks, not on bar closes. A position opened during a bar is filled at a quote that existed within that bar. A stop or target attached to it is tested against the quotes in the order they arrived. This matters for the reason given in Chapter 3. A bar reports four prices but hides the order in which the price reached them. That order decides whether a stop was reached before a target, and which spread was crossed.

The account modelled is a swap-free raw-spread retail account. Each of the three charges described in Section 2.1.5 is applied at the rate that such an account pays. A buy is filled at the ask and a sell at the bid, both read from the tick tape at the instant of the fill. The spread is therefore charged by construction, as part of every fill, and is not deducted afterwards. The commission is charged on traded volume at 3.5 points per side. For a standard lot of 100,000 units this is 3.5 units of the quote currency per side, or 7 per round turn. The swap is zero. On a swap-free account this is the rate the account really charges, so it is not an approximation. Table 4.2 lists the full configuration.

Table 4.2: Cost and account configuration applied to every evaluation reported in this work.

Setting	Value	Meaning
Account type	Raw spread, swap-free	An account that a retail client can open, with the interbank spread and no charge for overnight financing
Spread	Accurate	The bid-ask difference recorded at the moment of the trade, read from the tick tape and not assumed
Commission	3.5 points per side	Charged on traded volume. This is 3.5 units of the quote currency per standard lot per side, or 7 per round turn
Swap	0 points	The rate a swap-free account charges, so not an approximation
Account currency	EUR	Profit and loss converted at the rate recorded on the tick that produced it
Initial balance	10,000	Reported across five starting balances, as described in Chapter 5
Leverage	30	The retail limit for major currency pairs
Position mode	Netting	One net position per instrument, not separate hedged positions

The choice to read the spread from the ticks, instead of assuming a value for it, should be supported with data. Assuming a value is common, and the error it causes is not small. Figure 4.2 plots the monthly mean spread of each pair across the evaluation window. For EUR/USD, the most liquid pair in the market, the monthly mean goes from 0.77 to 13.00 points. This is a factor of seventeen between the quietest month and the busiest one. USD/CHF reaches 37.82 points in January 2015, when the Swiss National Bank removed its floor against the Euro. For each of the seven pairs, the spread in the widest month is at least five times the spread in the narrowest month.

The pattern of this variation matters more than its size. Spreads widen when liquidity falls and around the events that move prices. These are the moments when a model trained on an average spread is most likely to act. A strategy charged a constant spread is therefore charged the wrong amount, and the error is correlated with its own trading activity. The error also goes in the direction that makes the

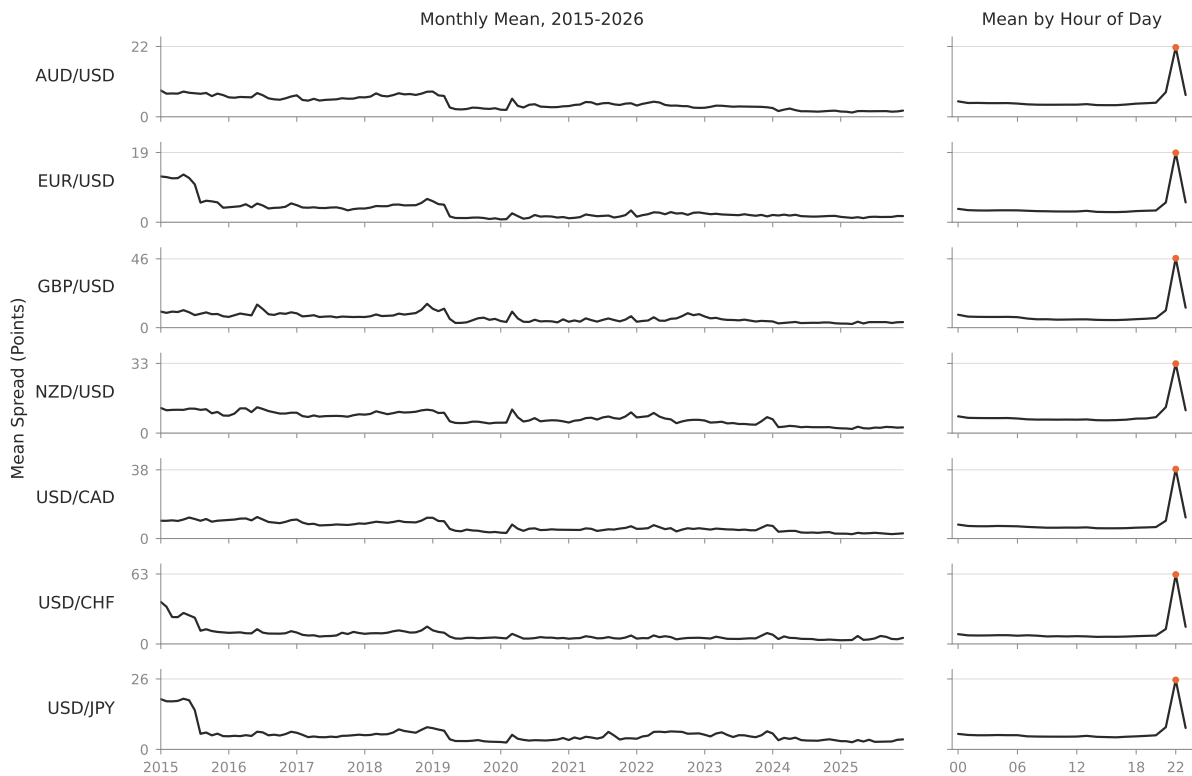


Figure 4.2: Monthly mean spread in points for the seven major pairs, 2015 to 2026, computed from the full tick tape. EUR/USD is drawn in black. A model evaluated at a fixed spread is evaluated in market conditions that do not occur in any month of this sample.

strategy look better.

The engine was validated against cTrader's own backtester, not against itself. Six reference configurations were used, covering two currency pairs and three windows. In all of them, the engine's trade, position, order and deal records reproduce the platform's output byte for byte. The conversion of profit and loss into the account currency was also checked separately against the platform's own figures. The only divergence is sub-pip. It comes from the resolution of the stored quotes, not from the accounting. The engine therefore reproduces the platform it was built against, and it can also do more than that platform. The same strategy is evaluated offline directly on the tick tape. This runs at a speed that the platform's own backtester cannot reach, and over windows that the backtester will not cover. The cost model can be set to any account the broker offers, not only to the account the terminal happens to be connected to.

Two consequences follow. Both are stated here so that Chapter 5 can be read correctly. First, the returns reported in this work are net of the spread actually quoted and of a real commission schedule. They are therefore lower than the same policy would show on bar closes with a fixed spread or no spread, and they cannot be compared directly with results obtained in that way. Second, because the

account is swap-free, no financing cost is estimated or left out. A strategy that holds positions over many nights is charged exactly what such an account would charge it.

4.2 Agent Design: Observation, Action and Reward

The environment of Section 4.1 supplies prices and charges for trades. Three more decisions are needed to turn it into a reinforcement learning problem: what the agent sees, what it is allowed to do, and what it is rewarded for. This section states these three decisions and then describes the algorithm that learns from them. Each decision is given with the reason for it. In every case the alternative was tried and worked worse, and those failures are part of the result.

4.2.1 Observation Space: Indicators, Encoding and Normalisation

The agent observes a vector of 38 values at each hourly bar. None of these values is a raw price. A price level carries no information that the agent can generalise from. A policy learned at a price of 1.05 has no reason to work at 1.20, and in eleven years of history the price moves by more than that. Every feature is therefore a ratio, and the whole observation is dimensionless. Sixteen technical indicators, listed in Table 4.3, give the market context. They are split into a fast group, which describes the last hours to days, and a slow group, which describes regimes that last months. This lets the agent tell a move within a trend apart from a change of trend.

Table 4.3: The sixteen technical indicators that form the market context of the observation. Periods are in hourly bars. Each moving average gives two features, its distance from the price and its own differential, both in units of the average true range. Each momentum reading gives one feature, scaled by the movement expected over its lookback.

Indicator	Family	Period	Role in the observation
ATR	Volatility	14	Volatility unit; divides every trend feature and sets the stop distance
RV (fast)	Volatility	16	Short-horizon realised volatility; divides the momentum features
RV (slow)	Volatility	480	Long-horizon realised volatility; its ratio to the fast reading gives the volatility regime
ER	Trend	120	Efficiency ratio; how directional recent movement has been, bounded in $[0, 1]$
ROC Fast	Momentum	24	One day
ROC Medium	Momentum	120	One week
ROC Slow	Momentum	480	One month
ROC Regime	Momentum	1,440	One quarter
ROC Epoch	Momentum	2,880	Two quarters
SMA Fast	Trend	24	One day
SMA Medium	Trend	120	One week
SMA Slow	Trend	480	One month
SMA Regime	Trend	1,440	One quarter
SMA Epoch	Trend	2,880	Two quarters
SMA Cycle	Trend	4,320	Three quarters
SMA Era	Trend	5,040	Roughly one year

The indicator values are encoded as follows. Here C is the bar close, σ_f and σ_s are the fast and slow

realised volatilities, A is the average true range, and M_i is the i -th moving average, with period N_i :

$$\frac{A}{C}, \quad \sigma_f, \quad \ln\left(\frac{\sigma_f}{\sigma_s}\right), \quad \text{ER} \quad (4.1)$$

$$\frac{\text{ROC}(N_j)}{\sigma_f \sqrt{N_j}}, \quad \frac{C - M_i}{A}, \quad \frac{M_i - M_i^{-1}}{A} \quad (4.2)$$

Each formula removes a different dependence. Dividing the average true range by the close gives volatility as a fraction of the price. The logarithm of the ratio of fast to slow volatility shows the volatility regime. It is zero when the two agree, positive when volatility is rising and negative when it is falling. Dividing a momentum reading by $\sigma_f \sqrt{N_j}$ scales it by the movement expected over that lookback under a random walk. A value near one then means a move of normal size, whatever the period. The distance from a moving average, measured in units of the average true range, says how far the price is from its trend compared with the current volatility. The differential of the same quantity gives the direction in which the trend is moving. The other features are the position in the calendar, encoded as four sine and cosine pairs, the current signed exposure as a fraction of the largest position the account could hold, and six values that describe the last bar as log returns of its open, high, low and close against the previous close, divided by σ_f .

Features that are not bounded by construction are standardised with a causal exponentially weighted z-score, with $\alpha = 1/200$. Here x_t is a raw value, and μ_t and s_t^2 are its running mean and variance:

$$z_t = \frac{x_t - \mu_{t-1}}{\sqrt{s_{t-1}^2 + \epsilon}} \quad (4.3)$$

$$\mu_t = \mu_{t-1} + \alpha \delta_t, \quad s_t^2 = (1 - \alpha) (s_{t-1}^2 + \alpha \delta_t^2), \quad \delta_t = x_t - \mu_{t-1} \quad (4.4)$$

The subscript $t-1$ in Equation (4.3) is the most important detail of this design. A value is standardised with statistics that end at the previous step. Only after that is the value added to those statistics. A z-score computed over the whole sample would leak future information into every observation, and it would inflate every result derived from it. Features that are already bounded, such as the exposure fraction, the efficiency ratio and the calendar encodings, have a flag that skips this layer, so their natural range is kept.

4.2.2 Action Space: Position Sizing and Risk

The agent outputs a single continuous value $a \in [-1, 1]$. Its sign is the direction of the position, and its magnitude is the fraction of a reference size to hold. Direction and size are therefore set by one decision. The discrete action sets reviewed in Chapter 3 cannot express this. The reference size is

fixed-fractional. Let B be the account balance, ρ the risk fraction per position, and k the stop distance in multiples of the average true range. The reference volume V_{ref} is the volume that loses ρB if the price moves $k \cdot A$ against it. The target volume is this reference scaled by the action:

$$V_{target} = a \cdot V_{ref}, \quad V_{ref} = \frac{\rho B}{k A} \quad (4.5)$$

with $\rho = 1\%$ and $k = 1.5$ throughout this work. The agent therefore never chooses an absolute number of units. It chooses a fraction of a size that the risk rule has already set, and this size becomes smaller on its own as volatility rises.

One defect in this calculation was found during the campaign. It is reported here because it affects one of the seven results. The volume calculation leaves out the conversion between the quote currency and the account currency. The risk actually taken is therefore ρ/p and not ρ , where p is the price of the pair. For pairs quoted close to one the difference does not matter, and the six pairs quoted against the US Dollar all take about the intended 1%. USD/JPY was quoted near 120 at the start of the sample, so its effective risk is 0.008%. This is so small that the computed volume falls below the broker's minimum, and no position can open at all. To correct for this, ρ was scaled by the price. This restores the intended risk per position and does not add leverage. Chapter 5 reports the level used for each pair. The defect is a limitation of the framework, not of the method, and it is listed as future work.

4.2.3 Reward Function: Log Return, Scaling and Clipping

The reward at each step is the log return of account equity, scaled and then clipped:

$$r_t = \text{clip}\left(\lambda \ln \frac{E_t}{E_{t-1}}, -c, +c\right), \quad \lambda = 1000, \quad c = 1 \quad (4.6)$$

The log return is a natural choice for a trading objective because it is additive over time. The per-step rewards telescope to $\ln(E_T/E_0)$, so maximising their sum maximises the final compounded wealth. The log return also does not depend on the account size. The scale factor is needed, and the reason for it comes from a measured result. Moody and Saffell describe differential risk-adjusted rewards, which accumulate an online estimate of a Sharpe or Sortino ratio. These rewards are scale-invariant: doubling every position leaves the ratio the same. An agent that optimises such a reward gets no gradient that pushes its position size up. In this environment the policy collapses to positions of about 3% of the size available to it. The resulting policy is directionally sensible, but it has no financial value. A scale-sensitive reward is therefore needed. The factor λ raises an hourly equity log return, which is usually of order 10^{-4} , to a magnitude that the critic can distinguish from its own approximation error.

The clip is a numerical safeguard of the kind introduced with the Deep Q-Network [12]. It is applied after the encoding. Its effect must be stated clearly here, because λ and c are not independent. Together

they clip the raw log return at $\pm c/\lambda = \pm 0.1\%$. On the delivered EUR/USD model the bound is reached on 38.2% of the 68 155 hourly steps, and the median absolute scaled reward is 0.70. So for about two steps in five the agent is told the direction of the equity move but not its magnitude. This is a real property of the objective. It is reported here so that the learned behaviour in Chapter 5 is read against the reward the agent actually received.

4.2.4 Learning Algorithm: DDPG and its Adaptations

The algorithm is the Deep Deterministic Policy Gradient of Section 2.4. The actor, the critic and their delayed targets are as described there, and they are trained from a replay buffer with Ornstein-Uhlenbeck exploration noise. The equations of that section are not repeated here. The rest of this section gives the six ways in which the implementation differs from the original, each with its reason. Table 4.4 compares every value with the published one.

Table 4.4: Hyperparameters compared with the original Deep Deterministic Policy Gradient [6]. Three values were changed and five mechanisms were added. The rest are the published values.

Parameter	Original	This work	Status	Reason for the difference
Actor hidden layers	400×300	64×32	Changed	Noisy, non-stationary series supports less capacity
Discount factor γ	0.99	0.9995	Changed	Horizon matched to the holding period, months and not days
Hidden normalisation	Batch	Layer	Changed	Inference on one observation must match training on batches
Actor learning rate	10^{-4}	10^{-4}	Unchanged	
Critic learning rate	10^{-3}	10^{-3}	Unchanged	
Soft update rate τ	0.001	0.001	Unchanged	
Batch size N	64	64	Unchanged	
Replay capacity	10^6	10^6	Unchanged	
Critic weight decay	10^{-2}	10^{-2}	Unchanged	
Noise θ, σ	0.15, 0.2	0.15, 0.2	Unchanged	
Final layer init	$U[-3 \times 10^{-3}, 3 \times 10^{-3}]$	same	Unchanged	
Gradient clip	—	1.0	Added	Bounds the size of one update caused by a rare large error
Warmup transitions	—	3,000	Added	First updates drawn from a varied buffer
Actor penalty β	—	0.001	Added	Prevents saturation of the output tangent and collapse to a constant action
Reward scale λ	—	1,000	Added	Raises an hourly log return above the critic's approximation error
Reward clip c	—	1	Added	Numerical safeguard; limits the raw log return to $\pm 0.1\%$

The first difference is the observation and the reward, described above.

The second is the size of the networks. The original uses hidden layers of 400 and 300 units for low-dimensional control tasks. Here they have 64 and 32 units. A 38-dimensional observation of a noisy, non-stationary series supports much less capacity than a physics simulator with a clean reward. Larger networks fitted the training window but did not generalise beyond it. The third is the discount factor, which was raised from 0.99 to 0.9995. With an hourly step, $\gamma = 0.99$ gives a horizon of about a hundred bars,

or about four days. This is shorter than the positions this strategy is meant to hold. At 0.9995 the horizon grows to about two thousand bars, or roughly three months, which matches the regime indicators in the observation.

The fourth is the normalisation inside the networks. The original applies batch normalisation to the hidden layers. This implementation applies layer normalisation instead. The reason is deployment. Batch normalisation behaves differently on a single sample than on a training batch, because it depends on running statistics collected during training. On a live account, the agent must act on exactly one observation at a time. Layer normalisation normalises across the features of one sample. The computation done during training is therefore the same as the computation done in deployment. The target networks also do not need a second set of running statistics that must be kept in sync.

The fifth is gradient clipping. The original uses none. Here the gradient norm of both the actor and the critic is clipped to 1 before each optimiser step. This limits the size of a single update. It stops a rare large temporal-difference error from moving the parameters far enough to make the critic unstable. It is only a stability measure. In particular, it does not solve the collapse discussed next, which is caused by a lack of gradient, not by too much of it.

The sixth is a penalty on the actor. This is the only change that alters the objective itself, and not just the way it is optimised. It was added because the published algorithm does not train on this problem. The actor's final layer applies a hyperbolic tangent to a pre-activation $u(s)$ to keep the action bounded. The derivative of this function vanishes as $|u|$ grows. If the policy is pushed into that region, it stops responding. The action saturates at ± 1 , no gradient flows back, and the agent holds one constant position for the rest of training. The fix is to penalise the magnitude of the pre-activation directly. This keeps the policy in the region where the tangent does not saturate. Figure 4.3 shows where in the network this penalty acts:

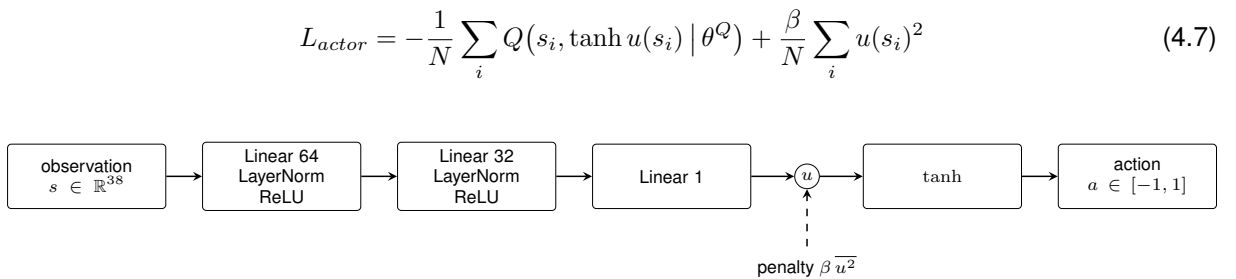


Figure 4.3: The actor. Layer normalisation replaces the batch normalisation of the original, so a single observation is processed exactly as a training batch is. The penalty of Equation (4.7) acts on the pre-activation u , before the tangent. This is where saturation would otherwise remove the gradient.

with $\beta = 0.001$. Setting $\beta = 0$ recovers the original actor loss exactly, so the change extends the original loss and does not replace it. Together with the scale-sensitive reward, this penalty prevents policy collapse. The two mechanisms deal with different parts of the same failure. The reward supplies

a gradient that rewards larger positions, and the penalty keeps the actor in the region where that gradient can act. Everything else follows the published algorithm. The learning rates, the soft update rate, the batch size, the replay capacity, the exploration noise parameters, the critic weight decay and the layer initialisation are the values given in the original work. A warmup period of 3000 transitions is used before learning begins. This way the first updates are drawn from a buffer with some variety in it, and not from the first few correlated steps of an episode.

4.3 Experimental Procedure: Training, Selection and Deployment

An agent design does not produce a result on its own. Training is stochastic, so the same configuration run twice gives two different policies. A procedure is needed to decide which of them is reported, and why. This section describes that procedure, the criteria used at each stage, and the two deployment decisions that sit between the learned policy and the orders actually sent.

4.3.1 Training Procedure: Seeds, Episodes and Data Splits

The eleven-year window is split once into three consecutive periods. The window is written as $[T_0, T_3)$, where T_0 is the first of January 2015 and T_3 is the first of January 2026:

$$\underbrace{[T_0, T_1)}_{\text{training, 9 years}} \quad \underbrace{[T_1, T_2)}_{\text{validation, 1 year}} \quad \underbrace{[T_2, T_3)}_{\text{testing, 1 year}} \quad (4.8)$$

Weights are updated on the training period only. The validation period decides which state of the network is kept. The testing period is used once, after every other decision has been made. Within this split, a single training run works as follows. A seed fixes the network initialisation, the stream of exploration noise and the order in which the replay buffer is sampled, so it determines the whole run. The agent then runs 30 episodes. An episode is one pass over the training period. Weights are not reset between episodes. The run is a single sequence of 30 passes, and the network at the start of episode $n + 1$ is the network at the end of episode n . After each episode the agent is evaluated once over the validation period, with learning turned off. This evaluation is reduced to a single number, the Calmar ratio, which is the annualised return divided by the maximum drawdown. Let f_n be this value and let g_n be an eligibility test defined below. The state kept after episode n is

$$\theta^* \leftarrow \theta_n \quad \text{if} \quad (g_n \wedge \neg g^*) \vee (g_n = g^* \wedge f_n > f^*) \quad (4.9)$$

which prefers an eligible episode to an ineligible one, whatever the scores. Otherwise it keeps the higher score within the same eligibility class. The saved state is a copy. The run continues training from

θ_n , not from θ^* , so a later episode never restarts from an earlier checkpoint. The eligibility test g_n rejects policies that are degenerate. It does not reject a policy only because it performs poorly. An episode passes only if it opened at least ten positions, spent at least three hundred bars on each side of the market, and held its weaker side for at least 30% of the time it held its stronger one. A policy that never trades, or that is long for the whole period, can reach a good Calmar ratio on a trending window without having learned anything about when to hold a position. The test removes such a policy before the scores are compared. Between 128 and 256 seeds were run for each currency pair, and the runs differed only in the seed. Figure 4.4 shows how the three stages fit together.

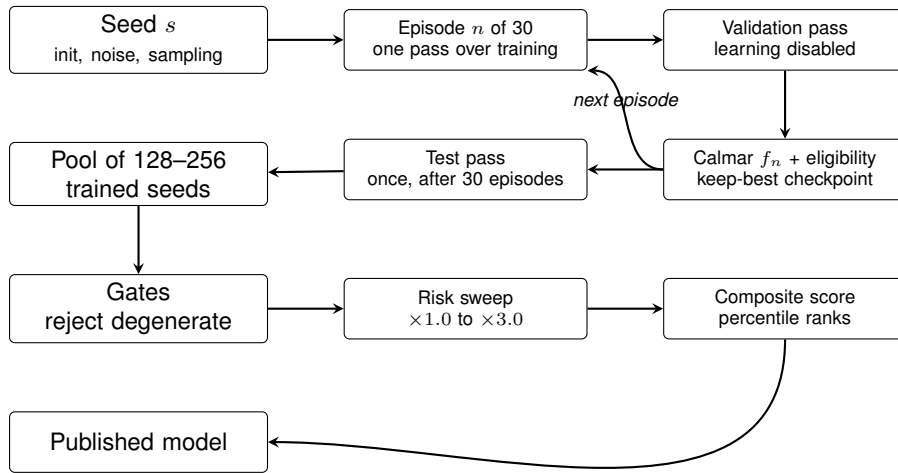


Figure 4.4: The three stages of the procedure. The Calmar ratio chooses among the episodes of one seed. The composite score of Section 4.3 chooses among seeds.

One property of this setup must be stated clearly, because it limits what the results in Chapter 5 can claim. The split of Equation (4.8) applies to the training of a single seed. The choice of which seed to publish, described next, is based on performance over the whole eleven-year window. The performance figures reported in this work are measured over that same window. The testing period is therefore held out from the training of each candidate and from the choice among its episodes. It is not held out from the choice among candidates, so the headline returns are not out-of-sample estimates. This is the weakest point of the method, and it is not hidden anywhere in the rest of this thesis.

4.3.2 Model Selection: Gates and Composite Ranking

Once the seeds for a pair have been trained, each one is evaluated over the full window with the cost model of Table 4.2, and one is chosen. The choice is made in two steps, first rejection and then ranking, because they answer two different questions. The first question is whether a policy is admissible at all. The second is which of the admissible policies is best.

Four gates reject a candidate directly. First, its return must be positive. Second, its weaker direction must make up at least 10% of its exposure. This removes policies that are long or short the whole time. Third, its mean holding period must be at least twenty-four bars. This removes policies whose signal is noise. Fourth, its regime score must be higher than the score of the same policy with its position sequence rotated in time. The regime score is the fraction of market movement during which the policy held the correct position. The last gate needs some explanation, because a fixed threshold was tried first and does not work. A rotated policy holds the same positions for the same durations, but at the wrong moments. Its score therefore measures what the policy's autocorrelation and holding pattern would earn by luck alone. Across the seven pairs these null scores range from 48 to 70. A fixed threshold of, for example, 55 would be below chance on one pair and too strict on another. The null score is therefore computed for each model and not assumed.

Candidates that pass the gates are ranked by a composite of percentile ranks within their own pool. For a metric m and a pool of K candidates, the rank of candidate i is

$$R_m(i) = 100 \cdot \frac{|\{j : v_m(j) < v_m(i)\}|}{K - 1} \quad (4.10)$$

where v_m is negated for metrics that are better when smaller, such as drawdown. The metrics are split into three groups, listed in Table 4.5. The score is the unweighted mean of the three group means:

$$S(i) = \frac{1}{3} \sum_{G \in \{\text{money, safety, behaviour}\}} \frac{1}{|G|} \sum_{m \in G} R_m(i) \quad (4.11)$$

Table 4.5: Parts of the composite selection score of Equation (4.11). The three groups have equal weight, so return makes up one ninth of the total and does not dominate it. The ratios are those defined in Section 2.1.5.

Group	Metrics, each used as a percentile rank within the candidate pool
Money	Annualised return, Calmar ratio, Sterling ratio
Safety	Sharpe ratio, Sortino ratio, inverse maximum drawdown, inverse downside volatility
Behaviour	Two-sidedness (the weaker exposure side as a fraction of half), regime edge over the model's own rotation null, mean holding period, longest holding period

Two design decisions are built into Equation (4.11). First, ranks are relative to the pool and not absolute. No threshold can therefore exclude a candidate that is simply better than all the others in its pool. Second, the three groups have equal weight, so a policy cannot win on return alone. Money, safety and behaviour each count for a third. A candidate with the highest return in its pool, but with no two-sidedness and no regime edge over its rotation null, scores poorly on two groups out of three. Each candidate that passes the gates is also evaluated at several position sizes, from the reference risk of Equation (4.5) up to three times that risk. Scaling the risk fraction changes only the size of the positions the policy takes. It does not change the learned policy. The level chosen for each pair is reported with

the results.

4.3.3 Deployment: Decision Frequency and Rebalance Threshold

Two decisions sit between the learned policy and the orders that are actually sent. Together they explain more of the delivered performance than any hyperparameter.

The first is the decision frequency, which is how often the agent may act. The agent observes every hourly bar, but it may act only once per calendar day. A key is built from the timestamp of each bar, and a new action is taken only when that key changes:

$$\kappa(t) = (\text{year}(t), \text{month}(t), \text{day}(t)) \quad (4.12)$$

The key is built from the calendar and not from a count of bars. This means that the gaps and short sessions described in Section 4.1 do not affect it. A week with a holiday still produces one decision per trading day, and the decisions stay aligned with the trading days. The second decision is a rebalance threshold. A continuous action changes slightly at every decision. Acting on every such change would send a stream of tiny orders, and each of them would pay the spread and a commission. An order is therefore sent only when the change in the target position is large enough to be worth its cost:

$$|V_{\text{target}} - V_{\text{current}}| \geq \max(V_{\text{min}}, \eta |V_{\text{ref}}|), \quad \eta = 0.20 \quad (4.13)$$

where V_{min} is the broker's minimum volume. The threshold is a fraction of the reference volume, not of the position currently held, so it does not shrink to zero as a position is reduced. Table 4.6 isolates the effect of each decision, measured on one model with nothing retrained. The comparison is between four configurations of the same weights, so the differences are attributable to the deployment layer alone.

Table 4.6: Effect of the two deployment decisions, measured on one EUR/USD model over the full window with nothing retrained. All four rows use the same weights with different execution settings, so the differences are attributable to the deployment layer alone. This model comes from before the seven-pair campaign and is used here only to isolate the effect of each decision.

Frequency	Threshold	Commission	Return	Sharpe	Max drawdown
Daily	—	none	+42.48%	0.319	24.4%
Hourly	—	none	−38.57%	−0.308	50.5%
Hourly	—	3.5 points	−64.70%	−0.732	70.5%
Daily	0.20	3.5 points	+34.02%	0.275	23.5%

Two things follow from that table. First, moving from hourly to daily decisions adds about eighty percentage points and turns a losing configuration into a profitable one. The reason is that hourly decisions act on noise that the eleven-year horizon of the discount factor was never meant to capture. Second, the rebalance threshold recovers about twelve percentage points under realistic costs, and at

the same time it reduces drawdown instead of increasing it. This is unusual. Commission alone costs over twenty percentage points without the filter and about six with it. The continuous action space makes the strategy expressive, and this filter keeps its trading costs low enough.

4.3.4 Worked Example: One Position From Decision to Settlement

The earlier parts of this chapter define each component separately. This section follows one position through all of them, using a trade taken by the delivered EUR/USD model. This allows the arithmetic to be checked instead of taken on trust.

At 02:00 on 30 January 2019 the daily decision key of Equation (4.12) changed, and the agent was asked for an action for the first time that day. It received the 38-value observation of Section 4.2, built from the bar that had just closed, and it returned a positive action. At that moment the reference volume of Equation (4.5) risked 1% of the balance over a stop of 1.5 average true ranges. The action scaled this volume, and the result was rounded to the broker's volume step. This gave a target of 12,000 units of the base currency. The change from the position held at that time was larger than the threshold of Equation (4.13), so an order was sent. The tick recorded at that instant quoted 1.14392 ask and 1.14387 bid, a spread of five points. The buy was filled at the ask. Twenty-four hours later the key changed again, and the agent's action changed sign. The position was closed at the bid quoted at that time, 1.14931. The profit follows from those two prices and the volume. It is expressed in the account currency at the conversion rate recorded on the closing tick:

$$12,000 \times (1.14931 - 1.14392) = 64.68 \text{ USD} \longrightarrow 56.27 \text{ EUR} \quad (4.14)$$

The spread is already included in that figure. It was paid at entry, by filling at the ask and not at the mid, and again at exit, by filling at the bid. This is why the round trip crosses the spread exactly once. The commission is charged separately at both ends:

$$2 \times 3.5 \times 10^{-5} \times 12,000 = 0.84 \text{ USD} \longrightarrow 0.72 \text{ EUR} \quad (4.15)$$

The position was held overnight, but the swap is zero because the account is swap-free. The position therefore settled at $56.27 - 0.72 = 55.55$ EUR, which is the figure in the exported trade record. One trade says little on its own. The delivered EUR/USD model took 1,777 trades over the eleven years. Across all of them, the same accounting gives a gross profit of 8,733.62 EUR, a commission charge of 1,669.50 EUR and a swap charge of exactly zero, for a net of 7,064.12 EUR. Commission alone took 19.1% of the gross profit. This figure does not include the spread, which cannot be separated from the gross figure because it is part of every fill. A result reported without these charges would describe a different strategy.

Everything described here was fixed before any result was seen. There is one architecture, one observation, one reward and one selection rule, and they were applied unchanged to seven instruments. This is what makes the results comparable across pairs.

It also makes the results falsifiable, because they can show the design to be wrong. A design that is tuned on one instrument until it works can always be made to work on that instrument. Its results then cannot tell a real strategy apart from a description of that instrument's past prices. When the same design is applied to seven instruments, a failure on any of them shows in the results. The results in Chapter 5 include a pair on which the design does not work.

5

Experimental Results

Contents

5.1	Evaluation Protocol	52
5.2	Benchmarks and Performance Metrics	52
5.3	Results Across the Seven Majors	53
5.4	Trading Behaviour: How the Agents Actually Trade	59
5.5	Path Robustness	65
5.6	Alpha and Beta Decomposition	66
5.7	Statistical Screening and the Rejected Candidates	68
5.8	Out-of-Sample Behaviour: The Held-Out Year	69
5.9	A Negative Result: USD/JPY	71
5.10	Portfolio: The Seven Models Combined	71

This chapter reports the results of the design of Chapter 4. Seven agents were trained, one for each major currency pair. Each agent was evaluated over eleven years under the cost model of Table 4.2. Every figure below comes from a stored run that can be opened again. The analysis was computed again from those runs. It was not copied from the training logs.

The order of this chapter was chosen on purpose. The returns come first. Then come the checks that decide whether those returns mean anything, then the year that was held back, and last the result of running the seven agents together.

5.1 Evaluation Protocol

The account, the cost model and the execution engine are described in Section 4.1 and are not repeated here. Three points are specific to the evaluation.

The evaluation window runs from 1 January 2015 to 1 January 2026, which is eleven years. It starts one year after the data begins. This way the slowest indicator in the observation, a moving average over five thousand hourly bars, is fully warmed up before the first decision.

Every result is measured from five different starting balances: 9 900, 10 000, 10 050, 10 100 and 10 200 euros. A strategy whose result depends on its starting point is not a real strategy. Small changes to the opening balance change the rounding of every position size, and so they change the whole sequence of trades. Reporting one balance would hide this sensitivity. The tables therefore give the mean across the five balances and the dispersion around it.

Position size is a parameter set for each pair separately. It was chosen by the sweep described in Section 4.3 and was not fixed in advance. Five pairs use the reference risk of Equation (4.5). NZD/USD uses one and a half times that risk, and USD/JPY uses a smaller fraction of it. Position size scales every trade, but it does not change the policy that chooses the trades. The ratios, the alpha and beta decomposition and the correlation structure are therefore not affected by it.

5.2 Benchmarks and Performance Metrics

The benchmark is to buy the pair at the start of the window and hold it to the end. This is the fair comparison for a directional strategy on a single instrument. Anyone can do it, and it costs one spread. A strategy that cannot beat it does not justify its extra complexity. The thesis proposal named a random-signal baseline and a moving-average crossover instead. Both were dropped, because a trader would not hold either of them as a real alternative.

The reported metrics are those defined in Section 2.1.5: total and annualised return, the Sharpe, Sortino, Calmar and Sterling ratios, maximum drawdown and the number of trades. Table 5.1 states what each one measures. All ratios use a risk-free rate of zero. They can therefore be compared across pairs without any assumption about interest rates.

For four of them, it is worth saying why all four are reported instead of choosing one. Return says how much was made. It says nothing about the risk taken to make it. The Sharpe ratio divides return by

Table 5.1: Summary of the performance metrics used in this work.

Metric	Description
Net Profit	Total profit or loss
Profit Factor	Ratio of gross profit to gross loss
Commission and Swap	Costs that reduce net profit
Max Balance and Equity Drawdown	Largest losses observed
Total Trades and Winning Trades	How much the strategy trades and how often it wins
Max Consecutive Winning and Losing Trades	How consistent the results are
Largest Winning and Losing Trades	How large single gains or losses can be
Average Trade	Average result per trade, a measure of trading efficiency
Sharpe Ratio and Sortino Ratio	Risk-adjusted return

total volatility. This penalises a strategy for moving up as much as for moving down. The Sortino ratio corrects this by counting only downside movement. Calmar and Sterling divide by drawdown instead of volatility. Calmar uses the worst loss ever suffered and Sterling the average one. They therefore measure how far the account fell, and not how volatile it was, which is the question an investor asks. A strategy can look strong on one of these and weak on another. Where that happens in this work, the text points it out.

One point applies to every number in this chapter. Returns are net of the spread quoted at the moment of each trade and of the account commission in Table 4.2. They therefore cannot be compared with results computed on bar closes with a fixed spread or with no spread. They are also lower than the same policies would show under those conditions.

5.3 Results Across the Seven Majors

Table 5.2 gives the result for each pair, in alphabetical order. Total return over the eleven years goes from +226.6% on USD/CAD, the highest, to +15.7% on USD/JPY, the lowest. Every pair is profitable, and every pair is profitable from all five starting balances.

Table 5.2: Result for each pair from 2015 to 2026. The return columns give the mean, standard deviation and range across the five starting balances of Section 5.1. Every pair is profitable from all five. Ratios and drawdown are from the reference run. USD/CAD returns the most, and USD/CHF is best on every risk measure.

Pair	Mean	SD	Min	Max	Sharpe	Sortino	Calmar	Max DD	Trades
AUD/USD	102.5%	10.6	87.2%	117.1%	0.348	0.496	0.143	41.1%	1,561
EUR/USD	73.9%	3.3	69.5%	77.8%	0.387	0.544	0.106	46.8%	1,778
GBP/USD	124.1%	5.1	115.1%	129.1%	0.397	0.563	0.155	50.4%	2,516
NZD/USD	56.4%	3.8	52.2%	61.7%	0.368	0.531	0.187	23.3%	1,011
USD/CAD	226.6%	10.8	216.9%	246.0%	0.603	0.870	0.228	49.1%	2,213
USD/CHF	50.6%	5.2	43.4%	56.4%	0.802	1.552	0.492	8.2%	220
USD/JPY	15.7%	0.7	14.5%	16.5%	0.251	0.342	0.057	24.0%	1,438

One fact is important for everything that follows, and it is easy to miss. **Five of the seven pairs lost value over the eleven years.** A trader who had simply bought and held would have finished down 26.1% on NZD/USD, 20.3% on USD/CHF, 18.2% on AUD/USD, 13.5% on GBP/USD and 2.9% on EUR/USD. Only USD/CAD, at +18.1%, and USD/JPY, at +30.7%, rose. The agents made money on all seven. On five of them, the return was therefore earned in a falling market, and not in a rising one.

Table 5.3: Each agent against buying and holding its own pair, over the same window and with the same costs. Five of the seven pairs lost value over the eleven years, so on those pairs the return was earned in a falling market. USD/JPY is the only pair whose benchmark rose by a large amount, and the only one the agent did not beat.

Pair	Market	Model	Excess	Positive Years
AUD/USD	−18.2%	+87.2%	+105.4 pp	8 of 11
EUR/USD	−2.9%	+70.6%	+73.4 pp	5 of 11
GBP/USD	−13.5%	+129.1%	+142.5 pp	7 of 11
NZD/USD	−26.1%	+60.1%	+86.1 pp	7 of 11
USD/CAD	+18.1%	+221.0%	+202.9 pp	8 of 11
USD/CHF	−20.3%	+54.8%	+75.2 pp	10 of 11
USD/JPY	+30.7%	+16.2%	−14.4 pp	5 of 11

The rest of this section discusses each pair in turn, in alphabetical order. For each pair it covers what the market did, what the agent returned in comparison, how that return decomposes into policy and market exposure, what the agent paid in costs, and how it traded.

5.3.1 AUD/USD

The Australian Dollar fell 18.2% against the US Dollar over the eleven years. The agent returned +87.2%, which is 105.4 percentage points above holding the pair. It earned this from the short side: only 27.9% of its 1 561 positions were long.

This result needs care in the alpha and beta decomposition. Its beta of -2.013 is the largest negative beta of the seven. This means that the agent was short the falling market at roughly twice its size. Of its 8.18% mean yearly return, 4.96 points are alpha and 3.22 points are beta return, from the general fall of the market. Close to two fifths of what it earned therefore came from the Australian Dollar falling, and not from choosing when to be short. The policy does make a profit, but part of that profit would turn into a loss in a decade where the pair rose.

It trades at a moderate pace. Its median holding period is 9.7 days, its average size is 0.77 of the reference size, and its win rate is 52.7%. It made 10 689 euros of gross profit and paid 2 200 in commission. That is a 20.6% share, the highest of the seven, and it comes directly from its turnover. The equity curve is uneven. Eight of eleven years are positive. The best year is 2024, at +72.4% and 81.6 points above the benchmark, and the worst is 2017, at -22.5% . Its maximum drawdown is 41.1%.

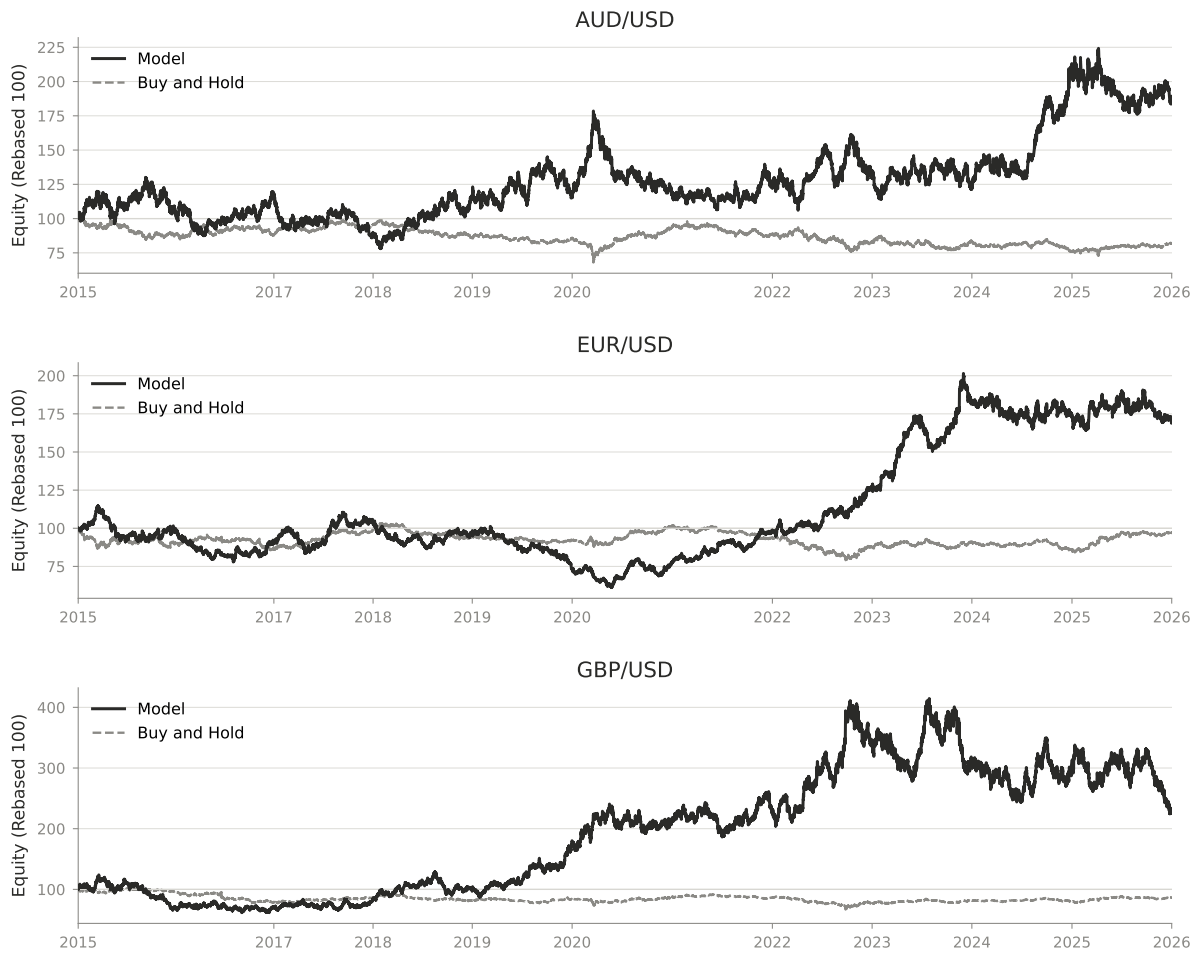


Figure 5.1: AUD/USD, EUR/USD and GBP/USD against buying and holding the same pair, rebased to 100. All three benchmarks finish below their starting level.

5.3.2 EUR/USD

EUR/USD is the clearest example in this work of a return that does not come from market exposure. The pair finished almost exactly where it started, down 2.9% over eleven years, and the agent returned +70.6%. Its beta is +0.266, and market exposure adds 0.01 percentage points per year. Of a 6.63% mean yearly return, 6.62 points are therefore attributable to the policy. The market had no trend for the agent to follow, so the return came from timing alone.

It has the shortest holding period of the seven, a median of 3.9 days, and it trades at a low 0.40 of the reference size. Its exposure is close to balanced, with 55.5% long across 1 778 positions. It won 51.9% of them. Gross profit of 8 734 euros became 7 064 after 1 669 euros of commission, a 19.1% share.

The equity curve in Figure 5.1 shows that the headline return also needs care here, for a different reason than on AUD/USD. Only five of eleven years are positive. The agent is roughly flat for the first six years of the window. It makes almost all of its return between 2021 and 2023, when the pair moved

in unusually wide ranges: +27.3%, +29.0% and +44.0% in those three years, against -25.3% in 2019. This policy needs volatility to work. Its maximum drawdown of 46.8% comes from the periods when there was not enough volatility.

5.3.3 GBP/USD

GBP/USD has the highest alpha of the seven, at +11.64% per year, with a beta of -0.054 that is statistically indistinguishable from zero. Its mean yearly return is 11.70%, and only 0.06 points of it come from market exposure. It is as close to pure policy as any result in this chapter. Sterling itself fell 13.5% over the window, so the agent's +129.1% is 142.5 percentage points above the benchmark.

It is the most active model, with 2516 positions. It also holds them the longest of the six working agents, with a median of 12.6 days. It therefore keeps many positions open at the same time, and does not trade quickly. It trades near full size, at 0.89, and wins 51.3% of the time. Gross profit was 16 630 euros against 3 168 of commission, which left 13 462.

Its returns come mostly from the years when sterling moved sharply. These show in Figure 5.1 as two steep sections: +82.2% in 2019 and +52.0% in 2022. Section 5.4 looks at the second of these in detail. The same aggressive trading also produces the worst single year of any agent, -31.6% in 2015, and the deepest drawdown, 50.4%. Seven of eleven years are positive. This is the most volatile of the profitable models. A reader should weigh its alpha against a drawdown that halved the account.

5.3.4 NZD/USD

The New Zealand Dollar fell the most of the seven, 26.1% over the window. The agent returned +60.1%, an excess return of 86.1 points over the benchmark. Like AUD/USD, it traded mostly from the short side, with 24.0% of positions long and a beta of -0.708.

Its alpha of +2.79% per year is the smallest of the six profitable models. Of its 4.62% mean return, 1.83 points are beta return. A large part of what it earned, though less than half, therefore came from the market falling and not from timing. By alpha and beta, it is the weakest of the six that worked.

Figure 5.2 shows it next to USD/CAD and USD/CHF. This agent is careful, and its risk numbers show the benefit. It takes the fewest positions of the active models, 1011, and holds an average of 0.24 of the reference size. Its maximum drawdown is 23.3%, the second lowest of the seven. Its Calmar ratio of 0.187 is better than that of four pairs that returned much more. This shows why return is measured against drawdown, and not on its own. Gross profit was 7 314 euros and commission 1 424, a 19.5% share. Seven of eleven years are positive. The best is 2022 at +14.5% and the worst is 2016 at -6.4%. It is the steadiest of the models of moderate size.

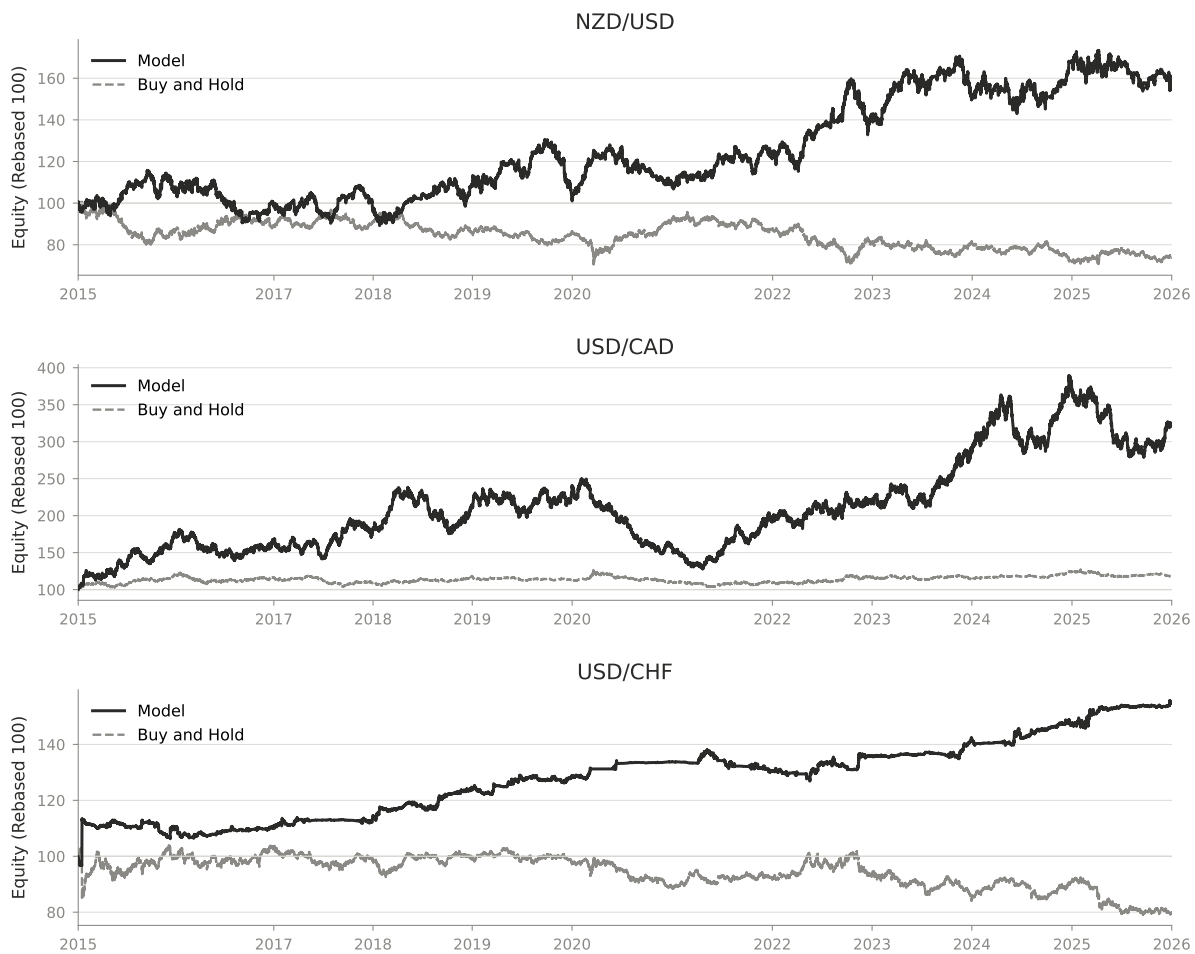


Figure 5.2: NZD/USD, USD/CAD and USD/CHF against buying and holding the same pair, rebased to 100. USD/CAD is the only one of the three whose benchmark rises.

5.3.5 USD/CAD

USD/CAD returned +226.6%, by far the largest total in this work. It is also the result that must be read most carefully. Its beta of +2.393 is the highest of the seven, and the pair itself rose 18.1%. The agent was therefore long a rising market at more than twice its size. Of a 14.23% mean yearly return, 4.21 points are beta return and 10.02 are alpha.

That alpha is real, and it is the second highest of the seven. So the model does not look good only because its market went up. But leverage is symmetric: it raises losses as much as gains, and the yearly record shows both. The best year is 2015, at +71.7% and 52.7 points above the benchmark. The worst is 2020, at -34.8%. Its maximum drawdown is 49.1%, and recovering from a 49% drawdown needs a 96% gain. Eight of eleven years are positive.

It is the second most active model, with 2 213 positions and a median holding period of 5.7 days. It trades at 0.79 of the reference size and has the highest win rate of the seven, at 55.0%. It also earned

the most in euros, 25 577 gross, and paid 3 432 in commission. That is a 13.4% share, lower than for the four pairs discussed above, because its profit per position was larger.

5.3.6 USD/CHF

USD/CHF returned +54.8%. It is fifth of seven on return and first on every risk-adjusted measure. Its Sharpe ratio of 0.802, Sortino ratio of 1.552 and Calmar ratio of 0.492 are the highest of the seven. Its maximum drawdown of 8.2% is a sixth of the worst. The Swiss Franc strengthened over the period, so the pair fell 20.3%. With a beta of -0.082 , the agent took almost none of that movement: of a 4.07% mean yearly return, 3.91 points are alpha.

Its trading explains all of these numbers. It takes 220 positions in eleven years, roughly one every eighteen days. It holds an average of 0.09 of its reference size, a tenth of what it is allowed. Because it trades so little, it pays almost nothing to trade: 130 euros of commission against 5 587 of gross profit. That is a 2.3% share, where the average of the seven is 16.0%. It keeps 97.7% of what it earns, while AUD/USD keeps 79.4%.

It is positive in ten of eleven years, the most consistent record in this work, and its worst year is -2.4% . Its win rate of 49.8% is the lowest of the seven. This point deserves attention. The best model of the seven is wrong slightly more often than a coin toss. It is still the best because of how it sizes and holds the positions it gets right. It is also the only model that made money in the held-out year of Section 5.8. On return alone it would have placed fifth and would not have been chosen. The selection criteria of Section 4.3 are there to prevent this.

5.3.7 USD/JPY

USD/JPY is the one failure of the seven, and it is the only pair whose benchmark rose by a large amount. The pair gained 30.7% over the window, while the agent returned +15.7%. It finished 14.4 percentage points behind an investor who bought the pair and did nothing. Its beta of +1.012 and its correlation of +0.946 with the pair's own yearly move show what it became: a plain long position in the pair, with extra trading and extra costs. Its alpha is -1.08% per year, the only negative figure in Table 5.7.

Every position it takes is long. Its median holding period is measured in years, not days. Its average action is 0.98. This means it holds almost full size all the time and no longer makes a sizing decision at all. It earned 1 761 euros gross, the least of the seven, and paid 155 in commission. Five of eleven years are positive.

Figure 5.3 shows this more clearly than the statistics do: the model and the benchmark move together, with the model slightly below. Section 5.9 explains the cause and what was tried to fix it.

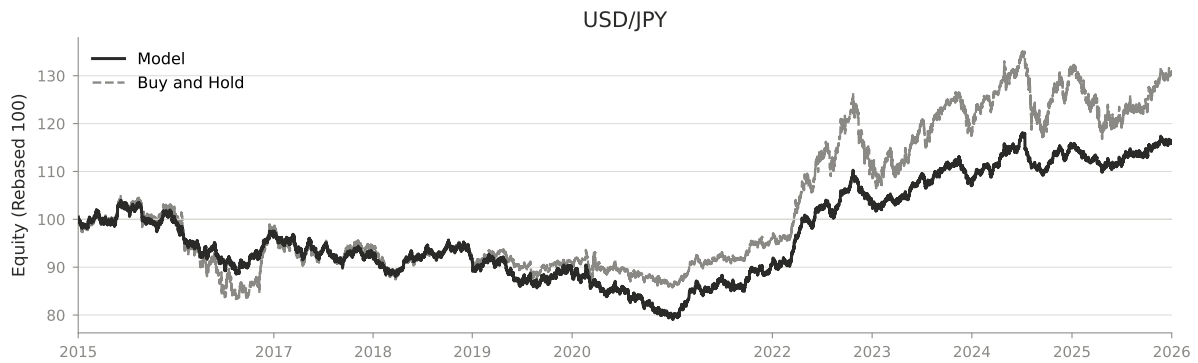


Figure 5.3: USD/JPY against buying and holding the pair. The two lines move closely together. This is what a beta of +1.012 looks like, and it is the reason this result is reported as a failure.

5.4 Trading Behaviour: How the Agents Actually Trade

A return figure says whether an agent made money. It says nothing about how, and here that matters for a specific reason. The main claim of this work is that a continuous action lets one decision set both the direction and the size of a position. If the agents had learned to use only the two ends of that action, they would be discrete traders with extra machinery, and the claim would mean nothing. This section examines what they did in practice.

5.4.1 Use of the Continuous Action

The agent outputs an action $a \in [-1, 1]$ and holds $|a|$ of the reference size defined in Equation (4.5). Figure 5.4 shows how this value is distributed over the whole evaluation window for each pair.

Table 5.4: Trading behaviour over the evaluation window. Mean $|a|$ is the average absolute action, that is, the average fraction of the reference position size held while in the market. Win rate is the share of closed positions with a positive net result after costs.

Pair	Positions	Long	Median Hold	Longest Hold	Win Rate	Mean $ a $
AUD/USD	1,561	27.9%	9.7 d	227 d	52.7%	0.77
EUR/USD	1,778	55.5%	3.9 d	43 d	51.9%	0.40
GBP/USD	2,516	46.9%	12.6 d	149 d	51.3%	0.89
NZD/USD	1,011	24.0%	5.9 d	119 d	52.8%	0.24
USD/CAD	2,213	65.6%	5.7 d	97 d	55.0%	0.79
USD/CHF	220	34.2%	8.0 d	345 d	49.8%	0.09
USD/JPY	1,438	100.0%	2,120 d	4,004 d	50.7%	0.98

The action does not saturate at +1 or −1. On every pair, fewer than one percent of decisions are above $|a| = 0.98$, and on six of the seven pairs the figure is zero to one decimal place. The mean absolute action goes from 0.09 on USD/CHF to 0.98 on USD/JPY. The agents therefore settled on very different position sizes from the same design and the same reference risk.

This dispersion of sizes is informative in itself. USD/CHF holds roughly a tenth of the size available

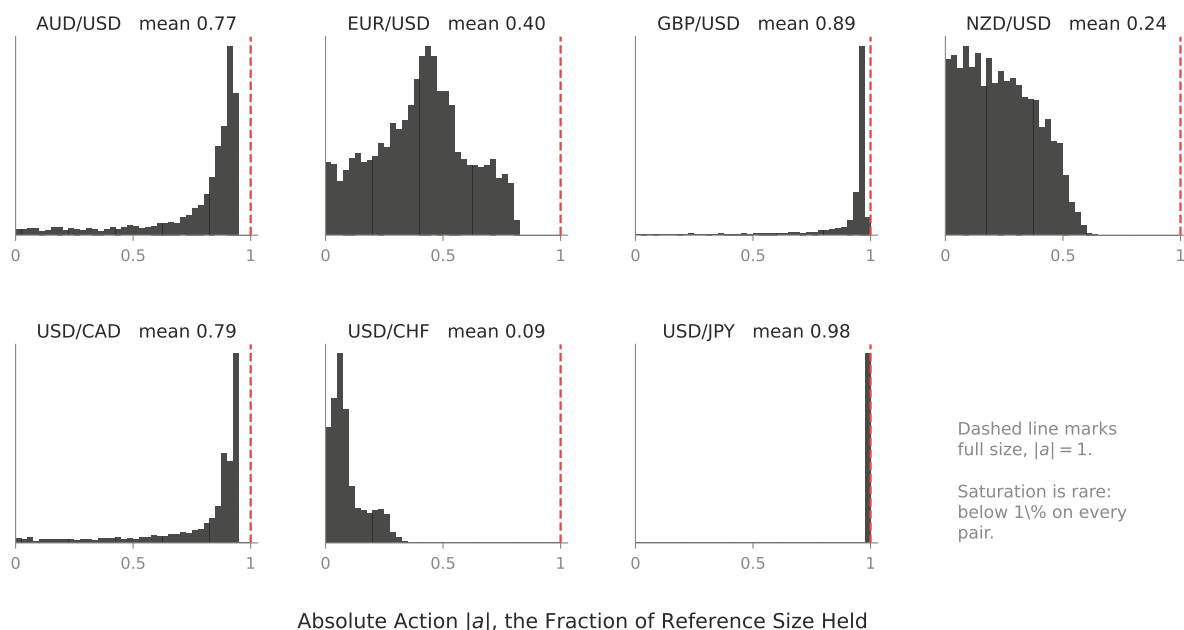


Figure 5.4: Distribution of the absolute action over the evaluation window. The dashed line marks full size. If the agents had turned the continuous action into a discrete one, these histograms would be concentrated against that line. Instead, most of the mass lies inside the range.

to it. This is why its maximum drawdown is 8.2% while its Sharpe ratio is the highest of the seven. It found a small advantage and kept its size small to match, instead of pushing it harder. NZD/USD, at 0.24, behaves in a similar way. USD/JPY is at the other end, with 0.98. This number is the symptom of the failure described in Section 5.9: an agent that always holds full size in one direction no longer makes a sizing decision at all.

The sizing also adapts in a second way, which the histogram does not show. The reference size is inversely proportional to the average true range, so the same action gives a smaller position when volatility rises. The action and the volatility scaling combine, and the position held is the product of the two.

5.4.2 Direction, Size and Outcome Together

Figure 5.5 follows EUR/USD over three years. It shows the price, the action of the agent, the position it held as a result, and the equity that followed.

Three behaviours can be seen. First, the action moves continuously and does not switch between fixed levels. Its magnitude changes as much as its sign. Second, the position follows the action, but in larger steps, because the rebalance threshold blocks small adjustments. Long periods of constant position are periods where the agent kept changing its target by less than the threshold. Third, the sign does not change often. Over the three years the agent reverses direction a few dozen times. This fits

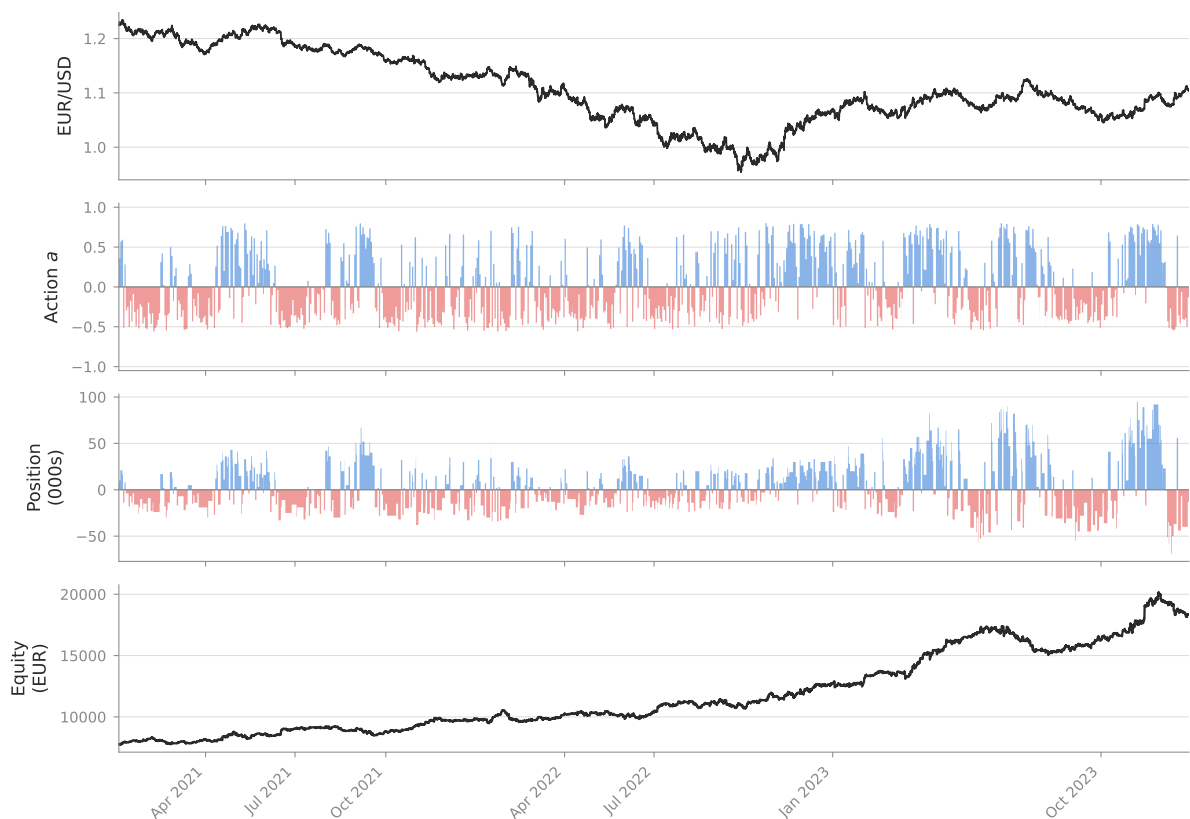


Figure 5.5: EUR/USD from 2021 to 2024. From top: the price; the action, positive for long and negative for short; the resulting net position in thousands of units; and account equity. The action changes sign well before the position does, because a change is only acted on when it is larger than the threshold of Equation (4.13).

the daily decision frequency, and does not fit an agent that reacts to every bar it observes.

5.4.3 Trend Identification and Reversal

The clearest example of an agent holding a position through a sustained move is GBP/USD in the second half of 2022, shown in Figure 5.6. Sterling fell from 1.22 to below 1.04 over roughly ten weeks, a fall of more than fifteen percent. The steepest part came after the United Kingdom fiscal announcement of late September.

The agent was short for almost the whole fall. It held short exposure of between fifty and one hundred thousand units, added to it as the move continued, and reduced it as the price neared the low. The marked position was opened at 1.1396 on 12 August and closed at 1.0403 on 26 September, forty-five days later. Its net profit was 2073 euros after costs. It is the largest single position of the eleven years on this pair.

Two details are worth pointing out, because they show judgement and not luck. The agent was briefly



Figure 5.6: GBP/USD from mid-July to mid-October 2022. Background shading marks the direction of the net position, red for short and blue for long. The lower panel gives the size of that position. The marked trade is the most profitable single position of the eleven years on this pair.

long at the end of July, at the local high before the fall began. It was long again in the days right after the bottom. Both times the size was small. It did not hold the short position forever. Once the price recovered above 1.12 in early October, the position was reduced and then reversed. This is how an agent behaves when it responds to the state it observes, and not when it is fixed on one direction.

5.4.4 Holding Periods and Two-Sided Exposure

Table 5.4 gives the trading statistics for each pair, and Figure 5.7 shows two of them. Median holding periods go from four days on EUR/USD to nearly thirteen on GBP/USD, and the longest single positions last for months. These agents are swing traders, which hold positions for days, and not scalpers, which trade in and out quickly. This is what the daily decision frequency of Section 4.3 was meant to produce.

The long and short shares differ a lot between pairs, from 24% long on NZD/USD to 66% on USD/CAD. EUR/USD and GBP/USD are close to balanced. None of the six working agents trades only one side. This comes from the design and is not an accident. The eligibility test of Section 4.3 rejected any episode whose weaker side fell below a threshold, so a one-sided policy could not be selected, however profitable it looked. USD/JPY is the exception, at 100% long. It is the exception because none of the 448 seeds that were tried on that pair could pass that test.

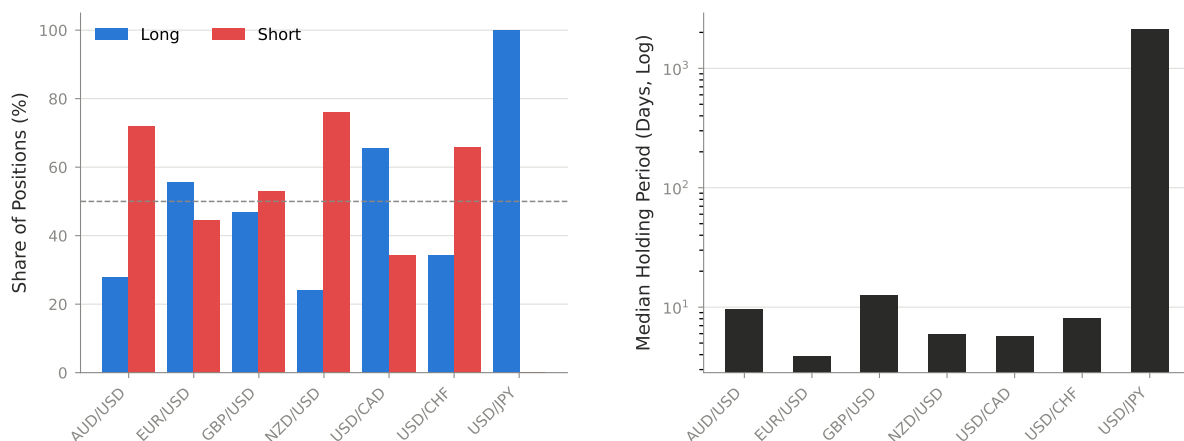


Figure 5.7: Left: the share of positions taken long and short. Right: the median holding period in days, on a logarithmic scale. USD/JPY is long in every position it takes and holds for years, which puts it far outside the range of the other six.

5.4.5 Where the Profit Comes From

One statistic in Table 5.4 deserves a closer look. The win rate is the fraction of positions closed at a profit. It lies between 49.8% and 55.0% across the seven pairs. Six of the seven are within three points of a coin toss. USD/CHF, the best model of the seven on every risk measure, has the lowest win rate of all, at 49.8%.

So the agents do not make money by being right more often than they are wrong. Their profit comes from what they do with a position once it is open. They hold a larger size when the position moves in their favour than when it does not, and they keep positions that work open longer than positions that do not. A continuous action makes this possible. An agent that can only choose a direction cannot do it. This also explains how USD/CHF can be the strongest model of the seven while being wrong slightly more often than a coin toss. It trades at small size and is patient, and the few positions it holds at a larger size are the ones that worked.

5.4.6 What the Trading Costs

The engine charges the spread quoted at each fill and a real commission schedule. The difference between what the policies earned and what the account kept can therefore be measured, and does not have to be estimated.

The commission is recorded directly on every trade. The spread is not, because it is paid inside each fill and is not deducted afterwards. A buy is filled at the ask and a sell at the bid, so the cost is already included in the entry and exit prices. It can still be recovered exactly, because the two charges scale in the same way. Commission is charged at 3.5 points per side, which is seven points for a round turn, that

is, one opening and one closing trade. The spread is crossed once per round turn. Both are proportional to the traded volume, and both are converted into the account currency at the same rate. So for any position

$$C_{spread} = |C_{commission}| \cdot \frac{s}{7} \quad (5.1)$$

where s is the spread in points at the moment of the trade. Applying Equation (5.1) to every position gives Table 5.5, with s taken from the recorded quotes for the month in which the position was opened.

Table 5.5: What the seven agents earned before costs and what they kept, in euros. Cost share is the spread and the commission together, as a fraction of profit before costs. The mean spread column is the commission-weighted average spread paid, in points. It differs from the pair's average spread because it gives weight to the moments when the agent traded.

Pair	Before Costs	Spread	Commission	Net	Cost Share	Mean Spread
AUD/USD	11,851	−1,162	−2,200	8,488	28.4%	3.70 pts
EUR/USD	9,372	−638	−1,669	7,064	24.6%	2.68 pts
GBP/USD	19,086	−2,456	−3,168	13,462	29.5%	5.43 pts
NZD/USD	8,552	−1,238	−1,424	5,890	31.1%	6.09 pts
USD/CAD	28,272	−2,695	−3,432	22,145	21.7%	5.50 pts
USD/CHF	5,735	−148	−130	5,457	4.8%	7.93 pts
USD/JPY	2,172	−411	−155	1,606	26.1%	18.53 pts
Total	85,039	−8,748	−12,179	64,112	24.6%	—

The seven agents earned 85 039 euros before costs and kept 64 112. Of the difference, 8 748 euros went to the spread and 12 179 to commission, so **costs took 24.6% of trading profit**. Commission is the larger of the two charges here because the account is a raw-spread account. On such an account the spread is narrow by design, and the broker makes its money through the commission instead. On an account with wider spreads and no commission, the same trades would have paid a similar total, split in a different way.

Table 5.5 has no swap column because the swap is zero on every pair. A swap-free account charges no overnight financing, so zero is the exact figure and not an approximation. This matters at these holding periods. Agents that hold positions for a median of four to thirteen days would otherwise pay financing on several hundred nights each.

The differences between pairs are large, and they follow turnover almost exactly. USD/CHF takes 220 positions and gives up 4.8% of its profit before costs, so it keeps more than nineteen euros in every twenty. NZD/USD gives up 31.1% and GBP/USD 29.5%. A policy of the same quality therefore gives very different net results at different trading frequencies. This is why the selection criteria of Section 4.3 reward long holding periods directly, instead of relying on the return figure to include this effect. It is also why the rebalance threshold of Equation (4.13) contributes as much as Table 4.6 shows.

One consequence matters for how the rest of this chapter should be read. Every return quoted in this work is net of both charges. A study that tested the same policies on bar closes with no spread and

no commission would have reported the before-costs column of Table 5.5. That column is a third larger, and it describes a strategy that no account could have traded.

Recovering the spread through Equation (5.1) works, but it is indirect and it needs a commission that is not zero. The engine computes the spread charge internally when it opens a position. It does not yet write this charge to the exported trade record as a separate field. Recording it there, next to commission and swap, is a simple improvement and is noted in Section 6.3.

5.5 Path Robustness

Each pair was evaluated from the five starting balances described in Section 5.1. The result is the same in every case. All seven pairs are profitable from all five starting points, thirty-five runs in total, with no exceptions. Table 5.6 gives the individual figures.

Table 5.6: Total return from each of the five starting balances, in euros. All thirty-five runs are profitable. A broker accepts volume only in discrete steps, so a different opening balance rounds positions differently. It produces a truly different sequence of trades, and not a scaled copy of the same one.

Pair	9,900	10,000	10,050	10,100	10,200	Mean	SD	Relative SD
AUD/USD	94.2%	87.2%	117.1%	108.3%	105.7%	102.5%	10.58	10.3%
EUR/USD	69.5%	70.6%	76.5%	77.8%	75.1%	73.9%	3.27	4.4%
GBP/USD	126.5%	129.1%	127.9%	122.0%	115.1%	124.1%	5.11	4.1%
NZD/USD	61.7%	60.0%	54.4%	53.7%	52.1%	56.4%	3.76	6.7%
USD/CAD	230.4%	221.0%	218.5%	216.9%	246.0%	226.6%	10.81	4.8%
USD/CHF	56.4%	54.8%	43.4%	52.6%	45.5%	50.6%	5.16	10.2%
USD/JPY	15.9%	16.2%	16.4%	15.7%	14.5%	15.7%	0.69	4.4%

It is not obvious why this test is needed, so the reason is given here. A change of one hundred euros in the opening balance looks like a change of scale. If position sizes were continuous, it would be exactly that. Every trade would be one percent larger, the equity curve would be a scaled copy, and the percentage return would be the same. Sizes are not continuous. A broker accepts volume only in discrete steps, so the reference volume of Equation (4.5) is rounded before an order is sent. A slightly larger balance can round a position up where a smaller one rounds it down. This changes the equity available for the next decision, which changes the size of the next position, and so on. Two runs that differ only in their opening balance are therefore not the same run at a different scale. After a few dozen trades they hold different positions at different times.

The seven pairs stay profitable on all five paths, and their returns are close together and not scattered. This shows that the results do not depend on one particular sequence of roundings. The relative standard deviation, which is the standard deviation divided by the mean, is between 4.1% and 10.3%. GBP/USD has the smallest, 4.1%, although it is the most active model. AUD/USD has the largest, 10.3%.

Two observations follow from the dispersion of the individual figures. The pairs with the largest relative dispersion, AUD/USD at 10.3% and USD/CHF at 10.2%, are not the pairs with the largest returns.

The dispersion depends on how close the agent's usual position size is to a rounding boundary, and not on how much the agent makes. Also, no pair has a path that turns a profit into a loss, or even into a clearly different conclusion. The worst path on USD/CAD still returns +216.9% and the best on USD/JPY still returns +16.4%, so the ranking of the pairs holds on all five paths.

This test also has limits, and they should be stated as clearly as what it measures. It changes the starting path, not the training run. Each published model is one seed taken from a pool of between 128 and 256. The variation across that pool is a different and larger source of uncertainty. It is discussed in Section 5.7 and again in Section 5.8. A result can be fully stable across starting paths and still come from a lucky draw.

5.6 Alpha and Beta Decomposition

A currency pair moves on its own. An agent that is long a pair that happens to rise will show a profit that has nothing to do with the policy it learned. The same is true for an agent that is short a pair that happens to fall. Total return cannot separate this profit from the profit of the policy. The results are therefore decomposed into the part explained by the market and the part that is not.

For each pair, the yearly return of the model is regressed on the yearly move of the pair itself, over the eleven years from 2015 to 2025. With r_i the model return in year i and m_i the market move in the same year, ordinary least squares gives

$$\beta = \frac{\sum_i (m_i - \bar{m})(r_i - \bar{r})}{\sum_i (m_i - \bar{m})^2}, \quad \alpha = \bar{r} - \beta \bar{m} \quad (5.2)$$

β measures the model's exposure to the market's movement. α is the average yearly return that is left once that exposure is removed. The mean return then splits into an active part and a passive part:

$$\bar{r} = \underbrace{\alpha}_{\text{policy}} + \underbrace{\beta \bar{m}}_{\text{market exposure}} \quad (5.3)$$

Yearly returns are used instead of daily returns for a specific reason. The agents hold positions for days to weeks and act once a day. A daily regression would therefore measure how the model tracks the market within a holding period. It would not measure whether the model chose the right direction over the horizon it trades. Eleven yearly observations are a small sample, and each regression coefficient has a large uncertainty. The decomposition is good enough to sort the pairs into groups, but it does not give a precise value for any one of them.

Table 5.7 gives both terms. Six of the seven pairs have positive alpha, and the seven fall into three clear groups.

Three pairs have almost no market exposure. GBP/USD, EUR/USD and USD/CHF have betas

Table 5.7: Decomposition of Equation (5.3), from the yearly regression of Equation (5.2) over 2015 to 2025. Alpha is the return attributable to the policy, and the beta return is the part explained by the pair's own movement. The two add up to the mean yearly return.

Pair	Beta	Alpha (%/yr)	Beta Return (%/yr)	Mean (%/yr)	Reading
AUD/USD	−2.013	+4.96	+3.22	+8.18	Policy plus leveraged trend
EUR/USD	+0.266	+6.62	+0.01	+6.63	Policy, no market exposure
GBP/USD	−0.054	+11.64	+0.06	+11.70	Policy, no market exposure
NZD/USD	−0.708	+2.79	+1.83	+4.62	Policy plus leveraged trend
USD/CAD	+2.393	+10.02	+4.21	+14.23	Policy plus leveraged trend
USD/CHF	−0.082	+3.91	+0.16	+4.07	Policy, no market exposure
USD/JPY	+1.012	−1.08	+2.64	+1.57	No policy; the model follows the market

of −0.054, +0.266 and −0.082. Their market exposure adds 0.06, 0.01 and 0.16 percentage points per year to their returns. For GBP/USD, 11.64 of its 11.70 points per year come from the policy. These results would hold up if the market changed direction, because they hardly depend on the direction of the market.

Three pairs combine policy with leveraged exposure. USD/CAD at +2.393, AUD/USD at −2.013 and NZD/USD at −0.708 all have a large exposure to their market, and two of them are leveraged more than two to one. Their alpha is positive in every case, so the policy adds skill on top. But between a fifth and two fifths of what they earned came from the market moving in the direction of their exposure. USD/CAD is the clearest example: 4.21 of its 14.23 points per year came from the pair rising while the agent was net long. If the Canadian Dollar had become stronger over the decade instead, that component would have been a loss of similar size, and the reported +226.6% would look very different.

Figure 5.8 plots the seven together, and the three groups are clearly separate.

One pair has no policy at all. USD/JPY has a beta of +1.012 and a correlation of +0.946 with its own market, and its alpha is −1.08% per year, the only negative figure in the table. It is examined in Section 5.9.

The method was checked before it was used. The same regression, run on the model of the previous campaign, reproduces its recorded figures exactly: a correlation of +0.115, a beta of +0.130 and an alpha of +3.11% per year. This confirms that the decomposition here is computed in the same way as the one it is compared with.

This decomposition is given priority over total return, for a reason that affects how the whole chapter should be read. The model to publish was chosen from a pool of candidates, and any measure used to make that choice is inflated by it. Return is such a measure. Beta is not. Beta describes how a policy behaves against its market. Selection can give a model a high return by luck, but it cannot give it a low beta in the same way. A large alpha with a beta near zero is therefore a stronger claim than a large return, and it is the claim this work puts first.

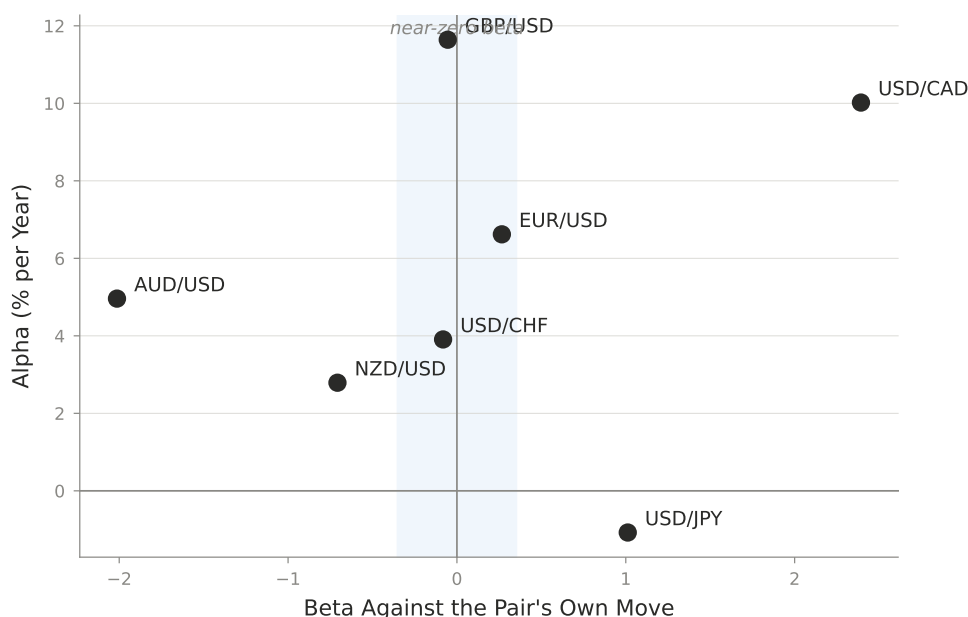


Figure 5.8: Alpha against beta for the seven agents. Points inside the shaded band have almost no market exposure, so their return is attributable to the policy. USD/JPY sits at a beta of one with negative alpha. This is the pattern of a model that has copied its own market.

5.7 Statistical Screening and the Rejected Candidates

Every published model had to pass a test against a null built from the model itself. The model's own sequence of positions is rotated in time. The rotated sequence holds the same positions for the same durations, but at the wrong moments. Its regime score is therefore the score that the model's holding structure earns by luck alone. The published model must beat that score.

The null must be computed for each model and cannot be a fixed number. Across the seven pairs it goes from 48.21 on GBP/USD to 69.82 on NZD/USD. A fixed threshold of, say, 55 would be below chance on one pair and well above it on another. It would accept noise in one market and reject skill in another. This was found by trying a fixed threshold first.

The screening gave a result that changes how the whole campaign should be read. **On every one of the seven pairs, the candidate with the highest return failed the test.** Table 5.8 lists the strongest cases. A portfolio built by ranking candidates on return would have chosen models that return roughly +250% on average. Almost all of that return is market exposure, not policy.

The USD/CHF case is the clearest. A candidate that returned +463.94%, more than nine times the published model, has a permutation p -value of 0.973. Its positions are timed no better than the same positions shuffled. Its return came from following the trend of the franc. The published model returns a ninth of that, and it is the one with measurable skill.

Two cases are reported because they are uncomfortable. An AUD/USD candidate that returned

Table 5.8: Candidates rejected by the screening of Section 5.7. On every pair, the candidate with the highest return failed. Ranking on return alone would have chosen these models over the ones published. They show roughly five times the return on paper, but almost none of the measurable skill.

Pair	Return	<i>p</i> -value	Why it was rejected
AUD/USD	+246.8%	0.146	Failed the screen
AUD/USD	+142.6%	0.987	Published during the campaign, then withdrawn when the test was run
EUR/USD	+141.6%	—	One-sided: long for 86% of the window
GBP/USD	+259.8%	0.163	Failed the screen
NZD/USD	+253.9%	0.258	Failed the screen
USD/CHF	+463.9%	0.973	Positions timed no better than the same positions shuffled; followed the franc trend
USD/CHF	+341.3%	0.865	As above

+142.55% was published during the campaign. It was withdrawn when its *p*-value came out at 0.987. On EUR/USD, the model in place at the time won on four of six metrics, including return, Sharpe, Sortino and drawdown. It was still replaced, because it lost on the one measure that decides whether a result is real.

This screening has limits, and they should be stated. Each published model was chosen from between 128 and 256 candidates, so a *p*-value near 0.05 is close to what that many draws give by chance. Under a strict multiple-testing correction, only USD/CAD and GBP/USD, both at 0.0015, would clearly pass. The permutation test is therefore used here as a screen that rejects bad models, which it clearly did. It is not used as a claim that the remaining models are statistically significant.

5.8 Out-of-Sample Behaviour: The Held-Out Year

The split of Equation (4.8) trains on 2015 to 2023, selects among episodes on 2024, and does not use 2025 at all. Because the trained weights are stored, 2025 can be scored on its own.

The result is negative. Table 5.9 gives the figures. Five of the seven pairs lose money in 2025. The mean model return is −6.8%, while the mean market move would have made money. Only USD/CHF, at +5.0%, and USD/JPY, at +0.6%, are positive.

Table 5.9: The held-out year compared with the validation year and the full window. No part of the training or selection procedure saw 2025. The year 2024 was used to choose which episode's weights were kept. The last column is the move of the pair itself in 2025.

Pair	2025 (Test)	2024 (Validation)	Full Window	2025 Market
AUD/USD	−11.7%	+72.4%	+102.5%	+7.6%
EUR/USD	−1.2%	−5.9%	+73.9%	+13.5%
GBP/USD	−22.9%	−3.6%	+124.1%	+7.6%
NZD/USD	−5.0%	+11.0%	+56.4%	+2.8%
USD/CAD	−13.4%	+28.8%	+226.6%	−4.6%
USD/CHF	+5.0%	+3.7%	+50.6%	−12.8%
USD/JPY	+0.6%	+7.6%	+15.7%	−0.7%
Portfolio	−9.8%	—	+91.3%	—

The comparison with 2024 is the important part, and Figure 5.9 shows it directly. 2024 is the best year in the whole sample, with a mean excess return of +16.4 percentage points over the benchmark. 2025 is the worst, at −8.7. The difference between the two years is that 2024 was used to decide which episode's weights were kept, and 2025 was not used for anything. This gap is the measured effect of selection. It is the reason the full-window figures in Table 5.2 are called an in-sample result throughout this thesis.

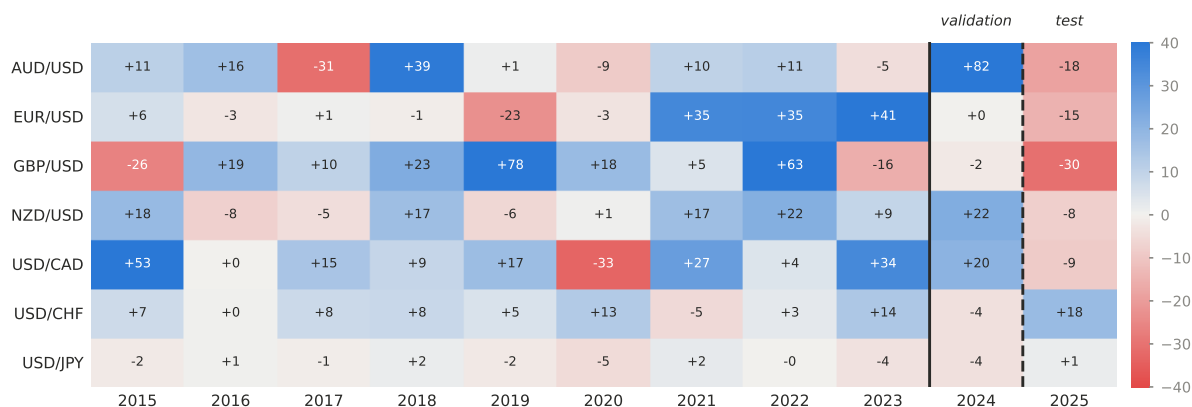


Figure 5.9: Yearly return of each agent minus the return of buying and holding the same pair, in percentage points. Blue marks outperformance of the benchmark. The solid line marks the end of the training data and the dashed line the end of the validation year, so only the last column was not seen by any part of the procedure.

There is a market explanation for the direction of the 2025 result. It is worth giving because it leads to a useful conclusion. In 2025 there was a broad dollar reversal. EUR/USD rose 13.5%, its largest yearly move in the eleven years, and the dollar became weaker against five of the seven pairs. The agents were trained on a decade in which the dollar mostly became stronger, and six of the seven were positioned for that. The seventh, USD/CHF, is the least directional model of the seven. It holds the fewest positions and has the lowest beta, and it is the one that made money. This agrees with the rest of the evidence and is not a coincidence. The model that depends least on the direction of the market is the one that held up when that direction changed.

The individual results should be read, and not only their average, because the models do not fail in the same way. GBP/USD is the worst, at −22.9%. It is the most aggressive model of the seven. It trades near full size and has the deepest drawdown over the full window, so a year that goes against it costs a lot. USD/CAD loses 13.4% and AUD/USD 11.7%. Both are leveraged-exposure models of Section 5.6, so a reversal in the direction of their beta is exactly the event that hurts them. EUR/USD loses only 1.2%, although the pair moved 13.5%. This agrees with its beta near zero: it did not take part in the reversal in either direction.

The two models that held up are the two that held the least. USD/CHF returned +5.0% while its pair

fell 12.8%. That is an excess return of 17.8 percentage points, and its best year against the benchmark in the whole sample. USD/JPY returned +0.6%. This is not skill. It only means that the model was not exposed to the pairs that moved. Together, the two show the pattern: the models that depended least on the direction of the market were the ones that held up when that direction changed.

One year is a small sample, and no conclusion should be drawn from it beyond the obvious one. The held-out year limits what can be claimed, but it is not a final judgement on the design. The design kept trading in a sensible way on data it had never seen. It lost money doing so, in a year whose direction was not in its training data.

5.9 A Negative Result: USD/JPY

USD/JPY did not work, and the reason is known.

The numbers show that the model behaves like the market. Its correlation with the pair's yearly move is +0.946 and its beta is +1.012, so its exposure to the pair is almost exactly one for one. Its alpha is -1.07% per year. It returns +15.7% over the eleven years, against roughly +25% for simply buying and holding the pair. It therefore underperforms a benchmark that costs nothing to run. The model is long at every point in the window.

The cause is in the data. USD/JPY rose from roughly 120 to 150 over the period, and the move was concentrated and mostly in one direction. On such a series, holding a long position is close to optimal, and any change from it is punished. The reward signal therefore gives no gradient towards changing the exposure, and the agent settles on the simplest policy that follows the trend. Over the same period the other six majors mean-revert often enough that trading both sides is rewarded, and their agents learned to do it.

This was not assumed. It was tested. Across seven configurations and 448 seeds, every run converged to a directional policy at full size. One augmentation shows the agent mirrored price series, and it was meant to remove exactly this bias. Raising it made the resulting policies more binary, not less.

The result is reported because it is informative. It shows the limit of the method. This design learns to time its exposure in markets that turn. On a market that does not turn, it ends up just holding the trend, and it pays trading costs to do so. A reader who is deciding whether to use the approach elsewhere needs to know this limit more than they need a seventh success.

5.10 Portfolio: The Seven Models Combined

A trader would not run just one of these agents. They would run all seven. This is a different question from how each agent does alone. For a portfolio, what matters is not only what each component returns,

but also whether the components lose money at the same time.

The seven published models are combined by giving each one a seventh of the capital at the start of the window, with no rebalancing afterwards. With $E_p(t)$ the equity of pair p at time t , the portfolio is

$$V(t) = \frac{1}{7} \sum_p \frac{E_p(t)}{E_p(0)} \quad (5.4)$$

By construction, the total return of such a portfolio is the mean of its components. The interesting part is therefore the risk, and not the return. Table 5.10 compares the portfolio with the average of the seven pairs taken separately.

Table 5.10: The equal-weight portfolio of Equation (5.4) against the average of its seven components. Return is the same by construction, so the whole effect is on risk.

Measure	Portfolio	Mean of the seven	Effect
Total return	+91.3%	+91.3%	Equal by construction
Annualised return	+6.07%	+6.07%	Equal by construction
Maximum drawdown	15.9%	34.5%	More than halved
Sharpe ratio	0.632	0.451	Above all but USD/CHF
Sortino ratio	0.906	0.685	Improved
Calmar ratio	0.381	0.194	Nearly doubled
Sterling ratio	1.059	0.670	Improved
Mean correlation	+0.062 across all 21 pairings		Close to independent

Figure 5.10 plots the portfolio against its components. The maximum drawdown falls from an average of 34.5% to 15.9%. The Sharpe ratio rises from an average of 0.451 to 0.632, which is above every single pair except USD/CHF. The portfolio gives the same return with less than half the worst loss.

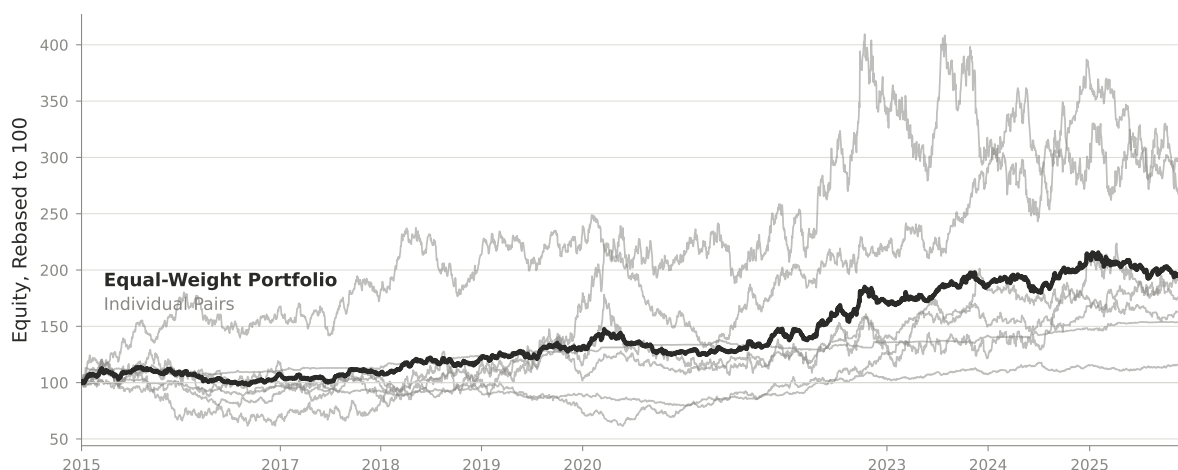


Figure 5.10: The equal-weight portfolio against its seven components, all rebased to 100. The portfolio is smoother than any component because the components do not fall together.

Figure 5.11 shows the reason. The mean correlation between the daily returns of any two of the seven models is +0.062, and no pair of models is above +0.46. Three pairings are negative. The seven

agents were trained by the same procedure on seven instruments that are themselves correlated, since all of them have the US Dollar on one side. Even so, their profits and losses come at different times. This follows from the design. Each agent learned the timing of its own market, so the agents share a method but not a position.

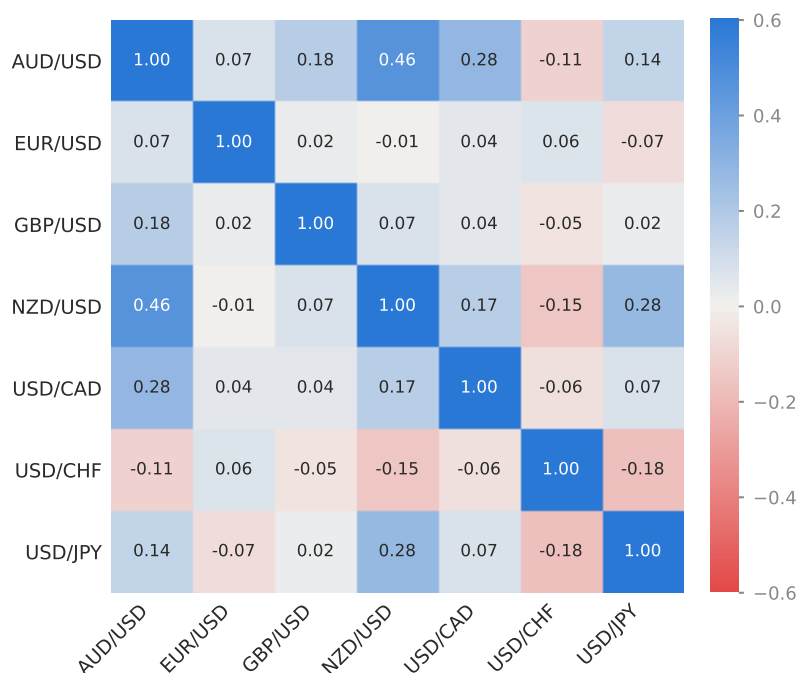


Figure 5.11: Correlation between the daily returns of the seven agents over the evaluation window. The mean of the off-diagonal entries is +0.062.

5.10.1 Does the Result Depend on the Position Sizes?

As Section 5.1 states, the seven models do not all run at the same position size. The portfolio above is therefore the combination as published, and it is not built on a common risk level. It is fair to ask whether this choice produces the result. The question can be checked directly.

Position size is proportional to the account balance. Multiplying the risk fraction by a constant therefore multiplies every position, and so every per-step return, by the same constant. A common-risk version can be built by rescaling each component's returns to the level it would have run at, and compounding them again. This ignores the rounding of volumes to the broker's step and any margin limit. It is an approximation and not a new run, but it is close enough to answer the question. Table 5.11 gives both versions.

The two portfolios are almost the same. Total return moves from +91.3% to +91.6%, the Sharpe ratio from 0.632 to 0.608, and the maximum drawdown from 15.9% to 19.0%. The common-risk version

Table 5.11: The equal-weight portfolio as published, against a version in which every component runs at a common risk level. The common-risk version is built by rescaling each component's per-step returns. This is exact for a strategy whose position size is proportional to balance, but it ignores volume rounding. The two differ little, so the diversification result does not depend on how positions are sized.

Measure	As Published	Common Risk	Difference
Total return	+91.3%	+91.6%	+0.3 pp
Annualised return	+6.07%	+6.09%	+0.02 pp
Sharpe ratio	0.632	0.608	−0.024
Sortino ratio	0.906	0.866	−0.040
Calmar ratio	0.381	0.320	−0.061
Maximum drawdown	15.9%	19.0%	+3.1 pp

is slightly worse on risk, as expected. Putting USD/JPY at the same risk level as the others makes its positions five times larger and takes its own maximum drawdown to roughly 80%. At a seventh of the capital, the portfolio absorbs even that. The conclusion of this section therefore does not depend on how the positions are sized. It depends on the correlation structure of the models, which sizing does not change.

Two limits apply to this result. As Section 5.1 states, the components do not all run at the same position size. The portfolio is therefore the combination as published, and not a risk-parity allocation. Making the sizes equal would change the weights, but not the correlation structure that produces the benefit. Also, the portfolio shares the weakness of its components. In the held-out year of Section 5.8 it returns −9.8%, better than five of the seven pairs and worse than two. This is what an average should do when most of its components fall together for a shared reason.

Two results matter most. First, six of the seven agents produced positive alpha against their own market. Three of those did so with a market sensitivity close to zero, and this is what separates a trading strategy from leveraged market exposure. Second, the seven agents combined behave better than any of them alone, because their returns are close to uncorrelated.

On the other hand, the held-out year is negative, and the selection is not corrected for multiple testing. Chapter 6 sets out what these two facts allow this work to claim and what they do not.

6

Conclusion

Contents

6.1 Summary	75
6.2 Limitations	77
6.3 Future Work	77

The aim of this work was to find out whether one deep reinforcement learning design gives a profitable policy with controlled risk on all seven major currency pairs. The design was trained separately for each instrument and was otherwise not changed. The evaluation charged the same costs that a real account charges.

The answer is yes, but with conditions. These conditions are as much a part of the result as the answer itself.

6.1 Summary

This work built three things and answered one question.

The first is a trading framework. In it, one strategy implementation runs in three places: an offline backtest over stored quotes, the cTrader backtesting tab, and a live account. Its execution engine re-

produces the platform's own results byte for byte on six reference configurations. It also goes further than the platform. It steps through every tick instead of only the bar closes, and it charges the spread recorded at the moment of each trade. The framework reads from a market database of 1.66 billion quotes for the seven pairs, which takes up 295 GB. Every bar series used in this work is built from this database.

The second is an agent. It observes thirty-eight dimensionless values, built from sixteen technical indicators. It outputs one continuous action that sets both the direction and the size of its position at the same time. Six changes to the published algorithm were needed to make it train on this problem, and two of them matter most. The first is a scale-sensitive reward. A scale-invariant reward gives no gradient towards holding a position large enough to matter. The second is a penalty on the actor's pre-activation. Without it, the policy saturates at +1 or -1 and collapses to a constant action.

The third is a selection procedure. First, gates reject candidates with degenerate policies, such as those that hardly trade or keep to one side of the market. Next, each candidate is tested against a null built from itself, by rotating the model's own sequence of positions in time. Last, the candidates are ranked by a composite score in which return is one of nine parts. The procedure was not designed to be elegant. It was built because ranking on return does not work, and Section 5.7 shows this with measured results.

The answer to the question is as follows. All seven agents are profitable over the eleven-year window, from each of the five starting balances. Their total returns range from +15.7% to +226.6% net of the spread and commission of a swap-free raw-spread account. Six of the seven have a positive alpha when their return is decomposed against their own market. Three of these six do this with almost no market exposure. The seventh, USD/JPY, only follows its market. It is reported as a failure, together with a diagnosis of its cause. An equal-weight portfolio of the seven lowers the maximum drawdown from an average of 34.5% to 15.9% and raises the Sharpe ratio to 0.632. This happens because the agents are close to uncorrelated with one another.

Two findings lie behind these numbers. They are more likely than the numbers themselves to carry over to other problems. First, the agents really use the continuous action. The mean absolute action ranges from 0.09 to 0.98 across the pairs, and saturation is below one percent on every pair. So the agents learned to size their positions, and they do not just switch between the two extremes. Second, win rates lie between 49.8% and 55.0%, and the strongest model in the set has the lowest win rate. So the profit does not come from being right more often. It comes from what the agents do with a position once it is open. A continuous action makes this possible and a discrete action does not.

6.2 Limitations

The most important limitation comes first, because it puts a limit on everything else. The performance figures in Chapter 5 are measured over a window that includes the years the agents were trained on. The procedure does keep one year out of training and out of the choice among an agent's own episodes. But the choice of which trained agent to publish was made on the full window, so the headline returns are an in-sample result. On the one year that no part of the procedure saw, the models lost money. The portfolio returned -9.8% in 2025, and five of the seven pairs were negative. That year had a broad dollar reversal, and the training decade had none. The one model that held up is the least directional model in the set. However, a single year is a small sample, and no stronger conclusion should be drawn from it in either direction.

There is a second way in which the selection is not corrected. Each published model was chosen from between 128 and 256 candidates. With that many draws, a permutation p -value near 0.05 is close to what chance alone produces. For this reason, the screening is reported as a filter that was shown to reject bad models. It is not reported as a claim that the models that passed are statistically significant. Under a strict correction, only USD/CAD and GBP/USD would clearly pass.

Three smaller limitations should also be recorded. First, the position size was chosen for each pair after the results were seen. It scales every trade and does not change the policy that makes the trades. Still, it is a parameter fitted on the same data as everything else. Second, training cannot be reproduced across environments. It is bit-exact within one installation on a single thread, but a different version of a numerical library changes the outcome. So the delivered models are archived artifacts whose replay has been verified. They are not a recipe that can produce the same models again. Third, two weeks of January 2016 are missing from all seven pairs, for reasons on the broker's side. The gap is the same for all pairs and lies outside the evaluation window, so it does not bias any comparison in this work.

Finally, the tests cover a narrower scope than the method itself. They use seven instruments from one asset class, one eleven-year period, one decision frequency and one cost model. This is only one case out of many possible ones. Nothing in this work shows that the design carries over to other markets, other periods or other holding horizons. The USD/JPY result shows that it does not carry over to every instrument, even among the seven.

6.3 Future Work

The limitations above set the order of the future work. The most useful next step is the one that would most change the value of the results.

The first step is a true held-out evaluation. Training would end before the reporting period starts. The choice among candidates would use only data from before that point. The reported figures would then

be an out-of-sample estimate. They would no longer describe the same window on which the models were chosen. Of all possible changes, this one would do the most to make the claims stronger. The framework already supports it. It needs a campaign run under this stricter protocol, and it needs no new features in the framework.

The second step follows from the first. The procedure used here splits the window only once. A walk-forward scheme with several folds would train each fold on the data before it and evaluate it on the data after it. This would give an out-of-sample record for the whole period, and not just for one year. The learned weights could also be carried from one fold into the next, instead of starting each fold from a new initialisation. This would test whether an agent adapts as the market changes. The framework supports this, but it was not used in this campaign.

The third step is to report how much the results vary between training runs. The path-robustness protocol used here changes the starting balance. This measures how sensitive a result is to the entry path, but not how sensitive it is to the training itself. The next step would report the distribution of outcomes across the whole pool of candidates, and where the published model falls within it. A reader could then see how much of a result comes from the method and how much comes from chance.

Apart from the evaluation, what the agents did suggests three further directions. The portfolio in Section 5.10 gives the seven agents equal weights. Weighting them by risk, or by their correlation structure, should do better. The seven agents are almost independent of each other, which suggests that the gain would be real. The observation holds thirty-eight values from a single point in time. The natural next step is a recurrent or attention-based encoder that reads a window of past data directly. This was part of the original plan for this work. Last, the framework already runs the same strategy on a live account. A live deployment over a long period would test whether the execution assumptions hold when the orders are real. No backtest can test this.

One improvement to the framework itself should also be noted. The execution engine charges the spread correctly, by filling orders at the ask and at the bid. However, the exported trade record does not show this cost as a separate field. To recover it, the indirect calculation of Equation (5.1) is needed. If every trade recorded the spread cost next to its commission and swap, the cost decomposition in Table 5.5 could be read directly and would not have to be inferred. It could also be reported for each position, and not only in total.

The USD/JPY result needs a separate note. The current explanation of the failure on this pair is that the price window is monotonic, which means the price moves in one direction only. Such a window gives no gradient towards changing the size of the exposure. It has not been shown that this is the whole explanation. The pair should be studied again once the evaluation protocol above is in place.

Some reported returns suggest that deep reinforcement learning for currency trading is a solved problem. Its out-of-sample record suggests that it leads nowhere. Neither view is correct. This work

shows that the main difficulty is not in the learning algorithm. It is in the environment the agent is trained in and in the costs it is charged. Most of all, it is in a careful and strict way of deciding which of many trained models to trust. Six of the seven agents produced returns that still hold after a decomposition into policy and market exposure. For all seven pairs, the candidate that looked best on return did not pass this test.

Bibliography

- [1] R. Teo, "Candlestick Pattern OHLC." [Online]. Available: <https://www.tradingwithrayner.com/wp-content/uploads/2019/04/candlestick-pattern-ohlc.png>
- [2] R. Pramoditha, "Human Neuron vs Artificial Neuron." [Online]. Available: https://miro.medium.com/v2/resize:fit:2902/format:webp/1*hkYITODpjJgo32DoCOWN5w.png
- [3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. MIT Press, Nov. 2018, google-Books-ID: sWV0DwAAQBAJ.
- [4] P. Brandimarte, *An Introduction to Financial Markets: A Quantitative Approach*. John Wiley & Sons, Feb. 2018, google-Books-ID: _2tODwAAQBAJ.
- [5] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, Nov. 2016, google-Books-ID: omivDQAAQBAJ.
- [6] T. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *CoRR*, Sep. 2015. [Online]. Available: <https://www.semanticscholar.org/paper/Continuous-control-with-deep-reinforcement-learning-Lillicrap-Hunt/024006d4c2a89f7acacc6e4438d156525b60a98f>
- [7] S. Jansen, *Machine Learning for Algorithmic Trading*. Packt Publishing Ltd, Jul. 2020, google-Books-ID: 4f30DwAAQBAJ.
- [8] BIS, "OTC foreign exchange turnover in April 2025," Sep. 2025. [Online]. Available: <https://www.bis.org/publications/202509-commentary-otc-derivatives>
- [9] E. F. Fama and B. G. Malkiel, "EFFICIENT CAPITAL MARKETS: A REVIEW OF THEORY AND EMPIRICAL WORK*," *The Journal of Finance*, vol. 25, no. 2, pp. 383–417, May 1970. [Online]. Available: <https://onlinelibrary.wiley.com/doi/10.1111/j.1540-6261.1970.tb00518.x>
- [10] E. P. Chan, *Quantitative Trading: How to Build Your Own Algorithmic Trading Business*. John Wiley & Sons, Jun. 2021, google-Books-ID: pWE0EAAAQBAJ.

- [11] B. Donnelly, *The Art of Currency Trading: A Professional's Guide to the Foreign Exchange Market*. John Wiley & Sons, Jun. 2019, google-Books-ID: r8ybDwAAQBAJ.
- [12] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015. [Online]. Available: <https://www.nature.com/articles/nature14236>
- [13] T. G. Fischer, "Reinforcement learning in financial markets - a survey," *Friedrich-Alexander-Universität Erlangen-Nürnberg, Institute for Economics*, 2018. [Online]. Available: <https://www.semanticscholar.org/paper/Reinforcement-learning-in-financial-markets-a-Fischer/922864ede84bc49be4ac676951278a9b568b6383>
- [14] B. Hambly, R. Xu, and H. Yang, "Recent advances in reinforcement learning in finance," *Mathematical Finance*, vol. 33, no. 3, pp. 437–503, Jul. 2023. [Online]. Available: <https://onlinelibrary.wiley.com/doi/10.1111/mafi.12382>
- [15] T.-V. Pricope, "Deep Reinforcement Learning in Quantitative Algorithmic Trading: A Review," *ArXiv*, May 2021. [Online]. Available: <https://www.semanticscholar.org/paper/Deep-Reinforcement-Learning-in-Quantitative-A-Pricope/a6a7e7be402fc5f7703e5c74a7b2afa84613a429>
- [16] M. M. López de Prado, *Advances in financial machine learning*. Hoboken, New Jersey: Wiley, 2018.
- [17] A. Alameer, H. Saleh, and K. Alshehri, "Reinforcement Learning in Quantitative Trading: A Survey," *TechRxiv*, Mar. 2022. [Online]. Available: <https://www.techrxiv.org/doi/full/10.36227/techrxiv.19303853.v1>
- [18] J. Carapuço, R. Neves, and N. Horta, "Reinforcement learning applied to Forex trading," *Applied Soft Computing*, vol. 73, pp. 783–794, Dec. 2018. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1568494618305349>
- [19] C.-Y. Huang, "Financial Trading as a Game: A Deep Reinforcement Learning Approach," *ArXiv*, Jul. 2018. [Online]. Available: <https://www.semanticscholar.org/paper/Financial-Trading-as-a-Game%3A-A-Deep-Reinforcement-Huang/a13a070940d48bce111f31ba9c0233884a17ae62>
- [20] F. Rundo, "Deep LSTM with Reinforcement Learning Layer for Financial Trend Prediction in FX High Frequency Trading Systems," *Applied Sciences*, vol. 9, no. 20, p. 4460, Oct. 2019. [Online]. Available: <https://www.mdpi.com/2076-3417/9/20/4460>

- [21] D. H. Bailey, J. M. Borwein, M. López De Prado, and Q. J. Zhu, “Pseudo-Mathematics and Financial Charlatanism: The Effects of Backtest Overfitting on Out-of-Sample Performance,” *Notices of the American Mathematical Society*, vol. 61, no. 5, p. 458, May 2014. [Online]. Available: <https://www.ams.org/jourcgi/jour-getitem?pii=noti1105>
- [22] “Forex & Algorithmic Trading Community | cTrader Community.” [Online]. Available: <https://ctrader.com/>
- [23] W. F. Sharpe, “The Sharpe Ratio,” *The Journal of Portfolio Management*, vol. 21, no. 1, pp. 49–58, Oct. 1994. [Online]. Available: <http://pm-research.com/lookup/doi/10.3905/jpm.1994.409501>
- [24] F. A. Sortino and L. N. Price, “Performance Measurement in a Downside Risk Framework,” *The Journal of Investing*, vol. 3, no. 3, pp. 59–64, Aug. 1994. [Online]. Available: <http://pm-research.com/lookup/doi/10.3905/joi.3.3.59>
- [25] C. R. Bacon, *Practical portfolio performance measurement and attribution*, third edition ed., ser. Wiley finance. Hoboken, NJ: Wiley, 2023.
- [26] R. Rhoads, *Candlestick Charting For Dummies*. John Wiley & Sons, Jun. 2022, google-Books-ID: 92l6EAAAQBAJ.
- [27] M. A. Lim, *The Handbook of Technical Analysis + Test Bank: The Practitioner’s Comprehensive Guide to Technical Analysis*. John Wiley & Sons, Dec. 2015, google-Books-ID: N01_BwAAQBAJ.
- [28] T. M. Mitchell, *Machine Learning*. McGraw-Hill, 1997, google-Books-ID: EoYBngEACAAJ.
- [29] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958. [Online]. Available: <https://doi.apa.org/doi/10.1037/h0042519>
- [30] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, no. 6088, pp. 533–536, Oct. 1986. [Online]. Available: <https://www.nature.com/articles/323533a0>
- [31] K. Hornik, M. Stinchcombe, and H. White, “Multilayer feedforward networks are universal approximators,” *Neural Networks*, vol. 2, no. 5, pp. 359–366, Jan. 1989. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/0893608089900208>
- [32] B. F. Skinner, “Operant behavior,” *American Psychologist*, vol. 18, no. 8, pp. 503–515, 1963, place: US Publisher: American Psychological Association.

- [33] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. A. Riedmiller, "Playing Atari with Deep Reinforcement Learning," *ArXiv*, Dec. 2013. [Online]. Available: <https://www.semanticscholar.org/paper/Playing-Atari-with-Deep-Reinforcement-Learning-Mnih-Kavukcuoglu/2319a491378867c7049b3da055c5df60e1671158>
- [34] M. Riedmiller, "Neural Fitted Q Iteration – First Experiences with a Data Efficient Neural Reinforcement Learning Method," *Machine Learning: ECML 2005*, vol. 3720, pp. 317–328, 2005, series Title: Lecture Notes in Computer Science. [Online]. Available: http://link.springer.com/10.1007/11564096_32
- [35] H. Van Hasselt, A. Guez, and D. Silver, "Deep Reinforcement Learning with Double Q-Learning," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, Mar. 2016, iSSN: 2374-3468, 2159-5399 Issue: 1 Journal Abbreviation: AAAI. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/10295>
- [36] T. L. Meng and M. Khushi, "Reinforcement Learning in Financial Markets," *Data*, vol. 4, no. 3, p. 110, Jul. 2019. [Online]. Available: <https://www.mdpi.com/2306-5729/4/3/110>
- [37] J. Sim and B. Kirk, "Generalized Deep Reinforcement Learning for Trading," *Journal of Student Research*, vol. 12, no. 1, Feb. 2023. [Online]. Available: <https://www.jsr.org/hs/index.php/path/article/view/4316>
- [38] T. Théate and D. Ernst, "An application of deep reinforcement learning to algorithmic trading," *Expert Systems with Applications*, vol. 173, p. 114632, Jul. 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0957417421000737>
- [39] Y. Li, W. Zheng, and Z. Zheng, "Deep Robust Reinforcement Learning for Practical Algorithmic Trading," *IEEE Access*, vol. 7, pp. 108 014–108 022, 2019. [Online]. Available: <https://ieeexplore.ieee.org/document/8786132/>
- [40] Z. Xiong, X.-Y. Liu, S. Zhong, H. Yang, and A. Elwalid, "Practical Deep Reinforcement Learning Approach for Stock Trading," *ArXiv*, Nov. 2018. [Online]. Available: <https://www.semanticscholar.org/paper/Practical-Deep-Reinforcement-Learning-Approach-for-Xiong-Liu/412fbb7ffa29aa6b2e172c1f545223baf18f1b6e>
- [41] N. Yousefi, "Deep Reinforcement Learning for Tehran Stock Trading," *Journal of Novel Engineering Science and Technology*, vol. 1, no. 02, pp. 37–42, Nov. 2022. [Online]. Available: <https://journal.iistr.org/index.php/JNEST/article/view/171>

- [42] P. K. Gran, A. J. K. Holm, and S. Søgård, "A Deep Reinforcement Learning Approach to Stock Trading," *NTNU*, 2019. [Online]. Available: <https://www.semanticscholar.org/paper/A-Deep-Reinforcement-Learning-Approach-to-Stock-Gran-Holm/ef9f709e845d282cfb9371708d05d61580d1e10c>
- [43] A. R. Azhikodan, A. G. K. Bhat, and M. V. Jadhav, "Stock Trading Bot Using Deep Reinforcement Learning," *Springer*, vol. 32, pp. 41–49, 2019, book Title: Innovations in Computer Science and Engineering Series Title: Lecture Notes in Networks and Systems. [Online]. Available: http://link.springer.com/10.1007/978-981-10-8201-6_5
- [44] N. Majidi, M. Shamsi, and F. Marvasti, "Algorithmic Trading Using Continuous Action Space Deep Reinforcement Learning," *Expert Systems with Applications*, 2022, publisher: arXiv Version Number: 1. [Online]. Available: <https://arxiv.org/abs/2210.03469>
- [45] H. K. Sagiraju and S. Mogalla, "Application of multilayer perceptron to deep reinforcement learning for stock market trading and analysis," *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 24, no. 3, p. 1759, Dec. 2021. [Online]. Available: <http://ijeecs.iaescore.com/index.php/IJECS/article/view/24816>
- [46] H. Yang, X.-Y. Liu, S. Zhong, and A. Walid, "Deep reinforcement learning for automated stock trading: an ensemble strategy," *Proceedings of the First ACM International Conference on AI in Finance*, pp. 1–8, Oct. 2020, conference Name: ICAIF '20: ACM International Conference on AI in Finance ISBN: 9781450375849 Place: New York New York Publisher: ACM. [Online]. Available: <https://dl.acm.org/doi/10.1145/3383455.3422540>
- [47] M. Kong and J. So, "Empirical Analysis of Automated Stock Trading Using Deep Reinforcement Learning," *Applied Sciences*, vol. 13, no. 1, p. 633, Jan. 2023. [Online]. Available: <https://www.mdpi.com/2076-3417/13/1/633>
- [48] X. Wu, H. Chen, J. Wang, L. Troiano, V. Loia, and H. Fujita, "Adaptive stock trading strategies with deep reinforcement learning methods," *Information Sciences*, vol. 538, pp. 142–158, Oct. 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0020025520304692>
- [49] L. Chen and Q. Gao, "Application of Deep Reinforcement Learning on Automated Stock Trading," *2019 IEEE 10th International Conference on Software Engineering and Service Science (ICSESS)*, pp. 29–33, Oct. 2019, conference Name: 2019 IEEE 10th International Conference on Software Engineering and Service Science (ICSESS) ISBN: 9781728109459 Place: Beijing, China Publisher: IEEE. [Online]. Available: <https://ieeexplore.ieee.org/document/9040728/>
- [50] Z. Zhang, S. Zohren, and S. Roberts, "Deep Reinforcement Learning for Trading," *The Journal of Financial Data Science*, vol. 2, pp. 25–40, Apr. 2020, iSSN: 2640-3943 Issue: 2 Journal

Abbreviation: JFDS. [Online]. Available: <http://pm-research.com/lookup/doi/10.3905/jfds.2020.1.030>

- [51] Z. Jia, Q. Gao, and X. Peng, "LSTM-DDPG for Trading with Variable Positions," *Sensors*, vol. 21, no. 19, p. 6571, Sep. 2021. [Online]. Available: <https://www.mdpi.com/1424-8220/21/19/6571>



Technical Indicators

Table A.1: Summary of frequently used Trend Technical Indicators [7].

Indicator Name	Formula	Range
Simple Moving Average	$SMA(N)_t = \frac{1}{N} \sum_{i=1}^N Price_{t-N+i}$	-
Exponential Moving Average	$EMA(N)_t = \frac{2}{N+1} \times Price_t + (1 - \frac{2}{N+1}) \times EMA(N)_{t-1}$	-
Weighted Moving Average	$WMA(N)_t = \frac{2}{N \times (N+1)} \sum_{i=1}^N i \times Price_{t-N+i}$	-
Double Exponential Moving Average	$DEMA(N)_t = 2 \times EMA(N)_t - EMA_2(N)_t$	-
Triple Exponential Moving Average	$TEMA(N)_t = 3 \times [EMA(N)_t - EMA_2(N)_t] + EMA_3(N)_t$	-
Hull Moving Average	$HMA(N)_t = WMA\left(\sqrt{N}\right) \left[2 \times WMA\left(\frac{N}{2}\right)_t - WMA(N)_t \right]$	-
Triangular Moving Average	$TRIMA(N)_t = \frac{1}{N} \sum_{i=1}^N SMA(N)_{t-N+i}$	-
Efficiency Ratio	$ER(N)_t = \frac{ Price_t - Price_{t-N} }{\sum_{i=1}^N Price_{t-i+1} - Price_{t-i} }$	[0, 1]
Kaufman Adaptive Moving Average	$KAMA(N)_t = KAMA(N)_{t-1} + SC \times (Price_t - KAMA(N)_{t-1})$ $SC = [ER(N)_t \times (\phi_{short} - \phi_{long}) + \phi_{long}]^2, \phi_{short} = \frac{2}{N_{short}+1}, \phi_{long} = \frac{2}{N_{long}+1}$ where typically $N_{short} = 2$ and $N_{long} = 30$	-

Table A.2: Summary of frequently used Momentum Technical Indicators [7].

Indicator Name	Formula	Range
Price Momentum	$PM_t = \frac{Price_t - Price_{t-1}}{Price_{t-1}}$	-
Rate of Change	$ROC(N)_t = \ln \left(\frac{Price_t}{Price_{t-N}} \right)$	-
Price Acceleration	$PA_t = \frac{\text{Short Linear Regression Coefficient}}{\text{Long Linear Regression Coefficient}} = \frac{\beta_{\text{short}}}{\beta_{\text{long}}}$	-
Relative Strength Index	$RSI(N)_t = 100 - \frac{100}{1 + \frac{\text{Average Gain}_t}{\text{Average Loss}_t}}$ $\text{Average Gain}_t = \frac{1}{N} \sum_{i=1}^N \max(Price_i - Price_{i-1}, 0)$ $\text{Average Loss}_t = \frac{1}{N} \sum_{i=1}^N \max(Price_{i-1} - Price_i, 0)$	[0, 100]
Aroon Oscillator	$Aroon(N)_t = Aroon\ Up(N)_t - Aroon\ Down(N)_t$ $Aroon\ Up(N)_t = \left(\frac{N - \text{Periods Since Highest High in } N}{N} \right) \times 100$ $Aroon\ Down(N)_t = \left(\frac{N - \text{Periods Since Lowest Low in } N}{N} \right) \times 100$	[-100, 100]
Balance of Power	$BOP_t = \frac{Close_t - Open_t}{High_t - Low_t}$	[-1, 1]
Commodity Channel Index	$CCI(N)_t = \frac{TP_t - SMA_{TP}(N)_t}{0.015 \times \text{Mean Deviation}(N)_t}$ $TP_t = \frac{High_t + Low_t + Close_t}{3}$ $\text{Mean Deviation}(N)_t = \frac{1}{N} \sum_{i=1}^N TP_i - SMA_{TP}(N)_t $	-
Moving Average Convergence Divergence	$MACD_t = EMA(N_{\text{short}})_t - EMA(N_{\text{long}})_t$ where typically $N_{\text{short}} = 12$ and $N_{\text{long}} = 26$	-
Stochastic RSI	$StochRSI(N)_t = \frac{RSI(N)_t - \min(RSI(N))}{\max(RSI(N)) - \min(RSI(N))}$	[0, 1]
Stochastic Oscillator	$Stoch\ \%K(N)_t = \frac{Close_t - LL(N)_t}{HH(N)_t - LL(N)_t} \times 100$ $Stoch\ \%D(M)_t = SMA_{Stoch\ \%K(N)}(M)_t$ $HH(N)_t = \max_{i=t-N+1}^t (High_i)$, $LL(N)_t = \min_{i=t-N+1}^t (Low_i)$	[0, 100]
Ultimate Oscillator	$UO_t = \frac{4 \times \text{Avg}_t(N_1) + 2 \times \text{Avg}_t(N_2) + \text{Avg}_t(N_3)}{4 + 2 + 1} \times 100$ $\text{Avg}_t(N) = \frac{\sum_{i=0}^{N-1} BP_{t-i}}{\sum_{i=0}^{N-1} TR_{t-i}}$ $BP_t = Close_t - \min(Close_{t-1}, Low_t)$ $TR_t = \max(Close_{t-1}, High_t) - \min(Close_{t-1}, Low_t)$ where typically $N_1 = 7$, $N_2 = 14$, and $N_3 = 28$	[0, 100]
Williams %R	$Williams\ \%R(N)_t = \frac{HH(N)_t - Close_t}{HH(N)_t - LL(N)_t} \times -100$	[-100, 0]

Table A.3: Summary of frequently used Volume Technical Indicators [7].

Indicator Name	Formula	Range
Chaikin Accumulation/Distribution Line	$AD_t = AD_{t-1} + MFV_t$ $MFV_t = \frac{(Close_t - Low_t) - (High_t - Close_t)}{High_t - Low_t} \times Volume_t$	-
Chaikin Accumulation/Distribution Oscillator	$ADOSC_t = EMA_{AD}(N_{\text{short}})_t - EMA_{AD}(N_{\text{long}})_t$ where typically $N_{\text{short}} = 3$ and $N_{\text{long}} = 10$	-
On-Balance Volume	$OBV_t = \begin{cases} OBV_{t-1} + Volume_t & \text{if } Price_t > Price_{t-1} \\ OBV_{t-1} - Volume_t & \text{if } Price_t < Price_{t-1} \\ OBV_{t-1} & \text{otherwise} \end{cases}$	-

Table A.4: Summary of frequently used Volatility Technical Indicators [7].

Indicator Name	Formula	Range
Standard Deviation	$SD(N)_t = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (\text{Price}_i - \overline{\text{Price}})^2}$	-
Realised Volatility	$RV(N)_t = \sqrt{\sigma_t^2}$, where $\sigma_t^2 = \alpha \times r_t^2 + (1 - \alpha) \times \sigma_{t-1}^2$, $r_t = \ln\left(\frac{\text{Price}_t}{\text{Price}_{t-1}}\right)$ and $\alpha = \frac{1}{N}$	-
Bollinger Bands	Upper $BB(N, k)_t = SMA(N)_t + k \times SD(N)_t$ Lower $BB(N, k)_t = SMA(N)_t - k \times SD(N)_t$ where typically $k = 2$	-
Average True Range	$ATR(N)_t = \frac{(N-1) \times ATR(N)_{t-1} + TRANGE_t}{N}$ $TRANGE_t = \max(\text{High}_t - \text{Low}_t, \text{High}_t - \text{Close}_{t-1} , \text{Low}_t - \text{Close}_{t-1})$	-
Normalised Average True Range	$NATR(N)_t = \frac{ATR(N)_t}{\text{Close}_t} \times 100$	-
Chaikin Volatility Index	$CVI(N)_t = \frac{EMA_{HL}(N)_t - EMA_{HL}(N)_{t-N}}{EMA_{HL}(N)_{t-N}} \times 100$ $EMA_{HL}(N)_t = EMA(N)$ applied to $(\text{High}_t - \text{Low}_t)$	-

B

Reviewed Articles

Table B.1: Basic features of the related work articles.

Reference	Market	Dataset	Timeframe	Period	NN	RL	DRL	Key Findings
[18]	Forex	Raw Ticks	≈ 2 Hours	2010 - 2017	FNN	Critic-Only	DQN	Significantly outperformed the benchmark.
[19]	Forex	OHLCV	15 Minutes	2012 - 2017	RNN (LSTM)	Critic-Only	DRQN	Significantly outperformed the benchmark even with explicit and implicit costs.
[20]	Forex	OHLCV	Minute	2004 - 2018	RNN (LSTM)	Critic-Only	DRQN	Outperformed the benchmark, but with unusually high accuracy and maximum drawdown.
[43]	Stocks	OHLCV, News	Daily	≈ 5 Months	RNN (LSTM)	Actor-Critic	DDPG	Outperformed the benchmark. News features were important.
[42]	Stocks	OHLCV	Monthly	2005 - 2019	FNN	Actor-Critic	DDPG	Outperformed the benchmark even with explicit and implicit costs.
[47]	Stocks	OHLCV	Daily	2016 - 2020	FNN	Actor-Critic	DDPG, A2C, PPO	The ensemble underperformed the benchmark and had unstable returns.
[39]	Stocks, Futures	OHLCV	Minute	2010 - 2020	FNN (SDAE), RNN (LSTM)	Critic-Only, Actor-Critic	DQN, A3C	Both significantly outperformed the benchmark. A3C had better results and convergence.
[44]	Stocks, Cryptos	OHLCV	Daily	2010 - 2021	FNN	Actor-Critic	DDPG (TD3)	Outperformed the crypto benchmark but underperformed the stock benchmark.
[45]	Stocks	OHLCV	Daily	1992 - 2020	FNN	Critic-Only, Actor-Critic	DQN, DDPG, PPO, A2C	All outperformed the benchmark. PPO and DQN had better and more consistent results. DDPG and A2C did worse than expected.
[37]	Stocks	OHLCV	Daily	2006 - 2022	FNN	Critic-Only	DQN	Outperformed the benchmark and generalised well to out-of-sample assets.
[38]	Stocks	OHLCV	Daily	2012 - 2019	FNN	Critic-Only	DQN	Outperformed the benchmark with consistent results in stable and volatile markets.
[48]	Stocks	OHLCV	Daily	2008 - 2018	RNN (GRU)	Critic-Only, Actor-Critic	GDQN, GDPG	Both outperformed the benchmark with consistent results in stable and volatile markets. GDPG had better results.
[40]	Stocks	OHLCV	Daily	2009 - 2018	FNN	Actor-Critic	DDPG	Outperformed the benchmark.
[46]	Stocks	OHLCV	Daily	2009 - 2020	FNN	Actor-Critic	DDPG, A2C, PPO	The ensemble significantly outperformed the benchmark with explicit costs. After it, PPO had the best results, then A2C, then DDPG.
[41]	Stocks	OHLCV	Daily	2009 - 2021	FNN	Actor-Critic	DDPG, A2C	Both outperformed the benchmark. DDPG had better results, convergence and stability, due to experience replay and noise.
[49]	ETFs	OHLCV	Daily	2000 - 2018	FNN, RNN (LSTM)	Critic-Only	DQN, DRQN	Both outperformed the benchmark. DRQN had better results.
[51]	ETFs	OHLCV	Daily	2002 - 2020	RNN (LSTM)	Actor-Critic	DDPG	Outperformed the benchmark with variable position sizing, with explicit and implicit costs charged.
[50]	Futures	OHLCV	Daily	2005 - 2019	RNN (LSTM)	Actor-Critic	A2C	Outperformed the benchmark with explicit and implicit costs.

Table B.2: Advanced features of the related work articles.

Reference	State Space	Action Space	Reward Function	Bias/Variance	Exploration/Exploitation	Training Techniques
[18]	Ask, Bid, Volume, Technical Analysis	Discrete (1, 0, -1)	Total Return	Adjust Network, Early Stopping	Policy ϵ -greedy	Cross-Validation
[19]	OHLCV, Technical Analysis	Discrete (1, 0, -1)	Total Return	Adjust Replay Memory, Adjust RNN Sequence	Policy ϵ -greedy	Online Learning
[20]	OHLCV	Discrete (1, 0, -1)	Total Return	Adjust Network	-	Train-Validate-Test Split
[43]	OHLCV, Sentiment and Technical Analysis	Discrete (1, 0, -1)	Total Return	Adjust Network	Policy ϵ -greedy, Shuffling	Cross-Validation
[42]	Log Returns, Volume	Discrete (1, 0, -1)	Total Return	-	-	Pre-Train with GA
[47]	OHLCV, Technical Analysis	Discrete (1, 0, -1)	Sharpe Ratio	Ensemble Selection	Ensemble Selection	Cross-Validation
[39]	OHLCV, Metrics, Technical Analysis	Discrete (-n, ..., 0, ..., n)	Total Return	Denoising	Gaussian Noise	Adaptive Learning Rate
[44]	OHLCV	Continuous [-1, 1]	Log-Return	Normalisation, Regularisation	Policy ϵ -greedy Gaussian Noise	Train-Validate-Test Split
[45]	OHLCV	Discrete (1, 0, -1)	Sharpe Ratio	Wavelet Transform	-	Train-Validate-Test Split
[37]	OHLCV, Macroeconomic Metrics	Discrete (1, 0, -1)	Total Return, (1, 0, -1)	Normalisation, Regularisation	Policy ϵ -greedy (Adaptive)	Adaptive Synchronisation
[38]	OHLCV	Discrete (1, -1)	Total Return	Normalisation, Regularisation	Duplicate Environment (Inverse Positioning)	Adaptive Learning Rate, Huber Loss, Gradient Clip
[48]	OHLCV, Technical Analysis	Discrete (1, 0, -1)	Total Return	Normalisation, Regularisation	Policy ϵ -greedy Gaussian Noise	Train-Test Split
[40]	OHLCV	Discrete (1, 0, -1)	Total Return	Adjust Network	Gaussian Noise	Train-Validate-Test Split
[46]	OHLCV, Technical Analysis	Discrete (-n, ..., 0, ..., n)	Total Return (Turbulence Index)	Ensemble Selection	Ensemble Selection	Train-Validate-Test Split
[41]	OHLCV, Technical Analysis	Discrete (1, 0, -1)	Total Return	Normalisation	Gaussian Noise	Train-Test Split
[49]	OHLCV	Discrete (1, 0, -1)	Total Return	Adjust Learning Rate	-	-
[51]	OHLCV	Continuous (Position Sizing)	Sharpe Ratio	-	Gaussian Noise	Train-Validate-Test Split
[50]	OHLCV, Technical Analysis	Continuous [-1, 1]	Total Return (with Volatility)	Normalisation	-	Cross-Validation